



VILNIAUS UNIVERSITETAS
MATEMATIKOS IR INFORMATIKOS FAKULTETAS
STUDIJŲ PROGRAMA: INFORMATIKA

**Daugiaagentinių debatų įtaka lokalių didelių kalbos modelių
(LLM) kodo generavimo efektyvumui skirtingo sudėtingumo
užduotyse**

**The Impact of Multi-Agent Debate on the Code Generation
Effectiveness of Local Large Language Models (LLMs) Across
Varying Task Complexities**

Baigiamasis bakalauro darbas

Atliko: Lukaš Patrik Magalinski

VU el. p.: lukas.magalinski@mif.stud.vu.lt

Vadovas: Prof. Dr. Aistis Raudys

Vilnius
2026

Turinys

Ivadas	4
1. Kelių agentų debatai ir kodo generavimo kontekstas	6
1.1. Didieji kalbos modeliai kodo generavime ir jų ribotumai	6
1.2. Kelių agentų debatų (MAD) metodologija	6
1.3. „LLM-as-Judge“ paradigma	7
1.4. Susiję darbai ir tyrimo niša.....	8
2. Debatų sistemos projektavimas ir realizacija	10
2.1. Sistemos architektūra ir technologijų pasirinkimas.....	10
2.1.1. Bendras informacijos srautas ir viešojo programavimo sąsaja	10
2.1.2. Modulinis paketų skirstymas.....	11
2.1.3. Technologijų pasirinkimo pagrindimas.....	12
2.1.4. Inžinerinis iššūkis: konteksto lango parinkimas.....	12
2.1.5. Eksploatacijos optimizacijos HPC aplinkoje.....	13
2.2. Pagrindinis debatų ciklas	14
2.2.1. Bendroji ciklo struktūra.....	14
2.2.2. Pradinio pasiūlymo fazė	14
2.2.3. Kritikos fazė	15
2.2.4. Revizijos fazė	15
2.2.5. Balsavimo fazė	16
2.2.6. Lygiagretus vykdymas ir matomumo lygiai.....	16
2.2.7. Neprivalomas teisėjo vaidmuo	16
2.3. Konsensuso ir balsavimo algoritmas.....	17
2.3.1. Balsavimo užklausos struktūra ir atsakymo išskyrimas	17
2.3.2. Svertinis balsų skaičiavimas ir kokybės korekcijos	18
2.3.3. Konsensuso aptikimo kriterijai.....	18
2.3.4. Ankstyvos pabaigos sąlygos.....	19
2.4. Atsakymų kokybės kontrolės ir revizijos stabilizavimo mechanizmai	20
2.4.1. Anti-regresijos politika ir geriausio istorinio sprendimo grąžinimas	20
2.4.2. Modelių anomalijų aptikimas: sutrumpinimas, „tinginio“ kodas ir tapatus atsakymas	21
2.4.3. Adaptyvi temperatūra ir kritikos istorija	23
2.4.4. Įvairios revizijos strategijos.....	23
2.5. Daugiafailių užduočių apdorojimas ir dalinis generavimas	24
2.5.1. Daugiafailių užduočių struktūra	24
2.5.2. Dalinio generavimo schema	25
2.5.3. Atsakymo kodo išskyrimas iš modelio atsakymo	26
2.6. Užklausų konstravimas ir DKM atsakymų išskyrimas.....	26
2.6.1. Sistemos užklausos: trijų vaidmenų atskyrimas	26
2.6.2. Vartotojo užklausų konstravimas	27
2.6.3. Kodo išskyrimas iš modelio atsakymo	28
2.6.4. Kritikos atsakymų išskyrimas ir įvertinimų analizė	28
2.7. Užduočių vykdymas izoliuotoje aplinkoje ir kodo kokybės analizė.....	29
2.7.1. Izoliuoto vykdymo aplinka.....	29
2.7.2. Sprendimų sanitizacija prieš vykdymą.....	30
2.7.3. Rezultatų podėliavimas	30
2.7.4. Kodo kokybės analizė: „Pylint“ ir „Radon“	31
2.8. Žiniatinklio sąsaja interaktyviam tyrimui.....	32
2.8.1. Architektūros apžvalga ir technologijų rinkinys	32
2.8.2. REST API endpoints	32
2.8.3. Realio laiko įvykių srautas per „WebSocket“	33
2.8.4. Atsparumas atsijungimui: paskutinės būsenos perdavimas.....	34
2.9. Duomenų saugojimas ir analizės srautas.....	34
2.9.1. Reliacinė schema: penkios ORM lentelės	35
2.9.2. „SQLite“ konfigūracija ir gijų saugumas	35
2.9.3. CSV eksportas eksperimentinei analizei	36
2.9.4. Metrikų rinkimas ir vaizdavimas.....	36
3. Tyrimo metodika	38
3.1. Tyrimo logika ir palyginimo schema	38
3.2. Tiriamų konfigūracijų parinkimas	40

3.3. Užduočių rinkinio sudarymas.....	42
3.4. Eksperimentų scenarijai ir vykdymo sąlygos.....	45
3.4.1. Žiniatinklio sąsaja preliminariniams bandymams	45
3.4.2. Didelio našumo skaičiavimo (HPC) aplinka	45
3.5. Vertinimo metrikos.....	46
4. Eksperimentų rezultatai ir jų analizė	48
4.1. Solo bazinės linijos rezultatai.....	48
4.2. Debatų ir solo režimo palyginimas	51
4.3. Agentų rinkinio sudėties įtaka: Pool A (7B) ir Pool B (9B) palyginimas.....	58
4.4. Teisėjo režimo įtaka: be teisėjo, kodo specializacijos ir samprotavimo specializacijos teisėjai	62
4.5. Debatų raidos analizė	66
4.6. Kokybinė debatų analizė: tipiški scenarijai	70
4.7. Skaičiavimo sąnaudos ir statinė kodo kokybė.....	73
4.8. Rezultatų aptarimas literatūros kontekste	77
<i>Išvados ir pasiūlymai.....</i>	<i>81</i>
<i>Reziumė (Summary).....</i>	<i>85</i>
<i>Literatūros sąrašas</i>	<i>87</i>
<i>Priedai.....</i>	<i>90</i>

Įvadas

Didieji kalbos modeliai sparčiai keičia programinės įrangos kūrimo procesą, ypač automatinio kodo generavimo srityje. Vis dėlto aukščiausius rezultatus šioje srityje pasiekiantys uždarieji modeliai, pavyzdžiui, GPT-4 ar Claude, reikalauja išorinių paslaugų, brangių licencijų ir perduoda naudotojo kodą trečiosioms šalims. Dėl šios priežasties mokslo tyrimuose ir praktiniame darbe vis didesnis dėmesys skiriamas atvirųjų svorių 7-9 milijardų parametrų modeliams, kuriuos galima vykdyti vietoje ir visiškai kontroliuoti. Tačiau šie mažesni modeliai sudėtingesnėse kodo generavimo užduotyse dažnai pateikia neteisingą, nepilną arba netinkamai struktūruotą kodą. Taip susidaro praktinis atotrūkis tarp prieinamų atvirųjų svorių modelių galimybių ir realių programinės įrangos kūrimo poreikių.

Vienas perspektyviausių būdų šį atotrūkį sumažinti yra kelių agentų debatai, kuriuose keli savarankiški kalbos modeliai bendradarbiauja iteratyviai: pateikia pradinį sprendimą, kritikuoja vieni kitus, peržiūri savo kodą ir balsavimu nustato geriausią variantą. Tokiu būdu silpnescio modelio klaidas gali atskleisti kiti dalyviai, o kolektyvinis sprendimas dažnai pranoksta bet kurio atskiuro agento pasiūlytą atsakymą. Šiame kontekste atvirųjų svorių modeliai yra ypač patogi tyrimo aplinka, nes leidžia tiesiogiai keisti agentų rinkinį, papildomai įtraukti stipresnį teisėjo agentą, varijuoti revizijos strategijas ir sistemingai matuoti kiekvieno komponento įtaką. Be to, sukurta debatų sistema suprojektuota moduliškai, todėl ateityje, atlikus reikiamus integracijos pakeitimus, ta pati schema galėtų būti pritaikyta ir uždariesiems didelio masto modeliams.

Šio darbo problema yra ta, kad daugumoje žinomų kelių agentų debatų tyrimų naudojami didelio masto uždarieji modeliai, todėl iš jų rezultatų sunku spręsti, kaip toks metodas veikia su daug mažesniais atvirųjų svorių modeliais. Be to, nėra aišku, kurie debatų sistemos komponentai, tokie kaip agentų rinkinio sudėtis ir teisėjo modelio tipas, realiai prisideda prie kokybės pagerėjimo, o kurie reikšmingo poveikio neturi, ir koks yra tikrasis debatų pridėtinės vertės dydis lyginant su vieno modelio bazine linija (solo režimu). Praktinį taikymą dar labiau komplikuoja tai, kad kokybės pagerėjimas turi atsverti didesnes debatų metodo skaičiavimo sąnaudas. Todėl būtina sistemingai ištirti, kokiomis sąlygomis kelių agentų debatai iš tiesų pagerina atvirųjų svorių 7-9 milijardų parametrų modelių kodo generavimo testų išlaikymo rodiklį ir kaip šis pagerėjimas susijęs su konkrečiomis sistemos konfigūracijomis.

Darbo tikslas - eksperimentiškai ištirti kelių agentų debatų ir skirtingo tipo teisėjo agentų poveikį atvirųjų svorių 7-9 milijardų parametrų kalbos modelių kodo generavimo kokybei skirtingo sudėtingumo užduotyse, lyginant su vieno modelio bazine linija.

Darbo uždaviniai:

1. Išanalizuoti kelių agentų debatų ir LLM-as-Judge metodų literatūrą, nustatyti pagrindines architektūras bei žinomus apribojimus ir, remiantis literatūros bei aparatinės įrangos kriterijais, atrinkti tinkamus eksperimentams atvirųjų svorių 7-9 milijardų parametrų kodo generavimo modelius.
2. Suprojektuoti ir įgyvendinti modulinę kelių agentų debatų sistemą su konfigūruojamu agentų rinkiniu, neprivalomu teisėjo agentu, keliomis revizijos strategijomis ir kontrolės mechanizmais, skirtais riboti netipinių modelių elgesį.
3. Sudaryti standartizuotą šešiasdešimties kodo generavimo užduočių pagrindinį rinkinį, suskirstytą į keturis sudėtingumo lygius, ir papildomą tos pačios apimties sudėtingo bei ekstremalaus lygio užduočių rinkinį, iš kurių pusę sudaro specialiai šio darbo metu parengti nauji scenarijai, su izoliuotu vienetinių testų vykdymu, kodo kokybės metrikomis ir dokumentuojamais rezultatais.
4. Atlikti kontroliuojamą eksperimentinį tyrimą dviem etapais: pradinis konfigūracijų bandymus įvykdyti sukurtoje interaktyvioje žiniatinklio sąsajoje, o pagrindinį

eksperimentą - didelio našumo skaičiavimo aplinkoje, palyginant agentų rinkinio sudėtį (7B ir 9B klasės modeliai) bei teisėjo modelio tipą (be teisėjo, kodo specializacijos teisėjas, samprotavimo teisėjas), naudojant fiksuotus sistemos parametrus (adaptyvi temperatūra, kritikos istorija, geriausio kito agento sprendimo rodymo režimas), parinktus pagal preliminarinius bandymus.

5. Įvertinti, kuriomis sąlygomis kelių agentų debatai reikšmingai pranoksta vieno modelio bazinę liniją, ir suformuluoti rekomendacijas dėl praktinio metodo taikymo atvirųjų svorių 7-9 milijardų parametrų modeliams.

1. Kelių agentų debatai ir kodo generavimo kontekstas

Šiame skyriuje apžvelgiama kelių agentų debatų metodų taikymo sritis kodo generavimo užduotyse: aptariami didžiųjų kalbos modelių gebėjimai ir ribotumai, pristatoma kelių agentų debatų metodologija, „LLM-as-Judge“ paradigma bei aptariami susiję darbai, leidžiantys nustatyti šio tyrimo nišą.

1.1. Didieji kalbos modeliai kodo generavime ir jų ribotumai

Nuo pat „Transformer“ architektūros pristatymo [Vsp+17], didieji kalbos modeliai (DKM, angl. Large Language Models) tapo pagrindine technologija natūralios kalbos apdorojimo užduotyse, o pastaraisiais metais jų taikymo sritis sparčiai plečiasi į kodo generavimą. Komerciniai uždarieji modeliai, tokie kaip GPT-4 [Ope23] ir Claude 3 [Ant24], pasiekė tokius programavimo gebėjimus, kurie leidžia juos integruoti į industrines programinės įrangos kūrimo aplinkas - pavyzdžiui, „GitHub Copilot“ įrankis tiesiogiai siūlo kodo užbaigimą realiu laiku. Kontroliuojami eksperimentai patvirtina šių įrankių praktinę naudą: Peng ir kt. [Pkc+23] tyrime nustatyta, kad „GitHub Copilot“ naudotojai pavyzdinę programavimo užduotį (HTTP serverio realizaciją „JavaScript“ kalba) atliko 55,8% greičiau nei kontrolinė grupė, o „GitHub“ ir „Accenture“ įmonėje atlikta apklausa [Gg24] taip pat užfiksavo reikšmingą programuotojų produktyvumo padidėjimą.

Vis dėlto šiems rezultatams pasiekti reikalingi modeliai, kuriuos paprastai sudaro šimtai milijardų parametrų ir kurie pasiekiami tik per mokamas API, perduodančias naudotojo kodą trečiajai šaliai. Reaguojant į šį apribojimą, susiformavo paraleli tyrimų kryptis - atvirųjų svorių (angl. open-weight) kodo generavimo modelių šeima, kurios svoriai yra viešai prieinami ir kuriuos galima vykdyti vietinėje techninėje įrangoje. Šiai šeimai priklauso „Code Llama“ [Rgg+23], „Qwen2.5-Coder“ [Hyc+24], „DeepSeek-Coder“ [Gzy+24], „Granite Code“ [Ms2+24], „Yi“ [Ycl+24] bei „CodeGeeX“ [Zxz+23] modeliai, kurių vartotojui prieinamos versijos paprastai turi nuo 6 iki 9 milijardų parametrų. Hou ir kt. [Hzi+24] sisteminėje literatūros apžvalgoje, apimančioje programų inžinerijos darbus su didžiaisiais kalbos modeliais, pažymima, kad atvirųjų svorių modeliai vis dažniau pasiekia rezultatus, palyginamus su uždarytųjų modelių rezultatais standartiniuose vertinimo rinkiniuose.

Vis dėlto pačių vertinimo rinkinių patikimumas pastaruoju metu kelia abejonių. Liu ir kt. [Lxw+23] parodė, kad plačiai naudojamas „HumanEval“ rinkinys [Ctj+21] turi nepakankamai testų, todėl praplėtus jį papildomais testavimo atvejais (autorių pasiūlytas „EvalPlus“ rinkinys padidina „HumanEval“ testų skaičių iki 80 kartų), modelių sėkmės rodiklis (pass@k) sumažėja iki 28,9%. Dar ryškiau šis atotrūkis pasireiškia daugiafailėse užduotyse: Jimenez ir kt. [Jyw+24] pristatytas „SWE-bench“ rinkinys, sudarytas iš 2 294 realių „GitHub“ problemų 12-oje „Python“ repozitorijų, parodė, kad straipsnio publikavimo metu geriausiai veikęs „Claude 2“ modelis sugebėjo išspręsti tik 1,96% užduočių. Šie rezultatai rodo aiškų atotrūkį tarp paprastų vienos funkcijos užduočių ir realaus programinės įrangos kūrimo proceso, kuriame būtina suprasti kelis modulius, klasių sąsajas bei testų grįžtamąjį ryšį. Mažesni atvirųjų svorių modeliai, naudojami šiame darbe, šiame kontekste atsiduria dar nepalankesnėje padėtyje, todėl iškyla klausimas, kokios papildomos technikos gali pagerinti jų gebėjimus sudėtinguose scenarijuose.

1.2. Kelių agentų debatų (MAD) metodologija

Kelių agentų debatai (angl. multi-agent debate, MAD) - tai inferencijos metu taikoma technika, kurioje keli atskiri didžiųjų kalbos modelių egzemplioriai keliais raundais kritikuoja vieni kitų sprendimus ir balsavimu nustato galutinį atsakymą. Šis požiūris kontrastuoja su vieno agento metodais, tokiais kaip mąstymo grandinės (angl. Chain-of-Thought, CoT) skatinimas [Wws+22a] arba savarankiškas atsakymo perrašymas remiantis paties modelio kritika („Self-Refine“ [Mtg+23], „Reflexion“ [Scb+23]). Kelių agentų debatų prielaida yra ta, kad atskiri

modeliai turi nevienodą žinių aprėptį, klaidų šablonus bei generavimo polinkius, todėl jų sąveika gali atskleisti klaidas, kurias vienas modelis savarankiškai neidentifikuotų.

Vieną pirmųjų ir labiausiai cituojamų MAD darbų pristatė Du ir kt. [Dlt+23]. Jų schema numatė kelis modelio egzempliorius, kurie keliais raundais generuoja sprendimus ir argumentus, peržiūri kitų agentų atsakymus bei juos taiso. Autoriai parodė, kad toks debatas pagerina rezultatus matematinio bei strateginio samprotavimo užduotyse ir sumažina šiuolaikiniams modeliams būdingas haliucinacijas. Svarbu, kad metodas nekeičia modelio architektūros - jis taikomas tik inferencijos metu, naudojant identišką procedūrą bei užklausas (angl. prompts) visoms užduotims, todėl gali būti integruojamas į esamas sistemas neatlikus papildomo apmokymo.

Liang ir kt. [Lhj+23] pasiūlė formalesnę debatų schemą, kurioje agentai vykdo „akis už akį“ (angl. tit-for-tat) ginčą, o atskiras teisėjo agentas valdo proceso eigą ir pasirenka galutinį atsakymą. Autoriai identifikuoja problemą, kurią pavadina „minčių degeneracija“ (angl. Degeneration-of-Thought, DoT) - reiškini, kai modelis nesugeba sugeneruoti naujų idėjų savikritikos metu, net jei pradinė pozicija yra klaidinga. Kelių agentų adversariška sąveika, autorių teigimu, įveikia šį apribojimą, nes priverčia modelius nagrinėti alternatyvius problemos sprendimo kelius. Eksperimentai bendrosios sąmonės mašininio vertimo (angl. commonsense machine translation) bei intuicijai prieštaraujančio aritmetinio samprotavimo užduotyse parodė reikšmingą tikslumo pagerėjimą.

Be tiesioginio sprendimų generavimo, MAD principas pritaikomas ir vertinimo užduotyse. Chan ir kt. [Ccs+23] pristatytas „ChatEval“ karkasas formuoja kelių agentų vertintojų komandą, kurioje keli kalbos modeliai bendrai vertina atvirų klausimų ir natūralios kalbos generavimo užduočių atsakymus. Šis požiūris atspindi žmogiškąjį vertinimo procesą, kuriame keli ekspertai derina savo nuomones, ir, autorių duomenimis, leidžia pasiekti patikimesnius rezultatus nei vieno agento vertinimas. Šis darbas glaudžiai siejasi su „LLM-as-Judge“ paradigma, kuri detaliau aptariama 1.3 poskyryje.

Naujesni darbai atskleidžia subtilesnį MAD efektyvumo vaizdą. Khan ir kt. [Khv+24] eksperimentiškai parodė, kad kai du stipresni „ekspertų“ modeliai diskutuoja tarpusavyje, o silpnesnis „ne-eksperto“ modelis išrenka galutinį atsakymą, ne-eksperto tikslumas pakyla nuo 48% (be debatų) iki 76% (su debatais), o žmonių vertintojų - nuo 60% iki 88%. Tai palaiko MAD naudojimą DI saugumo suderinimo (angl. AI alignment) srityje, kur silpnesnė vertinimo pusė turi atskirti tiesą iš dviejų stipresnių argumentų. Vis dėlto Smit ir kt. [Sdg+24] sisteminiame palyginime nustatė, kad MAD sistemos savo dabartine forma „nepatikimai pranoksta kitas siūlomas užklausų strategijas“, tokias kaip vidinė darna (angl. self-consistency) ar atsakymų ansambliavimas. Autoriai pažymi, kad MAD yra jautrios hiperparametrų pasirinkimui - jas tinkamai suregulius, jos gali viršyti alternatyvas, tačiau be tinkamo derinimo MAD privalumai nepasireiškia. Šis kritinis požiūris parodo, kad MAD pranašumas nėra savaiminis ir priklauso nuo agentų sudėties, debato struktūros bei konkrečios užduoties pobūdžio.

1.3. „LLM-as-Judge“ paradigma

„LLM-as-Judge“ paradigma - tai praktika, kai didysis kalbos modelis naudojamas kaip automatinis vertintojas, sprendžiantis, kuris iš dviejų ar daugiau kandidatinių atsakymų yra geresnis arba kurio kokybės reitingas yra aukštesnis. Šios paradigmos motyvacija yra praktinė: tradicinis žmonių atliekamas atsakymo kokybės vertinimas yra brangus ir lėtas, todėl jis sunkiai pritaikomas, kai eksperimentai apima šimtus ar tūkstančius modelio atsakymų. Stiprūs šiuolaikiniai modeliai gali iš dalies pakeisti šį procesą, atlikdami vertinimą pagal iš anksto suformuluotą užklausą ir grąžindami struktūrizuotą vertinimo balą bei pagrindimą.

Pamatinį šios paradigmos darbą pristatė Zheng ir kt. [Zcs+23], kurie sukūrė du vienas kitą papildančius vertinimo įrankius: „MT-Bench” - daugiažingsnių pokalbių klausimų rinkinį, skirtą DKM pagrindu veikiančių pokalbių asistentų vertinimui, ir „Chatbot Arena” - bendruomenės balsavimo platformą, kurioje žmonės išreiškia savo pirmenybę vienam iš dviejų modelių atsakymų. Autoriai parodė, kad stiprūs kalbos modeliai-teisėjai, tokie kaip GPT-4, pasiekia daugiau nei 80% sutarimą su žmonių vertinimais - tai yra toks pat sutarimo lygis, kokį demonstruoja žmonių vertintojai tarpusavyje. Vis dėlto darbe identifikuojami keli sisteminiai šių teisėjų šališkumai: pozicijos šališkumas (angl. position bias) - sprendimą veikia atsakymų pateikimo eiliškumas; daugiažodiškumo šališkumas (angl. verbosity bias) - polinkis aukščiau vertinti ilgesnius atsakymus; bei palankumo sau šališkumas (angl. self-enhancement bias) - polinkis aukščiau vertinti atsakymus, panašius į paties teisėjo modelio stilių. Šie šališkumai svarbūs, nes jie nurodo ribas, ties kuriomis „LLM-as-Judge” negali iki galo pakeisti žmogaus.

Vieno kalbos modelio, kaip teisėjo, naudojimas yra natūralus pradinis taškas, tačiau vėlesni darbai siūlo plėsti šią paradigmą įtraukiant kelis vertintojus. Anksčiau aptartas Chan ir kt. [Ccs+23] „ChatEval” karkasas naudoja kelių agentų vertintojų komandą, kurioje atskiri kalbos modeliai diskutuoja prieš pateikdami galutinį verdiktą - tai sumažina vieno teisėjo šališkumo įtaką. Khan ir kt. [Khv+24] tyrime „LLM-as-Judge” mechanizmas pritaikomas DI saugumo suderinimo (angl. AI alignment) kontekstui: silpnesnis modelis vertina dviejų stipresnių „ekspertų” debatą ir, kaip jau buvo paminėta 1.2 poskyryje, debatų metu pasiekiamas reikšmingas vertintojo tikslumo padidėjimas. Šis kombinuotas požiūris - kai debatai derinami su „LLM-as-Judge” - sudaro koncepcinį pagrindą šio darbo tiriamai sistemai.

Šiame darbe „LLM-as-Judge” principas integruojamas į kelių agentų debatų ciklą per atskirą teisėjo agentą (angl. judge agent). Skirtingai nuo agentų, kurie generuoja kodo sprendimus, teisėjo agentas atsako tik už dvi funkcijas: kitų agentų sprendimų kritiką bei galutinį balsavimą. Toks atskyrimas leidžia pasitelkti stipresnį modelį (pavyzdžiui, kodo specializacijos modelį arba samprotavimo modelį, kurie turi daugiau parametrų nei debatuojančių agentų rinkinio modeliai), nereikalaujant iš jo generuoti pilnų sprendimų - tai sumažina skaičiavimo sąnaudas, kartu suteikiant debatams išorinį, mažiau šališką vertinimo signalą. 1.4 poskyryje aptariami susiję darbai bei tyrimo niša, kurioje atliekamas šis tyrimas.

1.4. Susiję darbai ir tyrimo niša

Šio darbo kontekstui aktualios kelios kalbos modelių pajėgumo gerinimo kryptys, kurios naudoja vieną modelį, o ne kelis agentus. Wei ir kt. [Wws+22a] mąstymo grandinės (angl. Chain-of-Thought) metodas skatina modelį generuoti tarpinį samprotavimą prieš galutinį atsakymą, o Wang ir kt. [Wws+22b] vidinės darnos (angl. self-consistency) strategija generuoja kelias mąstymo grandines iš to paties modelio ir balsavimu pasirenka dažniausią atsakymą. Madaan ir kt. [Mtg+23] „Self-Refine” karkasas remiasi idėja, kad modelis iteratyviai pateikia grįžtamąjį ryšį savo paties atsakymui ir jį tobulina, o Shinn ir kt. [Scb+23] „Reflexion” pademonstravo, kad verbalinė savirefleksija pagerina rezultatus iteracinėse užduotyse. Specifiškai kodo generavimo srityje Jiang ir kt. [Jdw+25] „Self-planning” metodas siūlo prieš kodo rašymą sukurti struktūrizuotą sprendimo žingsnių planą; autorių eksperimentai parodė iki 25,4% santykinį Pass@1 rodiklio padidėjimą, palyginti su tiesioginiu generavimu. Šie metodai parodo, kad atsakymo kokybė gali būti pagerinta net naudojant vieną modelį, tačiau tuo pat metu išryškina vieno modelio ribas: jis gali pakartoti savo paties klaidas keliose mąstymo grandinėse arba neaptikti loginės klaidos, kuri pačiam modeliui nėra akivaizdi.

Pastaraisiais metais išsiskiria keli plačiai cituojami kelių agentų karkasai. Wu ir kt. [Wbz+23] „AutoGen” yra bendrosios paskirties karkasas, leidžiantis konstruoti kelių agentų sistemas, kuriose modeliai bendrauja tarpusavyje atlikdami užduotis matematikos, programavimo, klausimų-atsakymų ir kitose srityse. Programų inžinerijos kontekste plėtojami labiau specializuoti darbai: Hong ir kt. [Hzc+24] „MetaGPT” karkasas bei Qian ir kt. [Qll+23]

„ChatDev” sistema. „MetaGPT” perkelia į kelių agentų sąveiką žmogiškos komandos darbo srautą, pritaikydamas standartines veiklos procedūras (angl. Standard Operating Procedures, SOP) skirtingiems agentų vaidmenims ir taip sumažindamas kaskadinę haliucinacijų grandinę, kuri atsiranda neapgalvotai sujungiant kalbos modelių iškvietus. „ChatDev” panašiai modeliuoja virtualios programų inžinerijos įmonės gyvavimo ciklą, suskirstydamas darbą į projektavimo, kodavimo ir testavimo etapus, kuriuose agentai bendrauja natūralia kalba projektavimo metu ir programavimo kalba derinant kodą. Abu darbai parodo, kad kelių agentų sąveika su aiškiai apibrėžtais vaidmenimis leidžia spręsti sudėtingesnes programų inžinerijos užduotis, nei tai įmanoma naudojant vieno agento požiūrį.

Verta pabrėžti esminį skirtumą tarp aukščiau aptartų darbų ir šiame tyrime nagrinėjamos sistemos. „MetaGPT” ir „ChatDev” kelių agentų sąveiką kuria per vaidmenų diferenciaciją: agentai pasidalina užduotį pagal vaidmenis (vienas projektuoja architektūrą, kitas rašo kodą, trečias jį testuoja). Šio darbo požiūris yra ortogonalus: visi pagrindiniai agentai atlieka tą patį vaidmenį - kodo generavimą, - tačiau juos sudaro skirtingi atvirųjų svorių modeliai („Qwen2.5-Coder”, „DeepSeek-Coder”, „Code Llama”, „Granite Code”, „CodeGeeX4”, „Yi-Coder”), todėl jų sąveikos metu atsiskleidžia ne darbo pasidalijimas, o skirtingų modelių žinių ir klaidų šablonų papildomumas. Be to, šiame darbe naudojami gerokai mažesni 7-9 milijardų parametrų modeliai, kurie sudėtingose programavimo užduotyse, kaip parodyta 1.1 poskyryje, demonstruoja reikšmingą atotrūkį nuo stipriausių uždarytųjų modelių - tai praplečia tyrimų lauką į mažesnės skaičiavimo apimties scenarijus, kuriems iki šiol skiriama mažiau dėmesio.

Skyriaus apibendrinimas. Šio darbo tyrimo niša susiformuoja per kelis specifinius pasirinkimus. Pirma, naudojami atvirųjų svorių 7-9 milijardų parametrų modeliai - sritis, kurioje sudėtingose užduotyse pasireiškia reikšmingas atotrūkis nuo uždarytųjų modelių. Antra, taikomas kelių agentų debatų principas su architektūriniu modelių įvairumu, o ne vaidmenimis grįstas darbo pasidalijimas. Trečia, į debatų ciklą įtraukiamas atskiras teisėjo agentas, įgyvendinantis „LLM-as-Judge” paradigmą, kurio modelio tipas (kodo specializacijos ar samprotavimo) gali būti varijuojamas eksperimentų metu. Sukurta sistema sudaro pagrindą empiriškai ištirti, kuriomis sąlygomis tokia konfigūracija pranoksta vieno modelio bazinę liniją kodo generavimo užduotyse - šis klausimas yra centrinis šio darbo tyrimo objektas, nagrinėjamas 4 skyriaus eksperimentų rezultatuose.

2. Debatų sistemos projektavimas ir realizacija

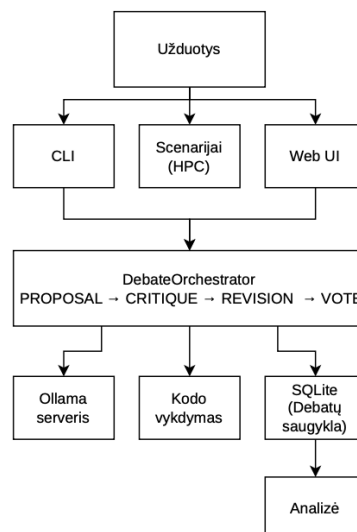
Šiame skyriuje pristatoma sukurta kelių agentų debatų sistema, skirta automatizuotam kodo generavimui atvirųjų svorių 7-9 milijardų parametru modeliais. Aptariama bendra architektūra, debatų ciklo realizacija, balsavimo bei konsensuso algoritmai, atsakymų kontrolės mechanizmai, daugiafailių užduočių apdorojimas, užklausų konstravimas ir atsakymų išskyrimas (angl. parsing), kodo vykdymas izoliuotoje aplinkoje, pagalbinių interaktyvi sąsaja ir duomenų saugojimo sluoksnis. Visi šiame skyriuje pateikiami techniniai sprendimai paremti realiu sistemos kodu.

2.1. Sistemos architektūra ir technologijų pasirinkimas

Sukurta sistema yra modulinė „Python“ programa, kurios bendra apimtis siekia apie 27 tūkstančius eilučių - didžiausią dalį (apie 8,7 tūkst.) sudaro pagrindinis sistemos kodas paketo `src/` viduje, likusi dalis pasiskirsto tarp vienetinių testų, pagalbinių paleidimo bei analizės scenarijų, žiniatinklio sąsajos šablono, didelio našumo skaičiavimo (HPC) scenarijų ir 120 užduočių aprašymų JSON formatu. Pagrindinis kodas suskirstytas į šešis nepriklausomus paketus, suprojektuotus pagal sluoksniavimo (angl. layered architecture) principą - aukštesnio sluoksnio komponentai naudoja žemesnio sluoksnio paslaugas, bet ne atvirkščiai, todėl kiekvieną sluoksnį galima testuoti ir keisti atskirai.

2.1.1. Bendras informacijos srautas ir viešojo programavimo sąsaja

Sistema turi tris nepriklausomus įėjimo taškus, per kuriuos galima paleisti debatų ciklą: komandinės eilutės sąsają (angl. Command Line Interface, CLI), realizuotą faile `src/main.py`; žiniatinklio sąsają, realizuotą faile `src/web/app.py`; ir paketinio režimo HPC scenarijus (`hpc/_lib_run.sh` ir jį naudojančius `hpc/run_*.sh` failus), kurie veikia kaip CLI apvalkalas ir leidžia eksperimentus paleisti SLURM klasteryje. Visi trys įėjimo taškai gauna užduotis iš to paties JSON formato užduočių rinkinio (`tasks/` ir `tasks2/` katalogai) ir naudoja tą pačią pagrindinę logikos klasę `DebateOrchestrator` (`src/core/orchestrator.py`), kuri valdo visą debatų ciklą - keturias pagrindines fazes (pradinį pasiūlymą, kritiką, reviziją ir balsavimą) bei jų rezultato apibendrinimą per konsensuso algoritmą. Orkestratorius bendrauja su trim žemesnio lygio komponentais: kalbos modelio klientu (`src/llm/`), atliekančiu HTTP užklausas „Ollama“ serveriui; kodo vykdytoju (`src/core/executor.py`), iš agentų gautą sprendimą paleidžiančiu izoliuotoje „pytest“ aplinkoje, papildomai vertinant kodo kokybę „pylint“ bei „radon“ įrankiais; ir repozitorijos sluoksniu (`src/database/`), išsaugančiu debatų rezultatus „SQLite“ duomenų bazėje. Po debatų ciklo papildomas analizės modulis (`src/analysis/`) skaito išsaugotus duomenis ir generuoja lengvai interpretuojamus protokolus bei CSV eksportus, naudojamus eksperimentų rezultatų analizei (žr. 4 skyrių). Bendras informacijos srautas pavaizduotas 1 diagramoje.



1 diagrama. Sistemos aukšto lygio architektūra.

Diagramoje parodyta, kad visi trys įėjimo taškai - CLI, paketinio režimo HPC scenarijai (techniškai - Bash apvalkalas aplink CLI, žr. 3.4.2 poskyrį) ir žiniatinklio sąsaja - užduotis gauna iš to paties JSON rinkinio ir kreipiasi į vieną orkestratorių, kuris vykdo keturias debatų fazes, o po kiekvieno balsavimo įvertina pasiektą konsensumą (žr. 2.3 poskyrį) bei paskirsto darbą trim pagrindiniams pagalbiniais komponentams. „SQLite” duomenų bazėje išsaugoti debatų protokolai vėliau naudojami atskirai paleidžiamame analizės modulyje, kuris pateikia rezultatus toliau analizei skirtu formatu.

Sistemos viešoji programavimo sąsaja (angl. Application Programming Interface, API) apibrėžta `src/__init__.py` faile - per ją išorinis kodas (testai, scenarijai, integracijos) gauna prieigą prie pagrindinių klasių. 1 kodo fragmente pateiktas šio failo turinys, eksportuojantis viešas klases.

```
# src/__init__.py
from .models import (
    Agent,
    AgentConfig,
    AgentRole,
    Debate,
    DebateConfig,
    Solution,
    Task,
)
from .core import DebateOrchestrator
from .llm import MultiModelClient, OllamaClient
__all__ = [
    "Agent", "AgentConfig", "AgentRole",
    "Debate", "DebateConfig", "DebateOrchestrator",
    "MultiModelClient", "OllamaClient",
    "Solution", "Task",
]
```

1 kodo fragmentas. Sistemos viešoji programavimo sąsaja.

Kaip matyti 1 kodo fragmente, kaip viešos eksportuojamos tik dešimt klasių - vidinės realizacijos detalės, tokios kaip užklausų generavimo procedūros, kritikų išskyrimo logika, balsavimo svėrimas ir podėliavimo (angl. caching) mechanizmas, lieka paslėptos paketuose. Toks inkapsuliavimo (angl. encapsulation) sprendimas leidžia ateityje keisti vidines realizacijas neliečiant sistemą naudojančio išorinio kodo.

2.1.2. Modulinis paketų skirstymas

`src/` paketas padalintas į šešis Python paketus, kurių kiekvienas turi vieną aiškiai apibrėžtą atsakomybę. Toks skirstymas leidžia bet kurį komponentą testuoti izoliuotai - pavyzdžiui, debatų logiką su imitaciniu (angl. mock) didžiojo kalbos modelio (toliau - DKM) klientu - bei keisti realizaciją neliečiant kitų sluoksnių, tarkime, pakeičiant „SQLite” saugojimą į „PostgreSQL”. 1 lentelėje pateikti šeši paketai su atsakomybių aprašais.

Paketas	Atsakomybė
core/	Debatų ciklas, kodo vykdymas, konsensusas, užklausų konstravimas
llm/	Kalbos modelio klientas, abstrahuojantis konkretų teikėją
models/	Duomenų klasės: Task, Solution, Critique, Vote, DebateConfig, metrikos
database/	Debatų rezultatų saugojimas SQLAlchemy ORM pagrindu
analysis/	Metrikų skaičiavimas, protokolų generavimas, CSV eksportas
web/	Žiniatinklio sąsaja su realaus laiko eigos atvaizdavimu

1 lentelė. Sistemos pagrindinio paketo `src/` modulinis skirstymas.

Iš šešių paketų didžiausią apimtį turi core/, kuriame koncentruojasi visa debatų logika - orkestratorius, užklausų konstravimas, kodo vykdymas ir konsensuso algoritmas. Toks architektūrinis sprendimas pasirinktas todėl, kad orkestratorius turi visišką debatų ciklo būsenos kontrolę - jo skaidymas į atskiras paslaugas apsunkintų nuoseklių fazių vykdymą bei sinchronizaciją. Likę penki paketai yra mažesni ir atlieka aiškiai atskirtas pagalbines funkcijas.

2.1.3. Technologijų pasirinkimo pagrindimas

Sistema realizuota „Python 3.11” kalba. Pasirinkimas pagrįstas trim kriterijais: „Python” pasižymi patogia integracija su „Ollama” HTTP API per oficialią klientų biblioteką, plačia mokslinių eksperimentų ekosistema („pandas”, „matplotlib”, „seaborn”), leidžiančia tiesiogiai apdoroti rezultatus toje pačioje aplinkoje, bei naujausios kalbos versijos asyncio modulio asinchroninio programavimo galimybėmis - šis modelis, suteikiantis neblokuojantį įvedimą-išvedimą (angl. non-blocking I/O), būtinas lygiagrečiam kelių agentų užklausų vykdymui. Atvirųjų svorių modelių valdymui pasirinkta vietinė „Ollama” HTTP API (priešvardas 11434), kuri palaiko visus tyrime naudotus 7-9 milijardų parametrų modelius - „Qwen2.5-Coder”, „DeepSeek-Coder”, „Code Llama”, „Granite Code”, „Yi-Coder” bei „CodeGeeX” (detalesnis modelių aprašymas su nuorodomis į originalius straipsnius pateiktas 3.2 poskyryje) - ir leidžia atmintyje lygiagrečiai laikyti kelis modelius (detalesnis „Ollama” eksploatacijos parametrų aprašymas - 2.1.5 poskyryje).

Asinchroninis HTTP klientas httpx kartu su tenacity biblioteka užtikrina laikinų tinklo gedimų atstatymą eksponentiniu pakartotinių užklausų (angl. retry) algoritmu - šis aspektas yra svarbus HPC aplinkoje, kur „Ollama” serverio paleidimas gali užtrukti iki trisdešimties sekundžių. Vidinės duomenų klasės realizuotos dataclasses standartu dėl mažesnio atminties poreikio, o DKM atsakymų struktūra papildomai validuojama pydantic biblioteka. Debatų rezultatai saugomi „SQLite” duomenų bazėje per SQLAlchemy ORM sluoksnį, kuris, prireikus, leidžia perjungti į „PostgreSQL” nepakeičiant aukštesnio lygio kodo. Kodo sprendimų vykdymui pasirinktas pytest su pytest-timeout plėtinio, ribojančiu begalines kilpas iki trisdešimties sekundžių (žr. 2.7 poskyrį). Statinė kodo analizė atliekama pylint ir radon įrankiais - pirmasis įvertina stilių ir loginius defektus dešimtbalėje skalėje, antrasis skaičiuoja ciklostatinį sudėtingumą; abi metrikos naudojamos balsavimo balui koreguoti (žr. 2.3 poskyrį). Žiniatinklio sąsaja paremta „Flask” karkasu su flask-socketio plėtinio ir eventlet asinchroniniu varikliu, leidžiančiais atvaizduoti debatų eigą realiu laiku per WebSocket kanalą (žr. 2.8 poskyrį). Komandinės eilutės sąsajai pasirinkti argparse ir rich įrankiai, formatuojantys konsolės išvestį ir pagerinantys vykdymo HPC klasterio stebėseną. Pilnas priklausomybių sąrašas su tiksliais versijomis pateiktas faile requirements.txt.

2.1.4. Inžinerinis iššūkis: konteksto lango parinkimas

Vienas kritinių sistemos veikimo parametrų yra „Ollama” konteksto lango dydis (parametras num_ctx), nustatantis maksimalų užklausos ir atsakymo žetonų kiekį, kurį modelis gali apdoroti vienu metu. Šio parametro derinimas pareikalavo dviejų iteracijų ir paaiškino didelę dalį pradinių eksperimentų metu pastebėtų neįprastos elgsenos atvejų.

Pirmoji problema buvo numatytoji „Ollama” reikšmė num_ctx=2048, kuri 7-9 milijardų parametrų modeliams kelių agentų debatuose yra nepakankama. Revizijos užklausa apima sistemos užklausa, užduoties tekstą, paties agento ankstesnį sprendimą, geriausio kito agento sprendimo (angl. peer solution) kodą bei kritikos santraukas - bendrai tai užima 2 000 ir daugiau žetonų, todėl atsakymo generavimui paprasčiausiai nelieta vietos. Šiame režime modeliai grąžindavo tuščius arba sintaksiškai netaisyklingus atsakymus, o klaidos priežastis nebuvo matoma debatų protokoluose.

Pirmoji iteracija konteksto langą padidino iki num_ctx=16384. Šio dydžio pakako standartinėms vieno failo užduotims (lengvas, vidutinis ir sudėtingas lygis), tačiau eksperimentai su ekstremalaus lygio daugiafailėmis užduotimis pademonstravo antrąją problemą: didesnės užklauskos su keliomis testų dalimis vis dar pasiekdavo 16 384 žetonų ribą, ir atsakymas būdavo nepastebimai sutrumpinamas (angl. silent truncation). Modelis grąžindavo formaliai tinkamą atsakymą, tačiau jis būdavo nutrauktas funkcijos viduryje. Antroji iteracija padidino reikšmę iki num_ctx=32768 - šis dydis tiksliai atitinka tyrimo metu naudoto „Qwen2.5-Coder“ modelio numatytąją konteksto lango konfigūraciją (originaliame config.json faile - 32 768 žetonai be papildomo YaRN mastelio keitimo) [Hyc+24], o „DeepSeek-Coder-V2“ [Dee24], naudotas kaip kodo specializacijos teisėjo modelis (žr. 3.2 poskyrį), palaiko net 128 tūkst. žetonų konteksto langą, todėl pasirinktoji 32 768 reikšmė yra konservatyvi abiem atvejais. Galutinis sprendimas užfiksuotas faile src/llm/ollama_client.py ir pateiktas 2 kodo fragmente.

```
# src/llm/ollama_client.py
"options": {
    "temperature": request.temperature,
    "num_predict": request.max_tokens,
    "num_ctx": 32768,
},
```

2 kodo fragmentas. „Ollama“ užklauskos parametrai su konteksto lango dydžiu.

Be konteksto lango padidinimo, papildomai realizuotas tiesioginis sutrumpinimo aptikimo mechanizmas: kiekvienai užklausiai tikrinamas „Ollama“ grąžinamas done_reason laukas, o jeigu jo reikšmė yra length, registruojamas įspėjimas su informacija apie modelį, sunaudotų žetonų skaičių, sukonfigūruotą maksimalų atsakymo dydį (num_predict parametras) ir užklauskos trukmę. Šis mechanizmas reikalingas tam, kad eksperimentų metu sutrumpinti atsakymai būtų aiškiai matomi žurnale ir nebūtų klaidingai interpretuojami kaip natūraliai trumpi modelio atsakymai. Neaptiktas sutrumpinimas pradinuose eksperimentuose iškreipė kelių užduočių rezultatus, kol nebuvo įdiegta diagnostika.

2.1.5. Eksploatacijos optimizacijos HPC aplinkoje

Sistema paleidžiama dviejose skirtingose aplinkose - vietinėje darbo stotyje preliminariniams bandymams ir VU MIF didelio našumo skaičiavimo klasteryje pagrindiniam eksperimentui. HPC aplinkoje papildomai įjungiamas „Flash Attention“ mechanizmas (OLLAMA_FLASH_ATTENTION=1), apie pusę sumažinantis raktų ir reikšmių podėlio (angl. KV-cache) atminties poreikį „NVIDIA V100“ grafikos plokštėje (32 GB), bei nustatomas OLLAMA_MAX_LOADED_MODELS=5 parametras, leidžiantis vienu metu atmintyje laikyti iki penkių modelių, todėl debatų ciklo metu agentų ir teisėjo modelių keitimas nereikalauja pakartotinio užkrovimo. Šie aplinkos kintamieji nustatomi paleidimo scenarijaus metu, kaip parodyta 3 kodo fragmente.

```
# hpc/_lib_run.sh
export OLLAMA_MODELS="${OLLAMA_DATA}"
export OLLAMA_MAX_LOADED_MODELS=5
export OLLAMA_FLASH_ATTENTION=1
```

3 kodo fragmentas. „Ollama“ eksploatacijos parametrai HPC aplinkoje.

Tokia konfigūracija leidžia vienos „V100“ grafikos plokštės atmintyje sutalpinti 9B modelių rinkinį kartu su 16B parametrų teisėju (angl. judge - papildomas, stipresnis modelis, kuris kritikuoja ir balsuoja, bet pats nesiūlo sprendimų; žr. 2.2 poskyrį). Bendra modelių svorių apimtis siekia apie 24 GB, todėl papildomų kompromisų nereikia. Originaliame faile tarp OLLAMA_MAX_LOADED_MODELS ir OLLAMA_FLASH_ATTENTION priskyrimų esantis aštuonių eilučių komentaras paaiškina detalų VRAM apskaičiavimą bei „Flash Attention“ matematinę ekvivalenciją standartiniam dėmesio (angl. attention) mechanizmui; 3

fragmente jis praleistas, nes esmė pateikta aukštesniame tekste. Konkretūs SLURM scenarijai, „Singularity” konteinerizacija bei detalūs VRAM apskaičiavimai pristatomi 3.4.2 poskyryje.

2.2. Pagrindinis debatų ciklas

DebateOrchestrator klasė (src/core/orchestrator.py) realizuoja keturių fazių debatų ciklą, paremtą Du ir kt. [Dlt+23] kelių agentų debatų schema bei Liang ir kt. [Lhj+23] formalizuota „akis už akį” struktūra. Skirtingai nuo tų darbų, kuriuose visi agentai vykdo identišką užklausą, šio darbo realizacijoje agentai turi skirtingus vaidmenis (pasiūlytojas, kritikas, balsuotojas, neprivalomas teisėjas), o informacijos srautas tarp fazių yra tikslingai apribojamas, kad atitiktų 7-9 milijardų parametrų modelių pajėgumus (žr. 2.2.6 poskyrį).

2.2.1. Bendroji ciklo struktūra

Visa debatų sesija susideda iš pirmojo (pasiūlymo) raundo ir 1-N tolesnių debatų raundų; numatytoji maksimali raundų riba - penki, minimumas - du. Pirmajame raunde visi pasiūlytojų vaidmens agentai lygiagrečiai sugeneruoja pradinį sprendimą pagal užduoties aprašą, o tolesni raundai vykdo tris fazes - kritiką, reviziją ir balsavimą - kol pasiekiamas konsensusas arba išnaudojama raundų riba. Kiekvieno raundo pabaigoje konsensuso aptikimo modulis (žr. 2.3 poskyrį) tikrina, ar bent 60% agentų balsų sutampa; jei pasiekama riba ir minimalus raundų skaičius - debatas baigiamas. 4 kodo fragmente pateiktas supaprastintas pagrindinis ciklas iš run_debate metodo.

```
# src/core/orchestrator.py, supaprastintas pagrindinis ciklas
round1 = await self._run_proposal_round(debate)
debate.add_round(round1)
# Ankstyva pabaiga: idealus sprendimas jau pirmame raunde
if self.config.early_stop_on_perfect:
    perfect = self.consensus_detector.check_perfect_solution(round1.solutions)
    if perfect:
        debate.finalize(status=DebateStatus.EARLY_STOP, ...)
        return debate
# Debatų raundai 2..N
for round_num in range(2, self.config.max_rounds + 1):
    round_summary = await self._run_debate_round(debate, round_num)
    debate.add_round(round_summary)
    # Tikriname konsensusą po kiekvieno raundo
    if (round_summary.consensus_result.reached
        and round_num >= self.config.min_rounds):
        return debate
```

4 kodo fragmentas. Pagrindinis debatų ciklas.

Reikia atkreipti dėmesį į ankstyvosios pabaigos (angl. early stop) mechanizmą: jei jau pirmame raunde bent vienas agentas pateikia sprendimą, išlaikęs visus testus, debatas nutraukiamas - tolesni debatų raundai neatneštų papildomos vertės, o sutaupytų skaičiavimo sąnaudos yra reikšmingos paprastoms užduotims, kuriose 7B modeliai dažnai pasiekia 100% išlaikymą iš pirmo karto (žr. 4.1 poskyrį). Konsensuso tikrinimas su minimalia raundų riba užkerta kelią pernelyg ankstyvai pabaigai - net jei agentai sutaria po pirmojo balsavimo, sistema vis tiek vykdo dar bent vieną raundą, kad būtų galimybė kritikoms patikslinti sprendimą.

2.2.2. Pradinio pasiūlymo fazė

Pirmojo raundo metu visi pasiūlytojo vaidmens agentai gauna identišką užklausą, sukonstruotą iš užduoties aprašo, funkcijos parašo, apribojimų ir izoliuotoje aplinkoje vykdomų vienetinių testų sąrašo. Užklausos generuojamos lygiagrečiai per asyncio.gather, o gautos atsakymo eilutės yra apdorojamos kodo išskyrimo (angl. parsing) procedūros (žr. 2.6 poskyrį). Kiekvienas sėkmingai gautas sprendimas iškart paleidžiamas „pytest” izoliuotoje aplinkoje (žr.

2.7 poskyrį), o jo kokybė įvertinama „pylint“ ir „radon“ įrankiais; rezultatai prisegami prie sprendimo objekto ir naudojami vėlesnėse fazėse balsavimo balui koreguoti. Teisėjo vaidmens agentas pirmajame raunde nedalyvauja - jis prisijungia tik kritikos ir balsavimo fazėse.

2.2.3. Kritikos fazė

Antrame ir tolesniuose raunduose pirmiausia vykdoma kritikos fazė. Skirtingai nuo revizijos fazės (žr. 2.2.4 poskyrį), kritikos metu kiekvienas agentas mato visus kitų agentų sprendimus - tai sąmoningas projektavimo sprendimas, leidžiantis kritikams aptikti klaidas tiek silpnesnėse, tiek stipresnėse alternatyvose. Kiekvienas kritikas pateikia struktūrizuotą įvertinimą trim ašimis (taisyklingumas, efektyvumas, skaitomumas), išvardija konkrečias klaidas su jų aprašymu bei nurodo, ar jis pats būtų pasirengęs perimti (angl. adopt) kritikuojamą sprendimą. Kiekvienas agentas, įskaitant pasiūlytojus, dalyvauja kritikos fazėje, tačiau kritikuoja tik kitų agentų sprendimus - savo paties pasiūlymo niekada nevertina. Toks mechanizmas užtikrina, kad agentas nevertintų savo paties darbo, taip išvengiant Zheng ir kt. [Zcs+23] aprašyto palankumo sau šališkumo (angl. self-enhancement bias).

Kad agentai nekartotų jau anksčiau pateiktų pastabų, kiekvienai kritikos užklausiai pridedama trumpa ankstesnio raundo kritikų santrauka - šis mechanizmas, vadinamas kritikos istorija (angl. critique history), valdomas konfigūracijos parametru `critique_history`. Nors kodo numatytoji šio parametro reikšmė yra `False`, šio darbo HPC eksperimentuose jis buvo įjungtas. Santraukoje paliekamos pirmos trys kiekvieno kritiko nurodytos klaidos, sutrumpintos iki šešiasdešimties simbolių, kad būtų taupomas konteksto langas. Visi kritikų atsakymai išskaidomi į struktūrizuotus laukus reguliariųjų išraiškų (angl. regex) pagrindu (parsavimo logika detaliau aprašyta 2.6 poskyryje), o nesėkmingo išskyrimo atveju panaudojamos atsarginės numatytosios reikšmės.

2.2.4. Revizijos fazė

Surinktos kritikos sugrupuojamos pagal tikslinį agentą, ir kiekvienas pasiūlytojas gauna tik tas pastabas, kurios nukreiptos į jo paties sprendimą. Skirtingai nuo kritikos fazės pilnos matomumo politikos, revizijos užklausoje agentui rodomas tik geriausias kito agento sprendimas (angl. peer solution), išrenkamas pagal aukščiausią testų išlaikymo rodiklį (žr. 5 kodo fragmentą). Šis sprendimas valdomas konfigūracijos parametru `revision_show_all_solutions`, kurio numatytoji reikšmė yra `False`.

```
# src/core/prompts.py, (supaprastinta)
other_sols = [s for s in all_solutions if s.agent_id != own_solution.agent_id]
if show_all_solutions:
    # Visi alternatyvūs sprendimai (didesnis konteksto poreikis)
    for sol in sorted(other_sols, key=lambda s: s.pass_rate, reverse=True):
        ...
else:
    # Tik geriausias kito agento sprendimas (numatytas režimas 7B modeliams)
    best_other = max(other_sols, key=lambda s: s.pass_rate)
    ...
```

5 kodo fragmentas. Revizijos užklausoje rodomų alternatyvių sprendimų atrinkimas.

Geriausio kito agento sprendimo (toliau - „best-only“) režimas pasirinktas dėl trijų inžinerinių priežasčių. Pirmą, silpnos alternatyvos (pavyzdžiui, sprendimas, neišlaikęs nė vieno testo) yra nenaudingos perėmimo prasme - agentui nereikia matyti viso jų kodo, kad nuspręstų jų neperimti. Antra, informacija apie šių silpnų sprendimų problemas jau pasiekia agentą per kritikos santraukas, todėl jų pilno kodo pateikimas yra perteklinis. Trečia, preliminariniuose bandymuose pastebėta, kad 7-9 milijardų parametrų modeliams dažnai sunku sukurti hibridinį sprendimą, suderinantį kelių alternatyvų privalumus - jiems tai yra sudėtinga generavimo užduotis su didele konteksto apkrova, todėl perteklinis kontekstas gali veikti kaip triukšmas

(angl. context noise) ir mažinti revizijos kokybę. Šis stebėjimas dera su Smit ir kt. [Sdg+24] platesnėmis išvadomis, kad MAD sistemų našumas yra jautrus konteksto pateikimo strategijai, tinkamai ją parinkus, pasiekiamas reikšmingas pagerėjimas, priešingu atveju - našumas krenta.

Greta best-only režimo, revizijos fazėje veikia trys papildomi mechanizmai. Adaptyvi temperatūra (adaptive_temperature) padidina generavimo nepastovumą, jei agentų geriausias rezultatas tarp dviejų gretimų raundų nepagerėja - kiekvienas stagnacinis raundas prideda 0,15 prie pagrindinės temperatūros (numatytoji 0,4), su viršutine riba 0,9. Sistema taip pat palaiko pasirenkamą įvairių revizijos strategijų režimą (revision_strategy=diverse), kuris cikliniu principu (angl. round-robin) priskiria agentams skirtingas perfrazavimo perspektyvas; pagrindiniuose HPC eksperimentuose naudota numatytoji uniform strategija (detaliau - 2.4.4 poskyryje). Galiausiai, anti-regresijos mechanizmas (should_revert_revision) tikrina, ar revizuotas sprendimas nėra prastesnis už istorinį to paties agento maksimumą; jeigu testų išlaikymas mažesnis daugiau nei vienu testu arba kodas turi sintaksinę klaidą, sistema grąžina ankstesnį geriausią sprendimą (detalesnis šių mechanizmų aprašymas pateiktas 2.4 poskyryje).

2.2.5. Balsavimo fazė

Kiekvieno raundo pabaigoje visi agentai - taip pat ir teisėjas, jeigu jis sukonfigūruotas - balsuoja už geriausią revizuotą sprendimą. Priešingai nei revizijoje, balsavimo metu agentai mato visus revizuotus sprendimus, įskaitant savo paties darbą; tačiau balso svoris papildomai koreguojamas pagal sprendimo testų išlaikymą bei „pylint“ ir „radon“ metrikas, todėl agento šališkumas savo paties naudai yra iš dalies kompensuojamas (svertinis balsavimo algoritmas detaliai aprašytas 2.3 poskyryje). Kiekvienas balsas turi tris privalomus laukus: pasirinktas sprendimas (VOTE), pasitikėjimas intervale nuo 0,0 iki 1,0 (CONFIDENCE) ir pagrindimas (REASONING); šie laukai išskiriami iš modelio atsakymo reguliariosiomis išraiškomis, o nesėkmingo išskyrimo atveju balsas pažymimas vėliava parse_failed=True ir į konsensuso skaičiavimą neįtraukiamas.

2.2.6. Lygiagretus vykdymas ir matomumo lygiai

Visos trys debatų fazės naudoja asyncio.gather lygiagretumą - kiekvienoje fazėje visi atitinkami agentai užklaunami vienu metu, o tik gavus visus atsakymus pereinama prie kitos fazės. Toks sinchroninis fazių derinimas su asinchroniniu agentų užklausų vykdymu užtikrina, kad visi sprendimai yra vienodame raundo kontekste (visi mato vieni kitus), tačiau lėtaus modelio užklausa nestabdo greitesniųjų. Praktikoje tai reikšmingai sumažina viso debato trukmę lyginant su nuosekliu agentų užklausimu, kadangi skirtingo dydžio modeliai (pavyzdžiui, „Code Llama 7B“ ir „CodeGeeX4 9B“) generuoja atsakymus skirtingu tempu.

Apibendrinant ciklo informacijos srautą, sistemoje tikslingai naudojami trys skirtingi matomumo lygiai. Kritikos ir balsavimo fazėse kiekvienas agentas mato visus kitų agentų sprendimus - tai leidžia aptikti klaidas tiek silpnose, tiek stipriose alternatyvose bei priimti informuotą balsavimo sprendimą. Revizijos fazėje matomumas tikslingai apribojamas iki savo paties sprendimo ir aukščiausią testų išlaikymo rodiklį turinčio kito agento sprendimo (best-only režimas), taip mažinant konteksto triukšmą 7-9 milijardų parametrų modeliams. Šis trijų lygių derinys yra esminis šio darbo inžinerinis sprendimas, leidžiantis pritaikyti Du ir kt. [Dlt+23] originalią kelių agentų debatų schemą riboto konteksto modeliams.

2.2.7. Neprivalomas teisėjo vaidmuo

Be trijų pagrindinių pasiūlytojų, sistema palaiko neprivalomą ketvirtąjį agentą - teisėją (angl. judge), realizuojantį Zheng ir kt. [Zcs+23] aprašytą „LLM-as-Judge“ paradigmą. Priešingai nei pasiūlytojai, teisėjas nesiūlo savų sprendimų ir nedalyvauja revizijos fazėje - jo vaidmuo apsiriboja kritika ir balsavimu. Toks vaidmenų atskyrimas leidžia teisėjo poziciją užimti stipresniam modeliui (pavyzdžiui, kodo specializacijos „DeepSeek-Coder-V2“ 16B [Dee24] arba samprotavimo modeliui „DeepSeek-R1“ [Dee25]; konkretūs pasirinkimai

aprašyti 3.2 poskyryje), nereikalaujant iš jo generuoti pilnų kodo sprendimų - tai sumažina skaičiavimo sąnaudas, kartu suteikiant debatams išorinį, mažiau šališką vertinimo signalą.

Khan ir kt. [Khv+24] eksperimentai parodė, kad debate dviejų stipresnių modelių porą vertinant silpnesniam modeliui, vertintojo tikslumas pakyla nuo 48% iki 76% - šis principas perkeliamas į šio darbo schemą priešinga kryptimi: stipresnis teisėjas vertina silpnesnių pasiūlytojų debatą. Teisėjo įtakos eksperimentinis vertinimas (su teisėju vs. be teisėjo) detaliai pristatomas 4.4 poskyryje.

2.3. Konsensuso ir balsavimo algoritmas

Konsensuso aptikimas yra realizuotas atskirame modulyje `src/core/consensus.py`, atskirtame nuo orkestratoriaus, kad svertinio balsavimo logiką būtų galima testuoti izoliuotai. Šio darbo balsavimo algoritmas remiasi Du ir kt. [Dlt+23] daugumos balsavimo principu, papildytu trimis šio darbo metu pridėtomis korekcijomis: balsų svėrimu pagal agento nurodytą pasitikėjimą (angl. confidence), premija už testus išlaikiusius sprendimus ir premija už statinio kodo analizatoriaus įvertintą kodo kokybę. Pirmoji korekcija yra inžinerinis išplėtimas, neturintis tiesioginio analogo aprašytuose MAD darbuose, o antroji ir trečioji panaudoja tai, kad kodo srityje galima objektyviai patikrinti sprendimo savybes - galimybę, kurios nėra abstrakčiose samprotavimo užduotyse.

2.3.1. Balsavimo užklausos struktūra ir atsakymo išskyrimas

Kiekviename debatų raunde, po revizijos fazės, visi raunde dalyvaujantys agentai (įskaitant teisėjo agentą, jei jis sukonfigūruotas) gauna identišką balsavimo užklausą, kurioje pateikiami visi revizuoti to raundo sprendimai su prie kiekvieno sprendimo pridėtu testų išlaikymo rodikliu. Užklausa generuojama naudojant `SYSTEM_PROMPT_JUDGE` sistemos užklausą ir žemą 0,1 temperatūrą (`temperature=0.1`), kad balsavimas būtų kiek įmanoma deterministinis ir mažiau priklausytų nuo atrankos atsitiktinumo. Užklausoje aiškiai pažymima, kuris sprendimas priklauso pačiam balsuojančiam agentui (žyme „YOUR SOLUTION”), o po sprendimų sąrašo pateikiama griežta taisyklė, pagal kurią draudžiama balsuoti už savo paties sprendimą - šis apribojimas reikalingas tam, kad būtų sumažintas Zheng ir kt. [Zcs+23] aprašytas palankumo sau šališkumas (angl. self-enhancement bias) ir agentai būtų skatinami vertinti darbą kaip lygiavertę kolegų peržiūrą (angl. peer review).

Greta užklausos lygmens draudimo, sistemoje veikia antrasis - kodo lygmens - apsaugos sluoksnis. Jei agentas vis tiek pateikia balsą už savo paties sprendimą (užfiksuota silpnesnių 7B modelių elgsenoje), `_get_vote` metodas tokį balsą perrašo į `ABSTAIN` su atitinkamu įrašu žurnale (`src/core/orchestrator.py`). Analogiškas apsauginis perrašymas taikomas teisėjo agentui, kuris pirmajame raunde nedalyvauja kaip pasiūlytojas ir todėl neturi savo paties sprendimo, kurį galėtų ginti - jeigu modelis vis tiek pateikia `DEFEND` tipo balsą, jis perrašomas į `ABSTAIN`. Šie apsauginiai mechanizmai užtikrina, kad netipiški modelių atsakymai neiškreiptų konsensuso skaičiavimo.

Užklausa baigiasi instrukcija pateikti tiksliai tris eilutes - `VOTE: <skaičius>`, `CONFIDENCE: <0.0-1.0>` ir `REASONING: <vienas sakinys>`. Modelio atsakymas išskaidomas reguliariųjų išraiškų pagrindu (funkcija `parse_vote_response`); pagrindinis šablonas toleruoja „Markdown” formatavimą (pvz., `**VOTE:** 2`), skirtingiems separatoriams (dvitaškis, brūkšnelis, em-brūkšnis, rodyklė =>) ir buvo patikrintas trisdešimt vienu testavimo atveju (23 turinčiais atitikti ir 8 neturinčiais atitikti). Markdown tolerantiškumas buvo įvestas po to, kai realiaame transkripto pavyzdyje „Yi-Coder” pateikė `**CONFIDENCE:** 1.0`, o senesnė reguliarioji išraiška to neatpažino ir grąžino numatytąją 0,5 reikšmę. Nepavykus pritaikyti pirminio šablono, naudojami penki atsarginiai natūralios kalbos šablonai („I vote for solution N”, „solution N is the best” ir kiti); jei nesuveikia ir jie, balsas pažymimas vėliava `parse_failed=True` ir į konsensuso skaičiavimą neįtraukiamas. Pasitikėjimo lauko išskyrimui

skirtas atskiras atsarginis šablonas procentine forma (pvz., „80%” paverčiama į 0,8). Toks daugiapakopis atsakymo analizatorius (angl. parser) yra būtinas, nes 7-9 milijardų parametru modeliai dažnai nukrypsta nuo griežto formato (detalesiau apie užklausų atsakymų išskyrimą - 2.6 poskyryje).

2.3.2. Svertinis balsų skaičiavimas ir kokybės korekcijos

Surinkti balsai yra agreguojami pagal balso tipą - į konsensuso skaičiavimą įtraukiami ADOPT (perimti kito agento sprendimą) ir DEFEND (apginti savo paties sprendimą) tipų balsai, o ABSTAIN (susilaikyti) bei PROPOSE_NEW (siūlyti naują sprendimą) balsai praleidžiami. Kaip aprašyta 2.3.1 poskyryje, užklausoje agentams draudžiama balsuoti už savo paties sprendimą, todėl įprasto veikimo metu pateikiami tik ADOPT tipo balsai už kitų agentų sprendimus; DEFEND tipas yra atsarginė kategorija, kurią parseris atpažįsta retais atvejais, kai modelio atsakymas nukrypsta nuo griežto formato ir aiškiai pareiškia ginamąją poziciją. Jei pradinių balsų buvo daugiau nei tinkamų skaičiavimui (dėl išskyrimo nesėkmių ar perrašymo į ABSTAIN), į žurnalą įrašomas įspėjimas, leidžiantis eksperimentų metu atsekti modelių formatavimo problemas. Kiekvienas tinkamas balsas svėrimo metu padauginamas iš agento nurodyto pasitikėjimo, jei konfigūracijos parametras `weight_by_confidence` yra įjungtas (numatytoji reikšmė - True). Toks svėrimas leidžia agentams išreikšti netikrumą - silpnai pasirinktas variantas su CONFIDENCE: 0.3 daro mažesnę įtaką galutiniam rezultatui nei tvirtas pasirinkimas su CONFIDENCE: 0.9.

Pagrindinė svertinio balsavimo schema pateikta 6 kodo fragmente: kiekvienas sprendimo balsų svoris papildomai padauginamas iš dviejų objektyvių premijų - 50 procentų premijos už visų vienetinių testų išlaikymą ir iki 20 procentų premijos už „Pylint” kodo kokybės balą.

```
# src/core/consensus.py
if self.config.prefer_passing:
    sol_map = {s.id: s for s in solutions}
    for sol_id in vote_counts:
        sol = sol_map.get(sol_id)
        if sol:
            # 50% premija už visų testų išlaikymą
            if sol.execution_result and sol.execution_result.all_passed:
                vote_counts[sol_id] *= 1.5
            # iki 20% premijos už „Pylint” kodo kokybę
            if sol.quality_metrics:
                pylint_bonus = sol.quality_metrics.pylint_score / 10.0
                vote_counts[sol_id] *= (1 + pylint_bonus * 0.2)
```

6 kodo fragmentas. Premijos už testų išlaikymą ir kodo kokybę balsų skaičiavime.

Šios dvi premijos yra esminis šio darbo inžinerinis sprendimas - jos sušvelnina šališką pasiklovimą agentų subjektyvia nuomone ir pakelia verifikuojamų savybių (testų išlaikymas, kodo kokybė) svorį galutiniame rezultate. Didžiausią premiją gauna sprendimas, kuris vienu metu išlaiko visus testus ir turi maksimalų „Pylint” balą - tokiu atveju jo balsų svoris padauginamas iš 1,5 (testų premija) ir dar iš 1,2 (pilna kokybės premija), iš viso suteikiant 80 procentų pranašumą prieš tiek pat balsų surinkusį, tačiau testų ir kodo kokybės premijų negavusį sprendimą. „Pylint” balo normalizavimo faktorius 10 atitinka „Pylint” naudojamą dešimtbalę vertinimo skalę.

2.3.3. Konsensuso aptikimo kriterijai

Po svertinio skaičiavimo nustatomas pirmaujantis sprendimas - tas, kurio bendras balsų svoris yra didžiausias. Konsensusas laikomas pasiektu tik tada, kai vienu metu tenkinamos trys sąlygos: (1) pirmaujančio sprendimo svoris sudaro bent 60 procentų visų atiduotų balsų svorio (parametras `threshold=0.6`), (2) bent du nesvertiniai balsai atiduoti už šį sprendimą (parametras `min_votes=2`) ir (3) pirmaujantis sprendimas turi išlaikyti bent 50 procentų vienetinių testų

(parametras `min_pass_rate=0.5`). 7 kodo fragmente pateiktos numatytosios `ConsensusConfig` reikšmės.

```
# src/core/consensus.py
@dataclass
class ConsensusConfig:
    threshold: float = 0.6
    min_votes: int = 2
    weight_by_confidence: bool = True
    prefer_passing: bool = True
    min_pass_rate: float = 0.5
```

7 kodo fragmentas. Numatytoji konsensuso aptikimo konfigūracija.

Trijų sąlygų derinys yra sąmoningai konservatyvus. 60 procentų riba užtikrina, kad konsensuą palaikytų aiški dauguma, o ne minimalus pranašumas; minimalių dviejų balsų reikalavimas neleidžia paskelbti konsensuso, kai dauguma agentų balsavo už skirtingus sprendimus, o vienintelis aiškus balsas atsidūrė pirmaujančioje pozicijoje tik dėl pasitikėjimo svėrimo. Trečioji sąlyga - minimalus testų išlaikymas - yra svarbiausia: be jos sistema galėtų skelbti konsensuą tarp agentų, sutinkančių dėl iš esmės neteisingo sprendimo. Tokia rizika ypač aktuali ekstremalaus sudėtingumo užduotyse, kur visi agentai gali pasirinkti panašų klaidingą sprendimą - testų išlaikymo kontrolė neleidžia tokiam pseudo-konsensusui užbaigti debatų, ir sistema tęsia diskusiją, kol bent viena alternatyva pradės išlaikyti testus.

2.3.4. Ankstyvos pabaigos sąlygos

Greta įprastinio konsensuso po balsavimo, sistemoje veikia atskira ankstyvos pabaigos (angl. `early stop`) tikrinimo logika, realizuota `should_early_stop` metode. Ji aktyvuojama tik tada, kai jau pasiektas minimalus raundų skaičius (numatytasis `min_rounds=2`), ir nustato dvi galimas debato nutraukimo sąlygas. Pirmoji - tobulas sprendimas (angl. `perfect solution`): jei bent vienas iš dabartinių sprendimų išlaiko visus vienetinius testus, debatas nutraukiamas iškart, neatsižvelgiant į balsavimo rezultatus. Antroji - vienbalsis sutarimas: jei visi tinkami balsai atiduoti už tą patį sprendimą, debatas baigiamas, tačiau tik su papildoma sąlyga, kad sutarto sprendimo testų išlaikymas yra ne mažesnis nei `min_pass_rate=0.5`. Be šios sąlygos sistema galėtų ankstyvą pabaigą paskelbti dėl visiško agentų sutarimo dėl klaidingo sprendimo; todėl tokiu atveju ankstyva pabaiga netaikoma, į žurnalą įrašomas įspėjimas su konkrečia testų išlaikymo reikšme, o debatas tęsiamas įprastine tvarka.

Papildomai, jau pagrindiniame `run_debate` cikle veikia greitesnis tobulo sprendimo aptikimas pirmajame raunde (parametras `early_stop_on_perfect`, numatytoji reikšmė - `True`). Jei bent vienas pasiūlytojas iškart pateikia sprendimą su 100 procentų testų išlaikymu, sistema nutraukia debatą be tolesnių raundų, taip sutaupant skaičiavimo sąnaudų lengvosios užduotims. Šis mechanizmas yra svarbus debatų sistemos praktiškumui - eksperimentų rezultatai (žr. 4.1 poskyrį) rodo, kad lengvojo sudėtingumo užduotyse pasiūlytojai dažnai pateikia visus testus išlaikantį sprendimą jau pirmajame raunde, todėl ankstyvosios pabaigos mechanizmas leidžia reikšmingai sutaupyti skaičiavimo sąnaudas šios kategorijos užduotyse, neprarandant kokybės. Sudėtingesnių užduočių atveju debatų ciklas tęsiasi, ir jo nauda detalai vertinama 4 skyriuje.

Galutinis konsensuso ar ankstyvosios pabaigos rezultatas perduodamas atgal į orkestratorių, kuris užfiksuoja debato būseną (`CONSENSUS_REACHED`, `EARLY_STOP` arba `MAX_ROUNDS_REACHED`) ir kreipiasi į repozitorijos sluoksnį debato įrašui išsaugoti (žr. 2.9 poskyrį). Aprašyta balsavimo schema sudaro pagrindą 4 skyriaus eksperimentinei analizei. Joje atskirai vertinama trijų matavimo lygių, dviejų agentų rinkinio dydžių (7B ir 9B klasės) ir trijų teisėjo modelio tipų (be teisėjo, kodo specializacijos, samprotavimo) įtaka galutiniam testų išlaikymo rodikliui.

2.4. Atsakymų kokybės kontrolės ir revizijos stabilizavimo mechanizmai

Eksperimentai su 7-9 milijardų parametrų modeliais atskleidė kelis sisteminius netipinio elgesio atvejus, kurie be papildomų kontrolės mechanizmų galėtų reikšmingai iškreipti debatų rezultatus. Revizuotas sprendimas kartais tampa prastesnis už ankstesnę versiją, modelis gali pateikti klasių ar funkcijų „skeletą“ su tuščiomis metodų implementacijomis, atsakymas gali būti nutrauktas funkcijos viduryje dėl generavimo limito arba konteksto ribos pasiekimo, o kai kuriais atvejais modelis grąžina baitų lygiu tapatų ankstesnio sprendimo kodą, ignoruodamas gautą kritiką.

Todėl sistemoje įgyvendinta atsakymų kokybės kontrolės ir revizijos stabilizavimo mechanizmų grupė. Šiuos mechanizmus galima suskirstyti į dvi dalis. Pirmoji dalis apima apsauginius mechanizmus, kurie neleidžia prastesniems ar anomaliniams atsakymams pakeisti iki tol geriausios sprendimo versijos: anti-regresijos politiką, sutrumpinto atsakymo aptikimą, „tinginio“ kodo identifikavimą ir tapataus atsakymo diagnostiką. Antroji dalis apima generavimo valdymo mechanizmus, skirtus stagnacijai mažinti: adaptyvią temperatūrą, kritikos istoriją ir pasirenkamas revizijos strategijas.

Šie mechanizmai užtikrina, kad revizijos metu nebūtų prarandamas ankstesniuose raunduose pasiektas progresas, o stagnacijos ar anomalinių atsakymų atvejai būtų aiškiai matomi eksperimentų žurnaluose. Toks kontrolės sluoksnis reikalingas todėl, kad MAD sistemos yra jautrios realizacijos detalėms ir hiperparametrų pasirinkimui, kaip pabrėžia Smit ir kt. [Sdg+24]. Šiame poskyryje aptariama, kaip kiekvienas mechanizmas veikia, kokiomis sąlygomis jis aktyvuojamas ir kokią praktinę problemą sprendžia.

2.4.1. Anti-regresijos politika ir geriausio istorinio sprendimo grąžinimas

Įprastoje kelių agentų debatų schemos realizacijoje natūralus pasirinkimas būtų kiekviename raunde tęsti nuo paskutinio agento pateikto sprendimo. Tačiau praktikoje 7B modeliai kartais po sėkmingos pirmojo raundo įžvalgos vėlesniame raunde grąžina prastesnį kodą: pavyzdžiui, perima konkurento sprendimą su sintaksės klaida arba pateikia nepilnai sugeneruotą daugiafailės užduoties sprendimą. Kad toks regresinis pokytis nepersiduotų į vėlesnius raundus ir galutinį rezultatą, kiekvienam agentui per visus raundus saugomas geriausias istorinis sprendimas (angl. best historical).

Prieš kiekvieną revizijos užklausą iš šio žemėlapiu paimama didžiausią testų išlaikymo rodiklį turinti to agento sprendimo versija. Būtent ši versija, o ne paskutinio raundo rezultatas, tampa revizijos pradiniu tašku. Tokiu būdu agentas gali toliau tobulinti geriausią iki tol pasiektą sprendimą, o vienas nesėkmingas raundas automatiškai nepablogina visos debatų trajektorijos.

Po revizijos sprendimas paleidžiamas testų aplinkoje (žr. 2.7 poskyrį), o gautas rezultatas perduodamas funkcijai `should_revert_revision`. Ši funkcija pagal griežtai apibrėžtą sąlygų seką nusprendžia, ar palikti revizuotą sprendimą, ar grąžinti ankstesnį geriausią. Supaprastinta šios logikos schema pateikta 8 kodo fragmente.

```
# src/core/orchestrator.py, supaprastinta should_revert_revision logika
def should_revert_revision(prev_best, revised_result, revised_code=None):
    if prev_best is None or prev_best.execution_result is None:
        return (False, "no_prev_best")
    regression = prev_best.execution_result.tests_passed - revised_result.te
    sts_passed
    # 1. Sintaksinis ar struktūrinis lūžis
    if status in ("syntax_error", "runtime_error", "timeout") and tests_pass
    ed == 0:
        return (True, "syntax_broken")
    # 2. Sutrumpintas atsakymas
```

```

    if _looks_truncated(revised_code) and (regression >= 1 or pass_rate < 0.
5 * prev):
        return (True, "truncated_output")
    # 3. „Tinginio” kodo aptikimas
    if _looks_lazy(revised_code) and regression >= 0:
        return (True, "lazy_skeleton")
    # 4. Regresija bent dviem testais
    if regression >= 2:
        return (True, f"regressed_{regression}_tests")
    # 5. Regresija lygiai vienu testu paliekama tyrimui
    if regression == 1:
        return (False, "regressed_by_one_kept")
    return (False, "no_regression")

```

8 kodo fragmentas. Atstatymo sprendimo logikos supaprastintas vaizdas.

Originaliame faile `should_revert_revision` funkcijos dokumentacinėje eilutėje išsamiai aprašoma kiekvienos sąlygos motyvacija. Taip pat realizuotas papildomas atsarginis šablonas, skaitantis klaidos pranešimą tais atvejais, kai vykdytojo statusas nėra tiesiogiai aprašytas, pavyzdžiui, testuose su imituotu vykdymo rezultatu. Šios detalės į 8 kodo fragmentą neperkeliamos, nes jame siekiama parodyti tik pagrindinę sprendimo logiką.

Svarbiausias šios politikos bruožas yra vieno testo regresijos toleravimas: jei revizuotas sprendimas praranda tik vieną testą, jis neatstatomas automatiškai. Toks pasirinkimas yra sąmoninga nuokrypa nuo griežtos „bet kokia regresija - atstatymas” politikos, kuri pirminėje sistemos versijoje pernelyg anksti užrakindavo agentus lokaliuose optimumuose. Kai kurie pagrįsti perrašymai laikinai sumažindavo testų išlaikymą vienu testu, tačiau vėlesniuose raunduose sudarydavo sąlygas platesnei refaktORIZacijai ir geresniam sprendimui.

Ši problema susijusi su Liang ir kt. [Lhj+23] aprašyta „minčių degeneracijos” (angl. Degeneration-of-Thought, DoT) rizika: kai modelis pernelyg anksti įsitvirtina ties viena sprendimo kryptimi, vėlesnė refleksija nebepadeda sukurti naujų alternatyvų. Vieno testo regresijos toleravimas šioje sistemoje veikia kaip ribotas tyrinėjimo mechanizmas - agentui leidžiama atlikti mažą rizikingą žingsnį, jei jis potencialiai gali atvesti į geresnį sprendimą vėlesniuose raunduose.

Kai `should_revert_revision` grąžina `True`, orkestratoriaus kode (`orchestrator.py`) revizuotas sprendimas pakeičiamas ankstesniu geriausiu istoriniu sprendimu. Į žurnalą įrašomas išpėjimas su atstatymo priežastimi ir testų išlaikymo pokyčiu formatu „prieš → po”. Tai leidžia po eksperimento nustatyti, kuriose užduotyse ir kuriems agentams atstatymo mechanizmas suveikė, taip pat įvertinti, kurios anomalijų rūšys buvo dažniausios.

2.4.2. Modelių anomalijų aptikimas: sutrumpinimas, „tinginio” kodas ir tapatus atsakymas

Anti-regresijos politikoje naudojamos trys heuristinės anomalijų aptikimo funkcijos. Jos skirtos atvejams, kai testų rezultatas vienas pats nepakankamai tiksliai paaiškina, kodėl revizuotas sprendimas pablogėjo arba nustojo gerėti. Šios funkcijos leidžia atskirti įprastą nesėkmingą reviziją nuo specifinių modelio elgesio anomalijų: sutrumpinto atsakymo, „tinginio” kodo ir baitų lygiu tapataus atsakymo.

Pirmoji funkcija, `_looks_truncated`, aptinka atvejus, kai modelis generavimo metu pasiekia `num_predict` ribą arba kitą generavimo apribojimą ir nutraukia atsakymą sintaksinio konstrukto viduryje. Funkcija remiasi trimis alternatyviais požymiais. Pirmasis požymis yra skliaustų pusiausvyros pažeidimas, kai atvirų `{`, `[` arba `(` simbolių yra bent dviem daugiau nei uždaryų. Antrasis požymis yra atsakymo pabaiga raidiniu-skaitmeniniu simboliu kartu su bent viena neuždaryta struktūra, pavyzdžiui, realiame transkripte užfiksuotas fragmentas `self.precedence`

= {TokenType.PLUS: 1, ..., TokenT, kuriame paskutinis tokenas yra dalinai užrašytas identifikatorius. Trečiasis požymis yra paskutinėje eilutėje palikta valdymo konstrukcija be kūno, pavyzdžiui, else:, try: arba for x:.

Šie požymiai sąmoningai suformuluoti konservatyviai. Sintaksės klaidas jau aptinka `_BROKEN_STATUSES` mechanizmas, todėl `_looks_truncated` aktyvuojamas tik tada, kai kodas sintaksiškai priimamas, bet semantiškai lieka neužbaigtas. Sutrumpinimo aptikimas sukelia atstatymą tik tuo atveju, kai kartu stebima regresija bent vienu testu arba bendras testų išlaikymas tampa mažesnis nei pusė ankstesnio rezultato. Tokia sąlyga sumažina klaidingai teigiamo sutrumpinimo aptikimo riziką tais atvejais, kai modelis pateikia trumpesnį, bet vis tiek veikiantį kodą.

Antroji funkcija, `_looks_lazy`, identifikuoja atvejus, kai modelis pateikia klasių ir funkcijų skeletą su tuščiais arba vietą rezervuojančiais kūnais (`pass`, ... arba `raise NotImplementedError`), užuot pateikęs realią implementaciją. Toks elgesys ypač dažnai pasitaikė daugiakomponentėse užduotyse, pavyzdžiui, kuriant parserius, kompiliatorius ar interpretatorius. Tokiais atvejais modelis dažnai aprašo bendrą architektūrą, tačiau palieka esmines metodų implementacijas tuščias.

Kodas tokiu atveju gali būti sėkmingai sintaksiškai išanalizuotas ir paleistas, tačiau išlaikyti nulį testų. Jei toks atsakymas nebūtų sustabdytas, jis būtų perduotas į kitą kritikos ir revizijos ciklą ir galėtų iškreipti tolesnę debatų eigą. Heuristika naudoja Python `ast` modulį funkcijų deklaracijoms ir jų kūnams išskirti. Jei faile yra bent keturios `def` deklaracijos ir bent 25 procentai jų kūnų yra tušti arba vietą rezervuojantys, kodas žymimas kaip „tinginis“.

Slenksčiai kalibruoti pagal realias transkriptų situacijas: „Yi-Coder“ pateikė sprendimą su penkiomis funkcijomis ir dviem vietą rezervuojančiais kūnais (40 procentų), „CodeGeeX4“ - su keturiomis funkcijomis ir vienu tokiu kūnu (25 procentai, ribinis atvejis), o „Granite Code“ - su keturiomis funkcijomis ir trimis tokiais kūnais (75 procentai). Visi šie atvejai patenka į pasirinktą aptikimo ribą. Tuo tarpu teisėti atvejai, pavyzdžiui, abstrakčios bazinės klasės su konkrečiu poklasiu (3 funkcijos), nėra klasifikuojami kaip „tinginio“ kodas, nes nepasiekiamas minimalus funkcijų skaičius.

Daugiafailių sprendimų atveju, kurie žymimi `# FILE:` skirtukais, tikrinamas kiekvienas failas atskirai. Jei bent viename faile viršijamas slenkstis, visas sprendimas laikomas anomaliniu. Vis dėlto toks sprendimas atstatomas tik tada, kai kartu nėra aiškaus testų pagerėjimo. Ši sąlyga apsaugo nuo klaidingo atmetimo tais atvejais, kai dalinis perrašymas dar nėra visiškai užbaigtas, bet jau realiai pagerina testų išlaikymą.

Trečioji funkcija - baitų lygiu tapataus atsakymo aptikimas (`should_revert_revision:343-355`) - užfiksuoja atvejus, kai modelis revizijos metu grąžina savo ankstesnį kodą be jokių pakeitimų. Skirtingai nuo dviejų ankstesnių heuristikų, šis atvejis nesukelia atstatymo, nes sprendimas jau sutampa su ankstesniu geriausiu rezultatu, todėl atstatymas šiuo atveju nieko nepakeistų. Vis dėlto šis atvejis pažymimas atskira priežastimi `no_change_revision`, o orkestratoriaus kode (`orchestrator.py`) į žurnalą įrašomas įspėjimas su konkrečiu testų išlaikymo rodikliu.

Tokia diagnostika leidžia po eksperimento atskirti du iš pažiūros panašius atvejus. Pirmuoju atveju agento sprendimas iš tikrųjų stabilizavosi, nes buvo pasiektas pakankamai geras ar galutinis variantas. Antruoju atveju modelis ignoravo kritiką ir užstrigo stagnacijoje. Realiam transkripte buvo užfiksotas „CodeGeeX4“ elgesys `mini_database` ekstremalaus sudėtingumo užduotyje, kai modelis penkis raundus iš eilės grąžino baitų lygiu tapatų kodą - šis atvejis ir motyvavo atskiros diagnostikos pridėjimą.

2.4.3. Adaptyvi temperatūra ir kritikos istorija

Greta atstatymo politikos sistemoje veikia du papildomi mechanizmai, padedantys agentams ištrūkti iš stagnacijos ir nekartoti jau pateiktų pastabų: adaptyvi temperatūra ir kritikos istorija. Smit ir kt. [Sdg+24] pabrėžia, kad MAD sistemos yra jautrios hiperparametrų pasirinkimui ir sunkiai optimizuojamos. Šio darbo sistemoje į šią problemą reaguojama dviem dinaminiais valdymo elementais, kurie keičia debatų elgseną pagal stebimą progresą.

Pirmasis mechanizmas - adaptyvi temperatūra (`adaptive_temperature`, numatytoji reikšmė - `False`; HPC eksperimentuose įjungta) - automatiškai padidina revizijos užklausos generavimo temperatūrą, kai geriausias agentų testų išlaikymo rodiklis tarp dviejų gretimų raundų nepagerėja. Stagnacinių raundų skaičius skaičiuojamas atgal nuo paskutinio raundo, kol pasiekiamas pirmasis raundas, kuriame buvo užfiksuotas pagerėjimas. Gautas skaičius padauginamas iš 0,15 ir pridedamas prie pagrindinės revizijos temperatūros (numatytoji reikšmė - 0,4). Galutinė reikšmė apribojama 0,9 viršutine riba.

Pavyzdžiui, du iš eilės stagnaciniai raundai padidina temperatūrą nuo 0,4 iki 0,7. Tokiu būdu modeliui suteikiama daugiau laisvės nukrypti nuo ankstesnio sprendimo struktūros ir pasiūlyti alternatyvią taisymo kryptį. Šio mechanizmo motyvacija atitinka Wang ir kt. [Wws+22b] vidinės darnos schemoje taikomą įvairovės principą: vietoje vieno godžiosios atrankos rezultato generuojami keli skirtingi samprotavimo keliai. Skirtumas tas, kad šiame darbe įvairovės lygis yra ne fiksuotas, o dinaminis - jis didinamas tik tada, kai stebimas stagnacijos signalas. Logika realizuota orkestratoriuje (`orchestrator.py`).

Antrasis mechanizmas - kritikos istorija (`critique_history`, numatytoji reikšmė - `False`; HPC eksperimentuose įjungta) - kiekvieno raundo pradžioje sukuria trumpą ankstesnio raundo kritikos pastabų santrauką ir prideda ją tiek prie kritikos, tiek prie revizijos užklausų. Santrauka apima pirmąsias tris kiekvieno kritiko nurodytas klaidas, sutrumpintas iki šešiasdešimties simbolių, ir pateikiama formatu „kritikas → tikslinis agentas: klaidų sąrašas”.

Į kritikos užklausą pridedama instrukcija dėmesį telkti į dar neištaisytus klausimus arba aptikti naujas klaidas, nekartojant ankstesnių pastabų. Į revizijos užklausą įtraukiamas nurodymas sutelkti dėmesį į anksčiau neišspręstas problemas (`prompts.py`). Tokiu būdu kritikos istorija veikia kaip ribotos apimties kolektyvinė atmintis, leidžianti agentams negrįžti prie tų pačių pastabų kiekviename raunde.

Šis mechanizmas konceptiškai artimas Shinn ir kt. [Scb+23] „Reflexion” karkasui, kuriame agentai reflektuoja apie gautą grįžtamąjį ryšį ir saugo šią refleksiją epizodinės atminties buferyje, kad pagerintų sprendimų priėmimą vėlesniuose bandymuose. Šio darbo realizacijoje atminties turinį formuoja ne pats agentas, o orkestratorius, surinkdamas kitų agentų pateiktas kritikas. Toks centralizuotas atminties formavimas mažina pasikartojimą: be šio mechanizmo eksperimentuose buvo užfiksuota situacijų, kai keliuose raunduose kartojosi tos pačios kritikos pastabos apie tas pačias klaidas, o sprendimas negerėjo.

2.4.4. Įvairios revizijos strategijos

Liang ir kt. [Lhj+23] DMAD (angl. Divergent Multi-Agent Debate) schema siūlo agentų atsakymų įvairovę didinti pateikiant jiems skirtingus instrukcijų rinkinius. Šio darbo sistemoje šis principas realizuotas kaip pasirenkamas revizijos strategijų rinkinys, valdomas konfigūracijos parametru `revision_strategy`. Galimos reikšmės yra `uniform`, kai visi agentai gauna tą pačią revizijos užklausą, ir `diverse`, kai kiekvienam agentui cikliniu principu (angl. `round-robin`) priskiriama atskira strategija.

Strategijos apibrėžtos `prompts.py` faile, struktūroje `REVISION_STRATEGIES`. Rinkinį sudaro keturi variantai: žingsnis po žingsnio kodo vykdymo analizė (`step_by_step`), testais grįstas minimalus taisymas (`test_driven`), supaprastinimas perrašant sprendimą iš naujo (`simplify`) ir kraštinių atvejų analizė (`edge_cases`). Pirmasis ir trečiasis variantai yra iš esmės skirtingi: `step_by_step` strategija skatina išlaikyti esamą struktūrą ir taisyti klaidingą logiką, o `simplify` strategija skatina perrašyti sprendimą nuo nulio paprasčiausia įmanoma forma.

Strategijos agentams priskiriamos pagal `STRATEGY_ORDER` sąrašo tvarką. Be to, vartotojas gali rankiniu būdu nurodyti konkrečias strategijas konkreitiems agentams per `agent_strategies` žodyną; šis nustatymas turi pirmenybę prieš automatinę ciklinę rotaciją. Tokia konfigūracija leidžia lanksčiai eksperimentuoti su skirtingais agentų elgesio profiliais nekeičiant pagrindinės debatų ciklo logikos.

DMAD strategijų įjungimas šiame darbe laikomas pasirenkama eksperimento konfigūracija. Pagrindiniai HPC eksperimentai naudoja numatytąją uniform strategiją, todėl diverse režimas nėra pagrindinis šio darbo vertinimo objektas. Vis dėlto infrastruktūra paliekama paruošta vėlesniems palyginamiesiems eksperimentams, kuriuose būtų galima atskirai įvertinti, ar skirtingų revizijos strategijų taikymas pagerina testų išlaikymo rodiklį, palyginti su vienoda revizijos užklausa visiems agentams.

Apibendrinant, šiame poskyryje aprašyti kontrolės mechanizmai atlieka dvi funkcijas: apsaugo debatų ciklą nuo regresijos ir anomalinių modelių atsakymų bei suteikia sistemai priemonių mažinti stagnaciją. Anti-regresijos politika saugo geriausią pasiektą sprendimą, anomalijų aptikimo heuristikos leidžia identifikuoti sutrumpintą, „tinginį“ ar tapatų kodą, o adaptyvi temperatūra ir kritikos istorija padeda agentams ieškoti naujų taisymo krypčių. Toks daugiasluoksnis kontrolės sluoksnis yra praktinė sąlyga, leidžianti kelių agentų debatų schemą pritaikyti riboto pajėgumo modeliams, kuriems tipinės klasikinės MAD schemos prielaidos (pavyzdžiui, kad agentai visada konstruktyviai panaudoja gautą kritiką) ne visada galioja. Detalesnis trijų matomumo lygių ir agentų rinkinio sudėties poveikis testų išlaikymo rodikliui pristatomas 4 skyriaus eksperimentinėje analizėje.

2.5. Daugiafailių užduočių apdorojimas ir dalinis generavimas

Realistiškos programinės įrangos užduotys retai apsiriboja vienu failu - net paprastas išraiškų skaičiuotuvas natūraliai dalijasi į tokenizatoriaus ir vertintojo modulius, o sudėtingesnės sistemos apima dešimtis tarpusavyje susijusių komponentų. Jimenez ir kt. [Jyw+24] sudarytame „SWE-bench“ rinkinyje, atspindinčiame realias „GitHub“ problemas, autoriai pažymi, kad „problemos sprendimas dažnai reikalauja suprasti ir suderinti pakeitimus tarp kelių funkcijų, klasių ir net failų vienu metu“ (angl. „Resolving issues in SWE-bench frequently requires understanding and coordinating changes across multiple functions, classes, and even files simultaneously“). Šio darbo užduočių rinkinyje visos keturiasdešimt ekstremalaus sudėtingumo užduočių (`tasks2/extreme/` kataloge) reikalauja sukurti du ar daugiau modulių su tarpmoduliniais importais - pavyzdžiui, „calculator“ (`tokenizer.py` ir `evaluator.py`), „mini_database“ (`storage.py` ir `database.py`), „graph_algorithms“ (trijų modulių derinys) bei „http_client“ (trijų modulių derinys). Šio poskyrio paskirtis - paaiškinti, kaip sistema apdoroja tokias užduotis, kai vienos užklauskos generavimo galimybės yra ribotos 7-9 milijardų parametrų modeliuose.

2.5.1. Daugiafailių užduočių struktūra

Sistemoje užduotis (Task klasė, `src/models/solution.py`) aprašoma JSON formatu su trimis daugiafailei užduočiai specifiniais laukais. Laukas `required_files` išvardija failus, kuriuos modelis privalo sugeneruoti (pavyzdžiui, [`“tokenizer.py”`, `“evaluator.py”`]); laukas `test_imports` nurodo, kokius simbolius testavimo aplinka turi importuoti iš sugeneruotų modulių (pavyzdžiui, [`“from evaluator import ExpressionCalculator”`]); o laukas `signature` pateikia

pilnas funkcijų ir klasių sąsajas iš visų failų, kad modelis galėtų teisingai derinti tarpmodulinius importus. Daugiafailės užduoties požymis pateiktas kaip `Task.is_multi_file` savybė, kuri grąžina `True`, jei `required_files` sąrašas nėra tuščias.

Konkretus pavyzdys yra ekstremalaus sudėtingumo užduotis „calculator” (`tasks2/extreme/calculator.json`), kurioje modelis turi sukurti du modulius: `tokenizer.py`, kuriame `Tokenizer` klasė išraiškos eilutę paverčia žetonų sąrašu, ir `evaluator.py`, kuris importuoja `tokenizer.py` apibrėžtus komponentus ir realizuoja `ExpressionCalculator.evaluate()` metodą su operatorių pirmumo taisyklėmis. Aštuoni vienetiniai testai apima paprastas aritmetines operacijas, skliaustelius, įdėtus skliaustelius, unarųjį minusą, sveikųjų skaičių dalybą bei sudėtingesnes išraiškas. Tokia struktūra ne visada gali būti patikimai apdorota vienos užklauskos generavimo režimu net naudojant išplėstą 32 768 žetonų konteksto langą (žr. 2.1.4 poskyrį) - dviejų visiškai įgyvendintų modulių su klasių hierarchija ir importais bendra išvesties apimtis kartu su užduoties aprašu, sąsajomis ir kritikomis (revizijos atveju) gali artėti prie konteksto ribos. Šis scenarijus buvo viena iš pagrindinių nepastebimo sutrumpinimo priežasčių ekstremalaus sudėtingumo užduotyse, kol nebuvo įdiegtas dalinis generavimas.

2.5.2. Dalinio generavimo schema

Sprendimo esmė - kiekvieną failą generuoti atskira DKM užklausa, perduodant jau sugeneruotų failų kodą kaip kontekstą paskesnėms užklauskoms. Ši schema, kodo bazėje vadinama `chunked generation` (lietuviškai - dalinis generavimas), realizuota tiek pradinio pasiūlymo (`_get_proposal`), tiek revizijos (`_get_revision`) fazėms. Pagrindinis dalinio generavimo ciklas pateiktas 9 kodo fragmente.

```
# src/core/orchestrator.py, _get_proposal supaprastinta dalinio generavimo logika
if task.is_multi_file:
    code_files: dict[str, str] = {}
    was_truncated = False
    for filename in task.required_files:
        prompt = build_chunked_file_proposal_prompt(
            task, filename, already_generated=code_files or None,
        )
        response = await self.llm_client.generate(agent.model, request)
        if self._is_truncated(response):
            was_truncated = True
            logger.warning("Agent %s proposal for %s TRUNCATED ...", agent.id, filename)
        file_code = extract_code_from_response(response.content)
        code_files[filename] = file_code
```

9 kodo fragmentas. Pagrindinė dalinio generavimo kilpa pradinio pasiūlymo fazėje.

Funkcija `build_chunked_file_proposal_prompt` sukuria užklausą kiekvienam failui. Užklausa apima užduoties aprašą, pilnas visų failų sąsajas (kad modelis žinotų, kokius simbolius eksportuoja kiti moduliai), apribojimus bei specifinę instrukciją rašyti tik nurodyto failo turinį. Jei yra jau sugeneruotų failų, jų kodas pridedamas atskirame skyriuje „Already Implemented Files” su aiškiu nurodymu, kad rašomo failo importai turi atitikti šių failų sąsajas. Toks projektavimas užtikrina dvi savybes: pirma, kiekvienas DKM iškvietimas dirba su gerokai mažesne išvesties apimtimi nei pilnas sprendimas, todėl konteksto lango išnaudojimo rizika reikšmingai sumažėja; antra, tarpmoduliniai importai išlieka nuoseklūs, nes vėlesni failai mato anksčiau sugeneruotų failų pilną kodą, o ne tik abstrakčią sąsajos deklaraciją. Sutrumpinimo aptikimas (`done_reason == "length"` patikrinimas) taikomas kiekvienam iškvietimui atskirai, todėl, jei vienas failas buvo sutrumpintas, į žurnalą įrašoma atskira įspėjimo eilutė su konkretaus failo vardu - tai leidžia po eksperimento tiksliai nustatyti, kuriame faile įvyko sutrumpinimas.

Revizijos fazėje dalinio generavimo logika papildoma kritikų bei testų grįžamojo ryšio (angl. test feedback) perdavimu - kiekvienas failas perrašomas turint pilną testų išlaikymo informaciją ir visas kritikas apie ankstesnę agento sprendimo versiją (build_chunked_file_revision_prompt). Pati užklausa rodo modeliui visą dabartinį sprendimą su visais failais, bet prašo grąžinti tik nurodyto vieno failo turinį. Galutinis sprendimas formuojamas sujungiant atskirus failus į vieną tekstinį atvaizdą su # FILE: <vardas> skirtukais (orchestrator.py), kuris naudojamas tiek vidiniam sprendimų palyginimui (tarp agentų), tiek debatų protokoluose, tiek anomalijų aptikimo heuristikose (žr. 2.4.2 poskyrį, kuriame aprašyta, kaip _looks_lazy apdoroja daugiafailį kodą).

2.5.3. Atsakymo kodo išskyrimas iš modelio atsakymo

Net su tinkamai sukonstruota užklausa modelis gali atsakymo formatu nukrypti nuo numatyto šablono - tai ypač pasireiškia mažesniuose 7B modeliuose, kurie kartais pamiršta # FILE: antraštę, kartais grąžina kelis ```python blokus be aiškos žymos, kartais sukeičia atsakymo skyrius vietomis. Kad sistema būtų atspari tokiai įvairovei, extract_multi_file_code_from_response funkcija taiko keturių lygių daugiapakopį išskyrimą.

Pirmasis lygis ieško aiškiai pažymėtų blokų pagal šabloną # FILE: filename.py su iš karto po jo einančiu python bloku - tai oficialus sistemos formatas, kurio laikytis modelio prašoma užklausoje. Antrasis lygis tikrina alternatyvią „Markdown“ antraštę `### filename.py` arba `## filename.py`, kurią kartais sugeneruoja modeliai, įpratę pateikti atsakymus su antraštės sintakse. Trečiasis lygis yra atsarginė strategija: jei vis dar trūksta kurio nors reikalingo failo, sistema ima nepažymėtus python blokus atsakyme ir priskiria juos trūkstamiems failams pagal required_files sąrašo tvarką - prielaida tokia, kad modelis pateikė reikalingą skaičių blokų ta pačia tvarka, kuria jie buvo prašyti. Ketvirtasis lygis apdoroja netipinį atvejį, kai modelis naudoja # FILE: skirtukus, bet nepateikia ```python Markdown kodo bloko žymos - tokiu atveju kodas išskiriamas tiesiogiai iš teksto tarp dviejų gretimų skirtukų. Tokia strategijų kaskada didina sistemos atsparumą skirtingiems modelių formatavimo įpročiams. Nesėkmės atveju, kai nepavyksta išskirti visų reikalingų failų, sprendimas vėlesniame vykdymo arba testavimo etape pažymimas kaip nesėkmingas; revizijos fazėje toks rezultatas gali aktyvuoti anti-regresijos politiką ir būti pakeistas ankstesniu geriausiu sprendimu (žr. 2.4.1 poskyrį).

Galutinis sugeneruotas daugiafailis sprendimas perduodamas kodo vykdytojui, kuris kiekvieną failą įrašo į izoliuotą laikiną katalogą, taiko santykinį importų pataisymą (_fix_relative_imports) ir testų funkcijų pašalinimą (_strip_test_functions), o testavimo failas konstruojamas naudojant užduoties test_imports lauko reikšmes. Detalus daugiafailių užduočių vykdymo aprašymas - 2.7 poskyryje.

2.6. Užklausų konstravimas ir DKM atsakymų išskyrimas

Visa sąveika su kalbos modeliais sistemoje organizuojama per dvi tarpusavyje susijusias procedūras: užklausų konstravimą prieš DKM iškvietimą ir atsakymų išskyrimą (angl. parsing) po jo. Abi procedūros centralizuotos faile src/core/prompts.py, kuriame sutelkti keturių fazių užklausų šablonai, trys atskiri sistemos užklausų variantai bei keturios pagrindinės atsakymų analizavimo funkcijos. Šio poskyrio paskirtis - paaiškinti, kaip skirtingiems vaidmenims sukonfigūruojama agento elgsena, kokia struktūra taikoma vartotojo užklausoje ir kokie daugiapakopiai išskyrimo mechanizmai naudojami siekiant patikimai apdoroti 7-9 milijardų parametrų modelių formatavimo įvairovę.

2.6.1. Sistemos užklausa: trijų vaidmenų atskyrimas

Kiekviena DKM užklausa apima ne tik vartotojo žinutę su užduoties duomenimis, bet ir sistemos užklausa (angl. system prompt), nustatančią bendrą agento elgsenos profilį. Sistemoje apibrėžiami trys atskiri sistemos užklausų variantai, kurių kiekvienas naudojamas skirtingoje debatų fazėje (10 kodo fragmente pateikta sutrumpinta jų struktūra).

```
# src/core/prompts.py, sistemos užklausų antraštės (sutrumpintos)
SYSTEM_PROMPT_CODER = """You are an expert Python programmer participating
in a code review debate. Your goal is to write correct, efficient, and
readable Python code. [...] Always wrap your code in ```python and ```
markers."""
SYSTEM_PROMPT_CRITIC = """You are an expert code reviewer participating in
a code review debate. Your PRIMARY goal is to find bugs that cause test
failures and explain HOW to fix them. [...] PRIORITY: If a solution fails
tests, spend most of your analysis on correctness bugs."""
SYSTEM_PROMPT_JUDGE = """You are an impartial judge in a code review debate.
Your goal is to evaluate solutions fairly based on multiple criteria.
[...] 1. Correctness (40%), 2. Efficiency (25%), 3. Readability (20%),
4. Style (15%)."""
```

10 kodo fragmentas. Trijų sistemos užklausų variantai (sutrumpintos pirmosios eilutės iš prompts.py).

SYSTEM_PROMPT_CODER naudojamas pradinio pasiūlymo ir revizijos fazėse - jis nukreipia modelį į pilno, veikiančio kodo generavimą su konkrečiomis instrukcijomis dėl PEP 8 stiliaus, tipų žymėjimo, kraštinių atvejų apdorojimo bei privalomo ```python Markdown kodo bloko žymeklio. SYSTEM_PROMPT_CRITIC aktyvuojamas kritikos fazėje ir keičia modelio prioritetus: vietoj kodo generavimo akcentuojamas klaidų ieškojimas ir konkrečių taisymo pasiūlymų formulavimas, taikant aiškią prioritetų nustatymo taisyklę „jei sprendimas neišlaiko testų, didžioji analizės dalis turi būti skirta korektiškumo klaidoms“. SYSTEM_PROMPT_JUDGE taikomas balsavimo fazėje (žr. 2.3 poskyrį) ir formuoja bešališkos peržiūros elgseną su aiškiais kokybės kriterijų svoriais (korektiškumas - 40%, efektyvumas - 25%, skaitomumas - 20%, stilius - 15%). Atskirų sistemos užklausų naudojimas užtikrina, kad ta pati modelio instancija skirtinguose debatų etapuose veiktų pagal skirtingą elgsenos profilį, neperkraunant vartotojo užklauso pakartotinėmis vaidmens instrukcijomis.

2.6.2. Vartotojo užklausų konstravimas

Vartotojo užklauso formuojamos atskirais konstruktoriais kiekvienai debatų fazei: build_proposal_prompt (pradinis pasiūlymas), build_critique_prompt (kritika), build_revision_prompt (revizija) ir build_voting_prompt (balsavimas). Visi konstruktoriai laikosi vienodos struktūros - užduoties pavadinimas, aprašymas, funkcijų sąsajos, apribojimai, kontekstinė informacija (ankstesnio agento sprendimas, gautos kritikos, alternatyvūs sprendimai, jei taikoma) bei galutinis instrukcijų blokas su griežtu išvesties formato aprašu.

Ypač pažymėtinas pradinio pasiūlymo konstruktorius (build_proposal_prompt), kuriame į instrukcijų bloką įtraukti aiškūs nepageidaujamos elgsenos draudimai: nenaudoti pass, ..., # TODO arba raise NotImplementedError kaip funkcijos kūno, nepateikti tuščių klasių ar metodų kontūrų ir užtikrinti, kad kiekviena funkcija turėtų realią, vykdomą logiką, duodančią teisingą rezultatą. Šie draudimai veikia kaip preliminarus apsauginis sluoksnis prieš „tinginio“ kodo generavimą (žr. 2.4.2 poskyrį) - jie sumažina tokio kodo pasirodymo tikimybę, o jei modelis vis tiek jį sugeneruoja, toks atvejis vėlesniuose etapuose gali būti aptiktas _looks_lazy heuristika ir įtrauktas į anti-regresijos politiką.

Kritikos užklausa (build_critique_prompt) papildomai įtraukia detalų išvesties formato šabloną su privalomais skyriais kiekvienam alternatyviam sprendimui („Solution N Analysis“, „Bugs Found“, „Ratings“, „Improvements“, „Would Adopt“), nes būtent šis griežtas formatas vėliau leidžia patikimai išskirti struktūrizuotą atsakymo turinį (žr. 2.6.4 poskyrį). Užklausoje taip pat aiškiai reikalaujama pagrįsti kiekvieną klaidą konkrečiu nepavykusio testo pavadinimu arba kodo eilute (angl. „Ground every bug report in concrete evidence - cite a FAILING test name [...] Do NOT invent bugs that are not supported by a failing test or a real code defect“). Šis reikalavimas sprendžia nepagrįstų arba haliucinuojamų klaidų identifikavimo problemą -

atvejus, kai modelis pateikia spekuliatyvias arba neegzistuojančias klaidas, nesiremiančias nei testų rezultatais, nei realiu kodo defektu.

2.6.3. Kodo išskyrimas iš modelio atsakymo

Sistemos kodo bazėje yra dvi atskiros kodo išskyrimo funkcijos: `extract_code_from_response` vieno failo užduotims ir `extract_multi_file_code_from_response` daugiafailėms užduotims (žr. 2.5.3 poskyrį). Abi funkcijos pirmiausia paleidžia atsakymą per `_strip_special_tokens` filtrą, kuris pašalina tokenizatoriaus kontrolinius simbolius, kartais liekančius modelio išvestyje.

Kontrolinių simbolių problema yra reali - skirtingi modeliai naudoja skirtingus tokenizatorius, ir kai kuriais atvejais šie simboliai patenka į generuojamą tekstą. Sistema apdoroja keturiolika tokių šablonų: „DeepSeek” šeimos žetonus (`<|begin_of_sentence|>`, `<|end_of_sentence|>`, `<|fim_begin|>`, `<|fim_hole|>`, `<|fim_end|>` su pilno pločio vertikalaus brūkšnio simboliais), „GPT” šeimos žetoną (`<|endoftext|>`), „ChatML” formato žetonus, naudojamus „Qwen” šeimoje (`<|im_start|>`, `<|im_end|>`), pildymo viduryje (angl. fill-in-the-middle, FIM) žetonus (`<|fim_prefix|>`, `<|fim_middle|>`, `<|fim_suffix|>`), failo skirtuko žetoną (`<|file_separator|>`) bei bendro pobūdžio sekos pradžios ir pabaigos žetonus (`<s>`, `</s>`). Tokio kontrolinių simbolių patekimo į išvestį aptikimas svarbus, nes neapdoroti kontroliniai simboliai gali pažeisti AST analizatorių - kodo bazės komentaras nurodo konkretų pavyzdį, kai į kodo vidurį įterptas „DeepSeek” sekos pradžios žetonas eilutę `return <|begin_of_sentence|>0` paverčia sintaksiškai netaisyklingu Python kodu. Funkcija žurnale registruoja, kiek tokių simbolių buvo pašalinta, todėl eksperimentų metu galima atsekti, kurių modelių išvestyje tokie simboliai pasitaiko dažniausiai.

Po kontrolinių simbolių pašalinimo, vieno failo kodo išskyrimas vyksta keturių lygių kaskada. Pirmasis lygis ieško ````python ... ```` blokų ir, jei jų rasta keli, grąžina ilgiausią (prielaida - ilgiausias blokas yra pilnas sprendimas, o trumpesni gali būti pavyzdiniai fragmentai). Antrasis lygis tikrina bendrinius ```` ... ```` blokus be kalbos žymos ir tarp jų atrenka tuos, kurie atrodo kaip Python kodas pagal raktažodžių heuristiką (`_looks_like_python`). Trečiasis lygis bando aptikti Python kodą eilutė po eilutės, pradedant nuo eilutės, kuri prasideda `def`, `class`, `import`, `from`, `if`, `for`, `while`, `return` arba eilutės įtrauka - ši strategija reikalinga atvejais, kai modelis nepateikė Markdown kodo bloko žymų. Ketvirtasis lygis grąžina tuščią eilutę su įspėjimu žurnale; tuščias kodas pateikiamas testavimo vykdytojui, kuris jį užfiksuoja kaip nesėkmingą sprendimą, dažniausiai sintaksės klaidos kategorijoje. Revizijos fazėje toks rezultatas gali aktyvuoti anti-regresijos politiką (žr. 2.4.1 poskyrį) ir būti pakeistas ankstesniu geriausiu istoriniu sprendimu.

2.6.4. Kritikos atsakymų išskyrimas ir įvertinimų analizė

Skirtingai nuo kodo, kritikos ir balsavimo atsakymai yra struktūrizuotas tekstas su keliais privalomais laukais. Šių atsakymų išskyrimas centralizuotas `parse_critique_response` funkcijoje, kuri pirmiausia suskaido atsakymą į skyrius pagal šabloną `Solution N` (kur `N` - sprendimo numeris), o paskui kiekvieną skyrių analizuoja atskirai.

Klaidų sąrašas išskiriamas dviem strategijomis. Pirmoji strategija - skyrių žinanti (angl. scope-aware) - ieško „Bugs Found” žymos ir iš jos srities ima ženkliukais (- arba *) pradėtas eilutes; ši strategija yra patikimesnė, bet reikalauja, kad modelis išlaikytų griežtą formatą. Antroji strategija - atsarginė - taikoma tik tuo atveju, kai „Bugs Found” žymos nerasta; ji ieško raktažodžių „bug”, „error”, „issue”, „problem” su iš karto po jų einančia aprašymo eilute. Antros strategijos tikslumas yra mažesnis (raktažodžiai gali atsirasti ir kitose vietose, pavyzdžiui, įvertinimų pagrindimuose), todėl jai taikomi griežtesni filtrai (minimalus aprašymo ilgis dešimt simbolių, „rating” žodžio neturėjimas). Klaidos su aprašymais „none”, „none

found”, „no bugs”, „n/a”, „no bugs found” automatiškai filtruojamos kaip pripažinimas, kad modelis klaidų nerado.

Įvertinimai trimis ašimis - taisyklingumas, efektyvumas, skaitomumas - išskiriami atskira funkcija `_parse_ratings`, kuri ieško šablonų correctness: N, efficiency: N, readability: N ir riboja gautas reikšmes intervalu nuo 1 iki 10. Funkcija grąžina ne tik patį įvertinimų žodyną, bet ir loginę vėliavą `ratings_parsed`, nurodančią, ar visi trys įvertinimai buvo iš tikrųjų rasti. Nors Critique.ratings_parsed lauko (`models/critique.py`) numatytoji reikšmė yra True - nes įprastu atveju tikimasi realiai pateiktų įvertinimų - nesėkmingo išskyrimo atveju funkcija šią vėliavą nustato į False, taip atskiriant modelio pateiktus įvertinimus nuo atsarginių numatytųjų reikšmių. Tai svarbu vėlesnei debatų protokolų analizei, nes leidžia identifikuoti nestruktūrizuotus arba tik iš dalies struktūrizuotus kritikos atsakymus ir įvertinti modelių formataavimo patikimumą eksperimentų metu.

Sprendimas dėl perėmimo (angl. adopt) išskiriamas paprastu reguliariosios išraiškos šablonu `would\s*adopt[*:\s]*(yes|no)`, atspariu „Markdown” paryškinimo simboliams aplink reikšmę. Jei kritikos atsakyme nepavyko aptikti jokios struktūros (skyrių skaidymo šablonas nerado nė vieno „Solution N” žymeklio), funkcija sukuria numatytąją kritiką su numatytaisiais 5 balų įvertinimais visose ašyse, tuščiu klaidų sąrašu ir `ratings_parsed=False` vėliava - tai užtikrina, kad debatų ciklas nesustotų dėl vieno netipiško atsakymo, o eksperimentų metu būtų galima atsekti, kurie modeliai dažniausiai sugeneruoja nestruktūrizuotus atsakymus. Balsavimo atsakymų išskyrimas (`parse_vote_response`) yra atskira funkcija, kurios daugiapakopė išskyrimo logika detalai aprašyta 2.3.1 poskyryje.

2.7. Užduočių vykdymas izoliuotoje aplinkoje ir kodo kokybės analizė

Kiekvienas debatų ciklo metu sugeneruotas sprendimas turi būti įvertintas dviem nepriklausomais kriterijais: funkcinio korektiškumu (ar sprendimas išlaiko užduoties vienetinius testus) ir struktūrine kokybe (ar kodas atitinka skaitomumo bei sudėtingumo standartus). Šio darbo sistemoje abi dimensijos vertinamos `src/core/executor.py` modulyje, kuriame įgyvendintos dvi pagrindinės dalys: `CodeExecutor` klasė izoliuotam testų vykdymui ir `CodeQualityAnalyzer` klasė statinei kodo analizei. Tokio dvigubo vertinimo poreikį pagrindžia Zheng ir kt. [Zcm24] tyrimas, kurio autoriai pažymi, kad „dabartiniai etalonai pirmiausia vertina DKM generuojamo kodo tikslumą, tačiau neatsižvelgia į kitas svarbias dimensijas, kurios taip pat reikšmingai paveikia kodo kokybę realiaje programinės įrangos kūrime” (angl. „current benchmarks primarily assess the accuracy of LLM-generated code, while neglecting other critical dimensions that also significantly impact code quality in real-world development”). Šis poskyris aptaria, kaip sistema realizuoja izoliuotą testų vykdymą, kokie apsauginiai mechanizmai neleidžia netipinei DKM elgsenai iškreipti rezultatų, kaip rezultatų podėliavimas sumažina pakartotinio vykdymo sąnaudas ir kaip kokybės metrikos panaudojamos vertinimo procese.

2.7.1. Izoliuoto vykdymo aplinka

`CodeExecutor` klasė (`executor.py`) kiekvienam testavimui sukuria atskirą laikiną katalogą naudodama `tempfile.TemporaryDirectory` kontekstinį valdiklį - tai užtikrina, kad skirtingų agentų sprendimai vienas kito netrikdytų, o po kiekvieno vykdymo katalogas su jo turiniu būtų automatiškai pašalintas. Į laikiną katalogą įrašomas sprendimo kodas (vieno failo užduočiai - kaip `solution.py`, daugiafailei užduočiai - kaip atskiri failai, žr. 2.5.3 poskyrį) bei testavimo skripto failas `test_solution.py`, sukonstruotas iš užduoties testų sąrašo ir reikalingų importavimo direktyvų.

Testai vykdomi paleidžiant „pytest” kaip atskirą poprocesį per `asyncio.create_subprocess_exec` su parametrais `-v --tb=short`, leidžiančiais surinkti detalią informaciją apie kiekvieną testą. Vykdymo trukmė ribojama iki trisdešimties sekundžių

(timeout: int = 30) naudojant `asyncio.wait_for` apvaskalą - jei sprendimas viršija šią ribą (pavyzdžiui, dėl begalinės kilpos ar nebaigiamos rekursijos), vykdymas nutraukiamas ir grąžinamas rezultatas su būsena `SolutionStatus.TIMEOUT`. Šis ribojimas yra būtinas todėl, kad debatų metu agentai gali sugeneruoti neoptimalų arba nebaigtinį kodą, kurio vykdymas be priverstinio nutraukimo nesibaigtų. Pytest išvestis išskaidoma reguliariųjų išraiškų pagrindu pagal šabloną `.*::(\w+)\s+(PASSED|FAILED|ERROR)`, kuris atitinka „pytest” išsamios išvesties (angl. verbose output) formato eilutes ir leidžia patikimai suskaičiuoti išlaikytų bei neišlaikytų testų skaičių, kartu su konkrečių testų pavadinimais ir klaidų pranešimais.

2.7.2. Sprendimų sanitizacija prieš vykdymą

7-9 milijardų parametrų modeliai, sekdami pavyzdines užduočių struktūras, kartais į savo sprendimą įtraukia ir pačius testus arba naudoja santykinius importus, kurie netinka plokščios katalogo struktūros vykdymui. Sistemoje veikia du sanitizacijos mechanizmai, automatiškai pritaikomi kiekvienam sprendimui prieš įrašant jį į testavimo katalogą.

Pirmasis mechanizmas - `_strip_test_functions` - šalina iš sprendimo bet kokias aukščiausio lygio `test_` prefiksu prasidedančias funkcijas. Šis mechanizmas reikalingas todėl, kad „pytest” pagal numatytuosius nustatymus atranda visas funkcijas, kurių vardas prasideda `test_`, ir vertina jas kaip testus - jei DKM sprendime atsiranda papildoma `test_` funkcija (pavyzdžiui, modelio savarankiškai pridėtas demonstracinis testas), ji būtų suskaičiuota kaip dar vienas išlaikytas testas, taip dirbtinai padidinant testų išlaikymo rodiklį. Funkcija pirmiausia remiasi AST analize, o sintaksės klaidos atveju taiko atsarginį reguliariųjų išraiškų metodą (11 kodo fragmente parodyta supaprastinta logika).

```
# src/core/executor.py, supaprastinta _strip_test_functions logika
tree = ast.parse(code)
remove_ranges = []
for node in tree.body:
    if isinstance(node, ast.FunctionDef) and node.name.startswith("test_"):
        remove_ranges.append((node.lineno - 1, node.end_lineno))
# Pašalinama atgaline tvarka, kad indeksai išliktų galiojantys
for start, end in reversed(remove_ranges):
    lines[start:end] = []
```

11 kodo fragmentas. Aukščiausio lygio `test_` funkcijų pašalinimas iš DKM sprendimo prieš vykdymą.

Antrasis mechanizmas - `_fix_relative_imports` (`executor.py`) - konvertuoja santykinius importus (`from .module import X`) į absoliučius (`from module import X`), kadangi laikiname kataloge nėra `__init__.py` failo ir Python paketo struktūra neegzistuoja. Konvertavimas atliekamas reguliariaja išraiška `^(from\s+)\.(\w)`, kuri pakeičia tik priešdėlinį tašką, neliesdama įvardyto modulio kelio likučio. Šis mechanizmas adresuoja konkretų DKM elgsenos modelį - modeliai, treniruoti su didelėmis kodo bazėmis, kuriose santykiniai importai yra įprasti, kartais juos automatiškai įterpia net kai užduoties kontekste tai netinka.

Vieno failo sprendimo atveju veikia papildomas apsauginis sluoksnis - vietoje tiesioginio `from solution import * testavimo failas atlieka selektyvų importavimą`, importuodamas tik tuos pavadinimus, kurie nepradedami `test_` arba `_` prefiksais (`executor.py`). Taip užtikrinama, kad net jei AST sanitizacija praleido kokią nors testavimo funkciją (pavyzdžiui, dėl sintaksės klaidos sprendime), ji vis tiek nepateks į „pytest” atrankos sritį.

2.7.3. Rezultatų podėliavimas

Debatų ciklo metu tas pats kodas gali būti vykdomas kelis kartus - pavyzdžiui, kai agentas pateikia tą patį sprendimą be pakeitimų (žr. 2.4.2 poskyrį) arba kai keli agentai nepriklausomai konverguoja į vienodą sprendimą. Kad nereikėtų pakartotinai paleisti „pytest” tam pačiam kodui, `CodeExecutor` naudoja vidinį podėlį (angl. cache), kuriame raktas formuojamas iš MD5 maišos reikšmės, apskaičiuotos pagal sprendimo kodą ir testų tekstą. Vieno failo užduoties

atveju maišos įvestis yra paties kodo eilutė; daugiafailės užduoties atveju - JSON formatu serializuotas failų vardų ir turinio žemėlapis, surikiuotas pagal failo vardą, tai užtikrina nepriklausomumą nuo failų pateikimo tvarkos.

Podėlio dydis apribotas konstanta `MAX_CACHE_SIZE = 2048` (`executor.py`). Pasiėkus šią ribą, pašalinamas seniausiai įdėtas įrašas (FIFO eiliškumas, naudojant `next(iter(self._cache))`). Tokia paprasta įrašų šalinimo strategija pasirinkta todėl, kad debatų metu paprastai svarbiausi yra naujausi sprendimai, o seni rezultatai retai pakartotinai užklaunami. Eksperimentų metu podėlis ypač naudingas balsavimo fazėje, kur, jei agentų sprendimai yra panašūs, podėliuoti rezultatai grąžinami beveik akimirksniu.

2.7.4. Kodo kokybės analizė: „Pylint” ir „Radon”

Be funkcinio korektiškumo, kiekvienas sprendimas vertinamas ir kodo kokybės atžvilgiu naudojant `CodeQualityAnalyzer` klasę (`executor.py`). Klasė paleidžia du nepriklausomus statinės analizės įrankius: „Pylint” stiliumi ir loginių defektų aptikimui bei „Radon” sudėtingumo metrikoms skaičiuoti. Šių dviejų metrikų derinys yra atsakas į Zheng ir kt. [Zcm24] išvadą, kad „šie atradimai pabrėžia kelių dimensijų DKM kodo vertinimo poreikį, akcentuojant metrikas, viršijančias vien korektiškumą” (angl. „These findings highlight the need for a multidimensional evaluation of code LLMs, emphasizing metrics beyond correctness for real-world applications”). Pasirinkti būtent statinės analizės įrankiai - tai vienas iš būdų objektyviai išmatuoti tas pačias kokybės dimensijas, kurias Dillmann ir kt. [Dst24] nurodo kaip pagrindines DKM generuojamo kodo vertinimo ašis: „skaitomumas, suprantamumas, sudėtingumas, modulizacija ir bendras prižiūrimumas” (angl. „readability, understandability, complexity, modularisation, and overall maintainability”). „Pylint” apima skaitomumo bei stiliumi aspektus per defektų klasifikaciją, o „Radon” - sudėtingumo bei prižiūrimumo aspektus per ciklomatinių sudėtingumo ir prižiūrimumo indekso metrikas.

„Pylint” paleidžiamas su JSON išvesties formatu ir išjungtomis docstring'ų konvencijų taisyklėmis (C0114, C0115, C0116), kadangi DKM sprendimuose docstring'ų nebuvimas yra įprastas ir nesusijęs su realiomis kokybės problemomis. Sąvadas formuojamas pagal paprastą formulę $score = \max(0, 10 - 0,5 \times \text{visų Pylint pranešimų skaičius})$, normalizuotą dešimtbalėje skalėje, į pranešimų skaičių įtraukiami visi registruojami „Pylint” pranešimai, o pagrindinės kategorijos - klaidos (error), įspėjimai (warning) ir konvencijų pažeidimai (convention) - saugomos atskiruose statistikos laukuose. Jei „Pylint” iškvietimas nesėkmingas (pavyzdžiui, dėl sintaksės klaidos kode), naudojama atsarginė vidutinė reikšmė 5,0, todėl analizės nesėkmė nesustabdo viso debatų ciklo.

„Radon” išskiria dvi metrikų grupes: ciklomatinių sudėtingumą (cc posistemis), pagal kurį skaičiuojamas vidutinis ir maksimalus sudėtingumas, bei prižiūrimumo indeksą (angl. maintainability index, mi posistemis), kurio reikšmės yra intervale nuo 0 iki 100. Visos metrikos saugomos `CodeQualityMetrics` duomenų klasėje kartu su pagrindinėmis dimensijomis - kodo eilučių skaičiumi, tuščių eilučių skaičiumi ir komentarų skaičiumi - apskaičiuotomis tiesiogiai iš teksto. „Pylint” balas yra naudojamas svertinio balsavimo schemeje kaip premija (žr. 2.3.2 poskyrį), todėl sprendimai, kurie ne tik išlaiko testus, bet ir atitinka stiliumi standartus, įgyja papildomą pranašumą galutiniam rezultate.

Reikia pažymėti, kad vien testų išlaikymo metrika nėra pakankama patikimam DKM kodo vertinimui - Liu ir kt. [Lxw+23] „EvalPlus” tyrime parodyta, kad „testų nepakankamumas gali sukelti klaidingą reitingavimą” (angl. „test insufficiency can lead to mis-ranking”), kai modeliai, generuojantys paviršutiniškai veikiančią, tačiau loginiu požiūriu silpną kodą, gali aplenkti gilesnių sprendimų autorius. Šio darbo užduočių rinkinyje vienetinių testų skaičius yra vidutiniškai apie devyni vienai užduočiai (tiksliai - 8,91 vidurkis 120 užduočių rinkinyje, intervalas nuo keturių iki šešiolikos testų), tačiau klaidingo reitingavimo rizika lieka net su

didesniu testų tankiu. Kodo kokybės metrikos veikia kaip antras nepriklausomas vertinimo signalas: net jeigu du sprendimai išlaiko vienodą testų skaičių, pranašumą gauna tas, kurio struktūra ir stilius geriau įvertinti statinės analizės įrankiais.

2.8. Žiniatinklio sąsaja interaktyviam tyrimui

Be komandinės eilutės sąsajos ir kaip apvalkalas veikiančių paketinio režimo HPC scenarijų (žr. 2.1.1 poskyrį), sistema turi ir interaktyvią žiniatinklio sąsają, skirtą preliminariniams bandymams vietinėje darbo stotyje. Žiniatinklio sąsajos paskirtis - leisti vartotojui per grafinę formą sukonfigūruoti debatų parametrus, paleisti vieną debatą arba paketinį eksperimentą ir realiu laiku stebėti vykstantį procesą: agentų pasiūlymus, kritikas, balsavimą, raundų raidą bei galutinius rezultatus. Sąsaja realizuota `src/web/app.py` modulyje bei `src/web/templates/index.html` šablone, o jos veikimas paremtas „Flask“ karkasu su flask-socketio plėtinium, leidžiančiu dvikrypčiu būdu keistis pranešimais tarp serverio ir naršyklės per „WebSocket“ protokolą.

2.8.1. Architektūros apžvalga ir technologijų rinkinys

Sąsajos serverio dalis paleidžiama kaip „Flask“ aplikacija su flask-socketio plėtinium, naudojančiu gijomis paremtą asinchroninį režimą (`async_mode='threading'`), kuris leidžia lygiagrečiai aptarnauti HTTP užklausas ir „WebSocket“ pranešimus tame pačiame procese. Visi debatų ciklai vykdomi atskiroje foninėje gijoje (`threading.Thread`), kuri naudoja savo asincio įvykių ciklą (angl. event loop) - tai būtina, nes `DebateOrchestrator` yra asinchroninė klasė, o „Flask“ užklausų apdorojimas vyksta sinchroniškai. Vienu metu leidžiama vykdyti tik vieną debatų sesiją, naudojant `threading.Lock` (`_debate_lock`) mechanizmą, kad keli vartotojai vienu metu nekonkuruotų dėl tos pačios „Ollama“ instancijos išteklių.

Vartotojo sąsaja organizuota kaip vieno puslapio programa (angl. Single Page Application, SPA) su trimis panelėmis: kairėje - konfigūracijos skydelis (užduočių pasirinkimas, agentų rinkinys, raundų skaičius, kontrolės mechanizmų jungikliai), centre - debatų arena (raundų eiga, agentų sprendimai, kritikos), dešinėje - veiklos žurnalas (angl. activity feed) su detalizuotais pranešimais kiekvienai fazei. Šablone realizuoti modaliniai langai paketinio eksperimento konfigūravimui („Batch Experiment“), debatų protokolų peržiūrai („Transcripts“), solo ir debatų režimo palyginimui („Comparison“) bei detalių rezultatų vaizdavimui („Results“) su šešiomis kortelėmis: bendras vaizdas, raundų raida, kritikos analizė, agentų elgsena, kodo kokybė, palyginimas).

2.8.2. REST API endpoints

Serveris atveria tryliką REST API galinių taškų (angl. endpoints), suskirstytų į penkias grupes pagal funkciją (papildomai egzistuoja vienas šakninis `/` maršrutas, grąžinantis pagrindinio puslapio HTML šabloną). Pirmoji grupė - sistemos būsenos galiniai taškai: `/api/health` (serverio prieinamumas) ir `/api/models` (per „Ollama“ API gautas vietinių modelių sąrašas). Antroji grupė - užduočių galiniai taškai: `/api/tasks` (visų užduočių sąrašas su sudėtingumo filtravimu) ir `/api/tasks/<task_id>` (vienos užduoties detalūs duomenys). Trečioji grupė - debatų rezultatų galiniai taškai: `/api/debates` (visų išsaugotų debatų sąrašas), `/api/debates/<debate_id>` (vieno debato detalūs duomenys) bei `/api/stats` (agreguoti statistiniai rodikliai). Ketvirtoji grupė - protokolų galiniai taškai: `/api/transcripts` (žmogui skaitomų protokolų sąrašas su metaduomenimis), `/api/transcripts/<filename>` (vieno protokolo turinys) ir `/api/transcripts/<filename>/download` (atsisiuntimas su `Content-Disposition: attachment` antrašte). Penktoji grupė - analizės bei valdymo galiniai taškai: `/api/comparison` (solo ir debatų režimo palyginimas), `/api/export/csv` (rezultatų eksportas CSV formatu, naudojamas 4 skyriaus analizėje) bei `/api/clear-data` POST metodu (duomenų bazės išvalymas derinimo tikslams).

2.8.3. Realaus laiko įvykių srautas per „WebSocket“

Pagrindinė interaktyvios sąsajos savybė - galimybė matyti debatų eigą realiu laiku, o ne tik galutinį rezultatą. Tai realizuota per dvikryptę „SocketIO“ sąveiką, kurioje serveris ir naršyklė keičiasi struktūrizuotais įvykiais. Iš naršyklės serveris priima šešis įvykius: connect (prisijungimas), disconnect (atsijungimas), start_debate (vieno debato paleidimas), start_solo (solo režimo paleidimas), start_batch (paketinio eksperimento su keliomis užduotimis paleidimas) ir stop_debate (vykstančio debato sustabdymas). Iš serverio į naršyklę siunčiama dvylika skirtingų įvykių, atspindinčių debatų ciklo eigą (connected, debate_starting, debate_started, phase_start, agent_message, round_complete, debate_complete, debate_error, debate_stopped, batch_progress, batch_complete bei error).

Pagrindinė įvykių grandinė formuojama per tris grįžamojo iškviatimo (angl. callback) funkcijas, kurios perduodamos DebateOrchestrator konstruktoriui ir kviečiamos iš debatų ciklo vidaus (12 kodo fragmente parodyta jų struktūra).

```
# src/web/app.py, supaprastinta callback grandinė
def on_round_complete(round_summary):
    socketio.emit('round_complete', {
        'round_num': round_summary.round_num,
        'best_pass_rate': round_summary.best_pass_rate,
        'avg_pass_rate': round_summary.avg_pass_rate,
        'bugs_found': round_summary.bugs_found,
        'solutions': [s.to_dict() for s in round_summary.solutions],
        'votes': [v.to_dict() for v in round_summary.votes],
    })
def on_message(message):
    socketio.emit('agent_message', {
        'agent_id': message.agent_id,
        'content': message.content[:2000], # apkarpyta realiu laiku
    })
def on_phase(phase, round_num):
    socketio.emit('phase_start', {'phase': phase, 'round_num': round_num})
orchestrator = DebateOrchestrator(
    llm_client=llm_client, config=config,
    on_round_complete=on_round_complete,
    on_message=on_message,
    on_phase=on_phase,
)
```

12 kodo fragmentas. „SocketIO“ įvykių siuntimas iš debatų ciklo vidinių grįžtamųjų iškviatimų.

phase_start įvykis išsiunčiamas prieš kiekvieną iš keturių debatų fazių (propose, critique, revise, vote) ir leidžia sąsajai parodyti aktyvią fazę realiu laiku. agent_message įvykis išsiunčiamas, kai atskiras agentas baigia generuoti savo atsakymą, kartu su iki dviejų tūkstančių simbolių apkarpyta jo turinio versija (visa atsakymo versija lieka serverio pusėje ir vėliau saugoma protokole, žr. 2.9 poskyrį). round_complete įvykis išsiunčiamas po viso raundo pabaigos ir apima struktūrizuotus duomenis - sprendimus, balsus, geriausią ir vidutinį testų išlaikymą - kurie sąsajoje paverčiami diagramomis raidos vaizdavimui. Galutinis debate_complete įvykis siunčia pilnus metrikų duomenis kartu su raundų istorija bei kritikomis, kuriuos sąsaja atvaizduoja modaliniame rezultatų lange.

Paketinio režimo metu vietoj debate_complete išsiunčiamas batch_progress po kiekvienos užduoties (su laukais current, total, task_name, status) ir galutinis batch_complete po viso paketo, leidžiant vartotojui matyti pažangą per visą eksperimentą.

2.8.4. Atsparumas atsijungimui: paskutinės būsenos perdavimas

Realaus laiko sąsajose esminis iššūkis yra tinkamas sistemos atsakas, kai klientas atsijungia debatų metu - pavyzdžiui, dėl naršyklės kortelės perkrovimo, „WebSocket” ryšio nutraukimo, kilus tinklo problemoms arba operacinei sistemai perėjus į miego režimą. Be specialaus apdorojimo, iš naujo prisijungęs klientas pamatytų pasenusią sąsajos būseną, kurioje liko paskutinis matytas pranešimas (pavyzdžiui, „Round 3 - REVISE”), nors tikrasis debatas tuo metu jau galėjo būti pasibaigęs.

Sistemoje realizuotas paskutinės užbaigtos būsenos perdavimo (angl. replay) mechanizmas, kurio supaprastinta logika pateikta 13 kodo fragmente. Po kiekvieno `debate_complete` arba `debate_error` įvykio serveris išsaugo to įvykio turinį globaliame `_last_finished_state` žodyne su laiko žyma. Kai prisijungia naujas klientas (`connect` įvykis), serveris patikrina, ar yra išsaugota būsena ir ar jos amžius neviršija valandos; jei abi sąlygos tenkinamos, tas pats įvykis išsiunčiamas tam klientui, leidžiant sąsajai atvaizduoti galutinius rezultatus nepaisant praleisto realaus laiko srauto. Pradėjus naują debatą, išsaugota būsena ištrinama, kad vėlyvam klientui nebūtų pakartotinai pateikti ankstesnio debato rezultatai vietoje dabartinio.

```
# src/web/app.py, supaprastinta paskutinės būsenos perdavimo logika
_last_finished_state = None
_LAST_STATE_TTL_SEC = 60 * 60 # 1 valanda
def _remember_finished(event_name, payload):
    global _last_finished_state
    _last_finished_state = {
        "event": event_name,
        "payload": payload,
        "timestamp": time.time(),
    }
@socketio.on('connect')
def handle_connect():
    emit('connected', {'status': 'connected'})
    # Perduodame paskutinę užbaigtą būseną, jei ji vis dar šviežia
    if _last_finished_state is not None:
        age = time.time() - _last_finished_state["timestamp"]
        if age <= _LAST_STATE_TTL_SEC:
            emit(_last_finished_state["event"], _last_finished_state["payload"])
```

13 kodo fragmentas. Paskutinės užbaigtos būsenos perdavimo mechanizmas atsijungusiam klientui.

Šis mechanizmas yra pragmatinis sprendimas, atsiradęs iš praktinio naudojimo - debatai su 7-9 milijardų parametrų modeliais ekstremalaus sudėtingumo užduotyse gali užtrukti kelias minutes, todėl naudotojas dažnai palieka atvirą naršyklės skirtuką neaktyvioje būsenoje. Jei tuo metu sistema patenka į „macOS” miego režimą arba klientas atnaujiną puslapį tikrindamas eigą, perdavimo mechanizmas užtikrina, kad galutiniai rezultatai vis tiek pasiektų naudotoją.

2.9. Duomenų saugojimas ir analizės srautas

Po kiekvieno debato sistemoje sukaupiama struktūrizuotų duomenų: pasiūlymai, kritikos, balsai, konsensuso rezultatas, agentų statistikos bei generavimo trukmės. Šiems duomenims saugoti naudojama reliacinė „SQLite” duomenų bazė per „SQLAlchemy” objektinį-reliacinį atvaizdavimą (angl. Object-Relational Mapping, ORM), o tolesnei analizei jie eksportuojami į CSV formatą bei žmogui skaitomus tekstinius protokolus. Šis poskyris aprašo duomenų bazės schemą, „SQLite” konfigūracijos ypatumus daugiagijės aplinkos atveju, CSV eksporto struktūrą bei metrikų rinkimo ir vaizdavimo komponentus, kurie sudaro pagrindą 4 skyriaus eksperimentinei analizei.

2.9.1. Reliacinė schema: penkios ORM lentelės

Duomenų bazės schema apibrėžta `src/database/models.py` modulyje ir susideda iš penkių susijusių lentelių. Pagrindinė lentelė - `DebateRecord` - saugo vieno debato apibendrintus duomenis: identifikatorių, užduoties metaduomenis (`task_id`, `task_name`, `task_difficulty`), konfigūraciją (agentų modeliai kaip JSON masyvas, maksimalus raundų skaičius, konsensuso slenkstis), rezultatus (galutinis testų išlaikymo rodiklis, konsensuso būseną, laimėjęs agentas) bei visą serializuotą debato turinį `full_debate_data` JSON lauke. Lentelė taip pat turi is_solo loginę vėliavą, leidžiančią atskirti solo režimo bandymus nuo kelių agentų debatų - tai svarbu 4 skyriaus palyginimui tarp dviejų režimų.

Antrasis lygis - `RoundRecord` lentelė - saugo kiekvieno atskiro raundo duomenis su išorinio rakto (angl. foreign key) ryšiu į `DebateRecord`. Šiame lygyje fiksuojami raundo statistiniai rodikliai (geriausias ir vidutinis testų išlaikymas, rastų klaidų skaičius, raundo trukmė) bei serializuoti masyvai sprendimų, kritikų, balsų ir konsensuso duomenų. Trečioji lentelė - `AgentStatRecord` - saugo agentų lygio statistikas kiekvienam debatui: pasiūlymų skaičių, kritikų skaičių, rastų klaidų skaičių, nuomonės pakeitimų skaičių (`times_changed_mind`), savo sprendimo gynimų skaičių (`times_defended`) bei bendrą generavimo trukmę. Šios statistikos leidžia 4 skyriuje atlikti agentų elgsenos profiliavimą.

Papildomai scheme yra `TaskRecord` lentelė užduočių metaduomenims (sąsajos, apribojimai, žymos) ir `ExperimentRecord` lentelė kelių debatų grupavimui į eksperimentus su bendra konfigūracija; abi šios lentelės yra savarankiškos ir su `DebateRecord` tiesiogiai per `SQLAlchemy` ryšius nesusietos. Antrinės lentelės `RoundRecord` ir `AgentStatRecord` susietos su `DebateRecord` dvipusiais `SQLAlchemy` ryšiais, kuriems taikoma `cascade="all, delete-orphan"` taisyklė - ji užtikrina, kad ištrinus `DebateRecord` automatiškai pašalinami ir su juo susiję raundų bei agentų statistikos įrašai.

2.9.2. „SQLite“ konfigūracija ir gijų saugumas

„SQLite“ duomenų bazė pasirinkta dėl trijų priežasčių: ji nereikalauja atskiros serverio infrastruktūros, palaiko transakcijas, o `SQLAlchemy` abstrakcijos sluoksnis leidžia ateityje pereiti prie „PostgreSQL“ be aukštesnio lygio kodo pakeitimų. Tačiau „SQLite“ turi apribojimų daugiagijėje aplinkoje, o sistemoje veikia žiniatinklio sąsaja, kurioje debatų ciklas vykdomas atskiroje foninėje gijoje (žr. 2.8.1 poskyrį). Todėl variklio konfigūracijoje taikomi trys svarbūs nustatymai (14 kodo fragmente).

```
# src/database/models.py, „SQLite“ variklio konfigūracija
engine = create_engine(
    f"sqlite:///db_path",
    echo=False,
    connect_args={"check_same_thread": False, "timeout": 30},
    poolclass=StaticPool,
)
```

14 kodo fragmentas. „SQLite“ variklio konfigūracija daugiagijės aplinkos atveju.

Parametras `check_same_thread=False` reikalingas todėl, kad žiniatinklio programa variklio objektą sukuria pagrindinėje gijoje, o debatų rezultatų saugojimas vyksta atskiroje foninėje gijoje - be šio nustatymo „SQLite“ iškvietai iš kitos gijos sukeltų sunkiai interpretuojamas klaidas (kartais netiksliai pranešama apie „readonly database“ esant didelei apkrovai). Parametras `poolclass=StaticPool` nurodo „SQLAlchemy“ pakartotinai naudoti tą patį pagrindinį prisijungimą, o ne kurti naujus prisijungimus kiekvienai gijai; taip sumažinama skirtingų gijų prisijungimų būsenų nesuderinamumo rizika, kuri taip pat gali pasireikšti kaip „readonly“ klaida, kai įrašo priekio (angl. Write-Ahead Logging, WAL) arba žurnalo failai tampa blokuojami dėl lygiagrečios prieigos. Tokia konfigūracija šiame sistemos naudojimo scenarijuje yra tinkama, nes vienu metu vykdomų debatų skaičius ribojamas `_debate_lock`

mechanizmu (žr. 2.8.1 poskyrį). Trisdešimties sekundžių užklauso laukimo terminas (timeout=30) suteikia papildomą atsparumą trumpalaikių blokavimų atveju.

2.9.3. CSV eksportas eksperimentinei analizei

Norint atlikti agreguotą analizę dideliame debatų rinkiniui, sistema teikia du atskirus CSV eksporto formatus, realizuotus `src/analysis/csv_export.py` modulyje. Pirmasis - apibendrinamasis CSV - turi keturiasdešimt keturis stulpelius, padengiančius identifikavimą (`debate_id`, `task_id`, `task_name`, `difficulty`, `mode`), konfigūraciją (`agent_models`, `num_agents`), galutinius rezultatus (`final_pass_rate`, `tests_passed`, `tests_total`), `pass@k` metrikas (`pass_at_1`, `pass_at_3`), pagerėjimo rodiklius (`improvement_over_best_initial`, `improvement_over_avg_initial`), konsensuso duomenis (`total_rounds`, `consensus_reached`, `consensus_ratio`, `rounds_to_consensus`), kritikų agregacijas (`total_critiques`, `total_bugs_found`, `total_bugs_fixed`, `bug_fix_rate`, `total_improvements_suggested`), kokybės įvertinimus (`avg_correctness_rating`, `avg_efficiency_rating`, `avg_readability_rating`), kodo kokybės metrikas pradžioje ir pabaigoje (`initial_avg_pylint`, `final_pylint`, `initial_avg_complexity`, `final_complexity`), laiko sąnaudas (`duration_seconds`, `avg_round_duration`, `total_llm_time`, `total_execution_time`), sprendimų skaičius (`all_solutions_count`, `passing_solutions_count`), aktyviausius agentus (`most_active_agent`, `most_successful_agent`, `most_bugs_found_by`), bendrą būseną ir laimėtoją (`status`, `winning_agent`) bei specialias raundo evoliucijos metrikas (`best_round`, `best_round_pass_rate`, `peak_after_debate`).

Antrasis formatas - per-raundo CSV - turi septyniolika stulpelių ir saugo kiekvieno raundo duomenis atskira eilute kiekvienam dalyvavusiam agentui. Šis formatas yra detalesnis nei apibendrinamasis ir naudojamas raundų evoliucijos analizei: pavyzdžiui, įvertinti, ar konkretus agentas pirmajame raunde pateikė geresnį sprendimą už vėlesnius. Kiekvienoje eilutėje saugomi identifikatoriai (`debate_id`, `task_id`, `task_name`, `difficulty`, `agent_id`, `model`, `role`), raundo numeris, ar tai revizija ar pradinis pasiūlymas (`round_num`, `is_revision`), testavimo rezultatai (`pass_rate`, `tests_passed`, `tests_total`, `status`), generavimo trukmė ir kodo simbolių skaičius (`generation_time`, `code_chars`) bei vėliavos sutrumpinimo ir istorinio sprendimo grąžinimo aptikimui (`was_truncated`, `is_historical_best_reuse`).

Abu CSV failai generuojami iškviečiant `python3 -m src.analysis.csv_export` komandą su `-db`, `--out` ir `--out-rounds` parametrais; ši komanda integruota į HPC pagrindinį scenarijų (`hpc/_lib_run.sh`, žr. 3.4.2 poskyrį), todėl po kiekvieno eksperimento gaunami du CSV failai su unikaliais pavadinimais, įtraukiančiais SLURM užduoties identifikatorių. Tokia integracija užtikrina, kad eksperimento rezultatus galima iškart importuoti į „pandas“ duomenų rėmus 4 skyriaus analizei be papildomų transformavimo žingsnių.

Reikia pažymėti, kad `peak_after_debate` stulpelis yra specifinis indikatorius, šio darbo metu sukurtas debatų pridėtinei vertei įvertinti. Jo loginė reikšmė lygi `True`, jei geriausias debato metu pasiektas testų išlaikymo rodiklis pirmą kartą pasirodė antrame ar vėlesniame raunde, o ne pirmajame - tokiu būdu galima atskirti atvejus, kai debatai realiai prisidėjo prie galutinio rezultato, nuo atvejų, kai sprendimas buvo rastas iškart, o tolesni raundai tik pakartojo pirmojo raundo geriausią sprendimą. Šis indikatorius yra pagrindinis 4 skyriaus analizėje, leidžiantis kiekybiškai įvertinti, kuriais sudėtingumo lygiais debatai duoda papildomą naudą.

2.9.4. Metrikų rinkimas ir vaizdavimas

Metrikų skaičiavimui sistemoje veikia atskira `MetricsCollector` klasė (`src/analysis/metrics_collector.py`), kuri iš debato objekto išskiria daugiau nei tris dešimtis kiekybinių rodiklių - `pass@k`, pagerėjimo rodiklius, klaidų aptikimo bei jų ištaisymo rodiklius, agentų elgsenos profilius. Tarp jų svarbiausia yra `pass@k` metrika, įvesta Chen ir kt. [Ctj+21] „HumanEval“ tyrime kaip kodo generavimo modelių vertinimo standartas. Šio darbo realizacijoje pateikta formulė atitinka originalų autorių apibrėžimą (15 kodo fragmente).

```
# src/analysis/metrics_collector.py, pass@k formulė
def compute_pass_at_k(n: int, c: int, k: int) -> float:
    """Compute pass@k metric (Chen et al., 2021)."""
    if n <= 0 or k <= 0:
        return 0.0
    if c >= n or n - c < k:
        return 1.0
    return 1.0 - comb(n - c, k) / comb(n, k)
```

15 kodo fragmentas. „pass@k” metrikos realizacija pagal Chen ir kt. [Ctj+21] apibrėžimą.

Formulės reikšmė: turint n sugeneruotų sprendimų, iš kurių c išlaiko visus testus, $\text{pass}@k$ yra tikimybė, kad bent vienas iš atsitiktinai parinktų k sprendimų bus tarp išlaikiusių. Šio darbo eksperimentuose $\text{pass}@1$ ir $\text{pass}@3$ rodikliai padeda palyginti vieno bandymo (angl. greedy) ir kelių bandymų sėkmės rodiklius, o kelių agentų debatų atveju $\text{pass}@3$ natūraliai atitinka trijų sprendimus generuojančių pasiūlytojų rinkinį (teisėjo agentas, jei sukonfigūruotas, sprendimų nesiūlo, todėl į skaičiavimą neįtraukiamas).

Greta kiekybinių metrikų, sistema generuoja žmogui skaitomus tekstinius protokolus per DebateVisualizer klasę (src/analysis/visualizer.py). Po debatų ciklo iškviečiamas generate_full_transcript metodas, kuris suformuoja tekstinį failą su detaliu visų raundų aprašymu - pasiūlymais, kritikomis, balsais, konsensuso rezultatu - ir saugo jį transcripts/ kataloge; iškvietimas integruotas tiek komandinės eilutės sąsajoje (src/main.py), tiek žiniatinklio sąsajos foninėje gijoje (žr. 2.8.1 poskyrį). Šie protokolai naudojami kokybinei analizei 4 skyriuje, kur remiantis konkrečiais pavyzdžiais iliustruojama, kaip dėl kitų agentų kritikos per kelis raundus pagerėja pradinis modelio sprendimas.

Aprašyta duomenų saugojimo ir analizės infrastruktūra baigia sistemos projektavimo skyrių. Tolesniuose skyriuose aptariama, kaip ši infrastruktūra panaudojama eksperimentiniam tyrimui: 3 skyrius pristato tyrimo metodiką, įskaitant užduočių rinkinio sudarymą bei eksperimentinių konfigūracijų parinkimą, o 4 skyrius - eksperimentų rezultatus, gautus apdorojant CSV išvestį naudojant „pandas” duomenų rėmus ir vizualizuojant rezultatus „matplotlib” bei „seaborn” diagramomis.

Skyriaus apibendrinimas. Šiame skyriuje aprašyta sukurta modulinė kelių agentų debatų sistema, pritaikyta 7-9 milijardų parametrų atvirųjų svorių modeliams. Sistemos sprendimai orientuoti į praktines mažesnių modelių problemas: sprendimų regresiją po sėkmingų ankstesnių raundų, stagnaciją, sutrumpintus atsakymus ir daugiafailių užduočių generavimo ribotumus. Aprašyta architektūra, debatų ciklas, kokybės kontrolės mechanizmai, žiniatinklio sąsaja, HPC paleidimo scenarijai ir duomenų saugojimo sluoksnis sudaro pagrindą kitame skyriuje pristatomi tyrimo metodikai.

3. Tyrimo metodika

Šiame skyriuje pristatoma tyrimo metodika ir pagrindžiama lyginamojo eksperimento logika. Aptariami pasirinkti agentų rinkiniai bei teisėjo modeliai, kodo generavimo užduočių rinkinio sudarymo principai, dviejų etapų eksperimentų scenarijus (preliminarūs bandymai žiniatinklio sąsajoje ir pagrindinis eksperimentas didelio našumo skaičiavimo aplinkoje) bei vertinimo metrikos, kurios bus taikomos 4 skyriuje aptariant rezultatus. Skyrius nuosekliai naudoja lyginamosios analizės, klasifikacijos ir sisteminimo metodus - kiekvienas pasirinkimas pagrindžiamas literatūros išvadomis, sistemos kodu arba preliminariais bandymais.

3.1. Tyrimo logika ir palyginimo schema

Šio darbo tyrimas organizuojamas kaip lyginamasis eksperimentas, kuriame dėmesys skiriamas dviem klausimams: ar kelių agentų debatai patikimai pranoksta vieno modelio bazinę liniją kodo generavimo užduotyse ir kurios konkrečios sistemos konfigūracijos prisideda prie šio pranašumo. Tokia tyrimo logika atitinka kontroliuojamo lyginamojo eksperimento principą - sistemos architektūriniai parametrai (debatų raundų skaičius, kontrolės mechanizmai, balsavimo svėrimo taisyklės) yra fiksuojami, o eksperimentinis kintamasis yra tik tas faktorius, kurio poveikis vertinamas konkrečioje konfigūracijoje. Toks principas leidžia pastebėtus rezultatų skirtumus tiesiogiai sieti su pasirinkta konfigūracija, o ne su neapibrėžtais sistemos elgsenos pokyčiais.

Pirmoji palyginimo ašis yra agentų rinkinio sudėtis. Sistemoje fiksuotas trijų pasiūlytojų skaičius, tačiau pati rinkinio sudėtis sąmoningai varijuojama tarp dviejų klasių, kurias toliau vadiname Pool A (7B klasė) ir Pool B (9B klasė). Pool A apima tris šešių-septynių milijardų parametrų kodo specializacijos modelius - „Qwen2.5-Coder“, „DeepSeek-Coder“ ir „Code Llama“ instrukcijų vykdymo variantą; Pool B apima tris aštuonių-devynių milijardų parametrų modelius - „Granite Code“, „CodeGeeX4“ ir „Yi-Coder“. Toks padalijimas leidžia patikrinti hipotezę, kad didesnio parametrų skaičiaus modelių rinkinys pasiekia geresnį rezultatą debatuose, ir nustatyti, ar didesnio parametrų skaičiaus peer modelių rinkinio pasirinkimas yra praktiškai vertinga kryptis siekiant pagerinti atvirųjų svorių modelių kodo generavimo rezultatus.

Antroji palyginimo ašis yra teisėjo agento tipas. Šio kintamojo lygiai yra trys: be teisėjo (peer-only debatas), su kodo specializacijos teisėju („DeepSeek-Coder-V2“, šešių milijardų parametrų mišrių ekspertų (angl. Mixture of Experts, MoE) modeliu) ir su samprotavimo teisėju („DeepSeek-R1“, keturių milijardų parametrų). Tokia trijų lygių schema leidžia atskirti tris empirinius klausimus: ar teisėjo įvedimas iš viso pagerina debatų rezultatus, ar teisėjo modelio specializacijos tipas (kodo ar samprotavimo) turi reikšmingos įtakos kritikos ir balsavimo kokybei, ir kuris specializacijos tipas geriau dera su pasirinktais peer rinkiniais. Su teisėju vykdomi eksperimentai tiesiogiai įgyvendina „LLM-as-Judge“ paradigmą (Zheng ir kt. [Zcs+23]), o samprotavimo ir kodo specializacijos teisėjų palyginimas leidžia atsakyti į klausimą, kurio tipo modelis geriau atlieka kritiko ir bešališko vertintojo vaidmenį (žr. 2.2.7 poskyrį).

Trečioji palyginimo ašis yra debatų režimo ir vieno modelio bazinės linijos (solo) palyginimas. Solo bazinę liniją sudaro šeši nepriklausomi modelių paleidimai - kiekvienas iš pasiūlytojų vaidmenį atliekančių modelių (trys iš Pool A ir trys iš Pool B) vykdo užduočių rinkinį atskirai, be kritikos, be revizijos ir be balsavimo (iš viso 360 atskirų solo bandymų - šeši modeliai po šešiasdešimt užduočių). Šis lygis yra būtinas tam, kad būtų galima įvertinti debatų metodo pridėtinę vertę: be solo lyginamojo taško debatų rezultatai negalėtų būti interpretuojami nei kaip patobulinimas, nei kaip regresija. Solo paleidimai naudoja tuos pačius modelius ir tą pačią užduočių aplinką, kaip ir debatų pasiūlymo fazė, todėl skirtumai gali būti tiesiogiai priskirti debatų ciklo įtakai.

Lyginamojo eksperimento patikimumas priklauso ne tik nuo to, ką varijuojame, bet ir nuo to, ką fiksuojame. Visuose pagrindiniuose debatų eksperimentuose taikoma vienoda sistemos konfigūracija. Debatų raundų ribos parinktos vienodos visoms konfigūracijoms: maksimalus raundų skaičius - penki, minimalus - du, o konsensuso slenkstis pirmaujančiam sprendimui - šešiasdešimt procentų balsų svorio kartu su minimaliu penkiasdešimties procentų testų išlaikymu (žr. 2.3.3 poskyrį). Stagnacijos mažinimui visuose eksperimentuose įjungti du dinaminiai mechanizmai - adaptivi temperatūra ir kritikos istorija (komandinės eilutės vėliavos --adaptive-temperature ir --critique-history; žr. 2.4.3 poskyrį); revizijos užklausoje rodomas tik geriausias kito agento sprendimas (best-only režimas, žr. 2.2.4 poskyrį), o revizijos strategija - vienoda visiems agentams (revision_strategy=uniform). Įvairių revizijos strategijų režimas (revision_strategy=diverse), įgyvendinantis Liang ir kt. [Lhj+23] DMAD principą, sistemoje yra realizuotas, tačiau pagrindiniuose HPC eksperimentuose netaikomas - jo įvertinimas paliekamas vėlesniems tyrimams (žr. 2.4.4 poskyrį).

Pagal pasirinktą logiką tyrimas vykdomas dviem etapais. Pirmajame, preliminariniame etape, naudojant žiniatinklio sąsają atliekami pavieniai debatai vienai ar kelioms užduotims, siekiant ne įvertinti rezultatus statistiškai, o suderinti sistemos parametrus, patikrinti komponentų sąveiką ir nustatyti, kurie nustatymai turi būti fiksuoti pagrindiniame eksperimente. Antrajame, pagrindiniame etape, eksperimentai vykdomi VU MIF didelio našumo skaičiavimo (HPC) aplinkoje naudojant SLURM paketinę užduočių valdymo sistemą. Pagrindinį etapą sudaro vienuolika SLURM darbų - dešimt debatų konfigūracijų ir viena solo bazinės linijos užduotis, apimanti šešis pasiūlytojų modelius. Pirminiame užduočių rinkinyje (tasks/) vertinami abu peer rinkiniai su kodo specializacijos teisėju ir be jo (keturios debatų konfigūracijos), o išplėstiniame rinkinyje (tasks2/) tos pačios keturios konfigūracijos papildomos dar dviem - po vieną kiekvienam peer rinkiniui su samprotavimo teisėju „DeepSeek-R1” (iš viso šešios konfigūracijos). Iš viso šie eksperimentai apima 90 unikalių užduočių; duomenų bazėje sukaupiama 960 unikalių vykdymo įrašų (600 debatų ir 360 solo bandymų), o bendra pakartotinių CSV eksportų apimtis - po kiekvieno eksperimento eksportuojamas tuo metu sukauptas visas duomenų bazės turinys - siekia 6 060 įrašų. Visa eksperimentų matrica pateikta 2 lentelėje.

Eksperimento žymuo	Agentų rinkinys	Teisėjas	Užduočių rinkinys	Vykdymo įrašai
7b_no_judge	Pool A (7B)	be teisėjo	tasks/ (60)	60
7b_judge	Pool A (7B)	„DeepSeek-Coder-V2”	tasks/ (60)	60
9b_no_judge	Pool B (9B)	be teisėjo	tasks/ (60)	60
9b_judge	Pool B (9B)	„DeepSeek-Coder-V2”	tasks/ (60)	60
7b_no_judge2	Pool A (7B)	be teisėjo	tasks2/ (60)	60
7b_judge2	Pool A (7B)	„DeepSeek-Coder-V2”	tasks2/ (60)	60
9b_no_judge2	Pool B (9B)	be teisėjo	tasks2/ (60)	60
9b_judge2	Pool B (9B)	„DeepSeek-Coder-V2”	tasks2/ (60)	60
7b_judge2_r1	Pool A (7B)	„DeepSeek-R1”	tasks2/ (60)	60
9b_judge2_r1	Pool B (9B)	„DeepSeek-R1”	tasks2/ (60)	60
all6_solo	6 modeliai (po vieną)	be teisėjo	tasks/ (60)	360

2 lentelė. Pagrindinio etapo SLURM eksperimentų matrica.

Iš dešimties debatų eksperimentų ir solo bazinės linijos formuojami tiesioginiai poriniai palyginimai, kurių rezultatai pristatomi 4 skyriuje: solo prieš debatą, Pool A prieš Pool B, debatas su teisėju prieš debatą be jo bei kodo specializacijos teisėjas prieš samprotavimo teisėją. Šie palyginimai leidžia atsakyti į 1.4 poskyryje suformuluotus tyrimo klausimus apie atvirųjų svorių 7-9 milijardų parametrų modelių elgseną kelių agentų debatuose.

Papildomai prie pagrindinės eksperimentų matricos atlikta atskira kontroliuota adaptyvios temperatūros mechanizmo A/B abliacija (žr. 4.5 poskyrį). Joje keturios tasks2/ rinkinio debatų konfigūracijos pakartotinai vykdytos išjungus adaptyvią temperatūrą, todėl sukaupta 240 papildomų OFF vykdymo įrašų, porišcai lyginamų su atitinkamais pagrindinio etapo ON vykdymais. Taigi, įskaitant šią papildomą abliaciją, bendra tyrimo eksperimentinė bazė apima 15 SLURM darbų ir 1200 unikalių vykdymo įrašų.

3.2. Tiriamų konfigūracijų parinkimas

Šiame poskyryje aprašomi pagrindiniame eksperimente naudoti pasiūlytojų ir teisėjo modeliai. Pasiūlytojų modelių atranka grindžiama keturiais kriterijais: atvirųjų svorių (angl. open-weight) prieinamumu, leidžiančiu vykdyti modelius vietinėje aplinkoje be priklausomybės nuo komercinių paslaugų; 7-9 milijardų parametrų klase, kuri dera su vienos „NVIDIA V100“ grafikos plokštės 32 GB atminties biudžetu ir leidžia palyginti dvi skirtingo parametrų skaičiaus modelių klases; instrukcijų vykdymui pritaikytu apmokymu (angl. instruction-tuning), leidžiančiu modeliams vykdyti struktūrizuotas užklausas (PROPOSAL, CRITIQUE, VOTE), o ne tęsti įvesties tekstą natūraliu būdu; kodo specializacija arba aiškia programų inžinerijos užduočių aprėptimi. Teisėjo modeliams taikomi kiek kitokie kriterijai: didesnis parametrų skaičius (siekiant gauti kokybiškesnę kritikos signalą), platus konteksto langas ir galimybė veikti turimame aparatinės įrangos biudžete. Pasiūlytojų modeliai, neatitikę šių kriterijų, aptariami poskyrio pabaigoje.

Pool A apima tris kodui specializuotus modelius, kurių parametrų skaičius artimas septyniems milijardams. „Qwen2.5-Coder“ [Hyc+24] yra „Alibaba“ laboratorijos sukurtas modelis, sudarantis platesnės „Qwen2.5“ šeimos dalį [Yyz+25]. Autorių teigimu, „Qwen2.5-Coder“ šeima apima šešių dydžių modelius - nuo 0,5 iki 32 milijardų parametrų - ir buvo mokyta naudojant daugiau nei 5,5 trilijono žetonų [Hyc+24]. Eksperimentuose naudojama septynių milijardų parametrų versija (Ollama modelio žyma `qwen2.5-coder:7b`) su eksperimente nustatytu 32 768 žetonų konteksto langu - ši reikšmė atitinka sistemoje nustatomą Ollama parametą `num_ctx` (žr. 2.1.4 poskyrį). „DeepSeek-Coder“ [Gzy+24] yra ankstesnės kartos „DeepSeek“ programavimo kalbų modelių šeima, apimanti dydžius nuo 1,3 iki 33 milijardų parametrų, mokyta naudojant du trilijonus žetonų ir pasižyminti 16 tūkst. žetonų konteksto langu [Gzy+24]. Eksperimentuose naudojama 6,7 milijardo parametrų versija (Ollama modelio žyma `deepseek-coder:6.7b`). „Code Llama“ [Rgg+23] yra „Meta“ laboratorijos kodui specializuotų modelių šeima, paremta „Llama 2“ baziniu modeliu ir papildomai mokyta naudojant kodo korpusą. Originaliame straipsnyje pristatyti trys pagrindiniai modelių dydžiai - septynių, trylikos ir 34 milijardų parametrų - bei papildomas instrukcijų vykdymui pritaikytas variantas, leidžiantis modeliui vykdyti natūralios kalbos užduotis [Rgg+23]. Eksperimentuose naudojama septynių milijardų parametrų instrukcijų vykdymo versija (Ollama modelio žyma `codellama:7b-instruct`); bazinis, instrukcijoms nepritaikytas variantas šio rinkinio sudarymo metu buvo atmestas dėl negebėjimo vykdyti struktūrizuotų debatų užklausų (plačiau - poskyrio pabaigoje).

Pool B apima tris kiek didesnius kodui specializuotus modelius, kurių parametrų skaičius svyruoja nuo aštuonių iki devynių milijardų. „Granite Code“ [Msz+24] yra „IBM“ laboratorijos atvirųjų svorių modelių šeima, apimanti modelius nuo trijų iki 34 milijardų parametrų ir orientuota į kodo intelekto užduotis [Msz+24]. Eksperimentuose naudojama aštuonių milijardų parametrų versija (Ollama modelio žyma `granite-code:8b`). „CodeGeeX4“ yra devynių

milijardų parametrų „CodeGeeX” serijos ketvirtosios kartos programavimo kalbų modelis. Pirminė „CodeGeeX” serija pristatyta Zheng ir kt. [Zxz+23] darbe, kuriame aprašytas pirmosios kartos 13 milijardų parametrų modelis, mokytas naudojant 850 milijardų žetonų 23 programavimo kalbomis (2022 metų birželio mėnesio duomenimis); šis šaltinis šiame darbe naudojamas kaip „CodeGeeX” serijos kontekstas, o konkreti eksperimentuose naudota ketvirtosios kartos versija nurodoma pagal Ollama modelio žymą `codegeex4:9b`. „Yi-Coder” yra kodui specializuotas „Yi” šeimos darinys [Ycl+24], paremtas „01.AI” laboratorijos sukurta „Yi” šeima, kuri, autorių teigimu, buvo mokyta naudojant 3,1 trilijono žetonų anglų ir kinų kalbų duomenų rinkinyje ir pasižymi iki 200 tūkst. žetonų konteksto langu [Ycl+24]. Eksperimentuose naudojama devynių milijardų parametrų kodui specializuota versija (Ollama modelio žyma `yi-coder:9b`).

Teisėjo agento vaidmeniui buvo parinkti du skirtingos specializacijos modeliai - tai leidžia tirti, ar teisėjo specializacijos kryptis turi reikšmingos įtakos kritikos kokybei kelių agentų debatuose. Šio palyginimo eksperimentiniai rezultatai pristatomi 4.4 poskyryje.

„DeepSeek-Coder-V2” [Dee24] šiame darbe naudojamas kaip kodo specializacijos teisėjo modelis. Jis pasižymi 128 tūkst. žetonų konteksto langu ir, autorių teigimu, buvo papildomai mokytas naudojant šešių trilijonų žetonų duomenų rinkinį, lyginant su ankstesnės kartos „DeepSeek” modeliu [Dee24]. Didžioji šeimos versija (236 milijardų bendrų ir 21 milijardo aktyvių parametrų mišrių ekspertų modelis) reikalautų reikšmingai didesnių aparatinės įrangos išteklių, nei leidžia vienos „NVIDIA V100” grafikos plokštės 32 GB atminties biudžetas. Dėl šios priežasties eksperimentuose naudojama „DeepSeek-Coder-V2-Lite” versija (Ollama modelio žyma `deepseek-coder-v2:16b`) - 16 milijardų bendrų parametrų ir 2,4 milijardo aktyvių parametrų modelis, taip pat paremtas mišrių ekspertų (angl. Mixture of Experts, MoE) architektūra.

„DeepSeek-R1” [Dee25] šiame darbe naudojamas kaip samprotavimo specializacijos teisėjo modelis, sukurtas taikant pastiprinamojo mokymosi (angl. reinforcement learning) metodiką. Autorių teigimu, modelis pasižymi gerais rezultatais sprendžiant matematikos, varžybinio programavimo ir tikslųjų mokslų (STEM) uždavinius [Dee25]. Didžioji pirminio modelio versija yra 671 milijardo parametrų mišrių ekspertų sistema, todėl eksperimentuose, kaip ir „DeepSeek-Coder-V2” atveju, naudojama mažesnė distiliuota (angl. distillation) versija „DeepSeek-R1-Distill-Qwen-14B” (Ollama modelio žyma `deepseek-r1:14b`) - 14 milijardų parametrų modelis, gautas distiliuojant pirminio „DeepSeek-R1” samprotavimo gebėjimus į „Qwen2.5-14B” pagrindą. Autorių teigimu, toks distiliavimas leidžia išlaikyti didelę dalį originalaus modelio samprotavimo galimybių, kartu sumažindamas atminties poreikį taip, kad modelį būtų galima vykdyti vienos „NVIDIA V100” grafikos plokštės atminties biudžete.

Dviejų skirtingo profilio teisėjų pasirinkimas leidžia atskirti kodo specializacijos ir bendrojo samprotavimo poveikį debatų kritikos kokybei. „DeepSeek-Coder-V2-Lite” yra orientuotas į programavimo kalbų sintaksę ir logines kodo klaidas, todėl tikimasi, kad jis efektyviau aptiks konkrečias kodavimo klaidas. „DeepSeek-R1-Distill-Qwen-14B”, priešingai, yra bendresnės paskirties samprotavimo modelis su žingsnis po žingsnio mąstymo grandinės (angl. chain-of-thought) gebėjimu, kuris turėtų geriau atskleisti algoritminio lygmens klaidas. Šių dviejų profilių palyginimas leidžia atsakyti į klausimą, ar debatuose kodo specializacijos teisėjas pranoksta samprotavimo teisėją, ar atvirkščiai; šis klausimas yra pagrindinis 4.4 poskyrio eksperimentinis akcentas.

Preliminariuose bandymuose tikrintos kelios papildomos modelių alternatyvos, kurios nepateko į galutinį rinkinį. Bazinis, instrukcijoms nepritaikytas `codellama:7b` variantas ir „StarCoder2” modeliai nesugebėjo vykdyti debatų sistemoje naudojamų struktūrizuotų užklausų - užuot atsakę į PROPOSAL, CRITIQUE ar VOTE užklausą, šie modeliai bandė tęsti įvesties tekstą natūraliu būdu, todėl debatų protokolas iškart sutrikdavo. Tokio elgesio priežastis

- šie modeliai daugiausia mokyti kodo užbaigimo (angl. completion) tipo užduotims, be papildomo instrukcijų vykdymui pritaikyto apmokymo, kuriuo pasižymi visi pagrindiniuose eksperimentuose naudoti modeliai. „Mistral 7B” [Jsm+23] yra septynių milijardų parametrų atvirųjų svorių kalbos modelis, sukurtas „Mistral AI” laboratorijoje. Originalus straipsnis jį pristato kaip kalbos modelį, gebantį spręsti uždavinius samprotavimo, matematikos ir programavimo srityse [Jsm+23]. Nors paskutinė iš šių sričių apima ir programų kūrimą, „Mistral 7B” nebuvo specialiai papildomai mokytas naudojant didelės apimties kodo korpusą, kaip „Code Llama”, „Qwen2.5-Coder”, „DeepSeek-Coder” ir kiti šio darbo rinkinio modeliai. Dėl šios priežasties „Mistral 7B” į pagrindinį tyrimo rinkinį neįtraukiamas; šiame poskyryje jis aptariamas tik kaip preliminariniuose bandymuose atmestas bendrosios paskirties 7B klasės modelio pavyzdys. Visi tyrimo metu naudoti modeliai apibendrinti 3 lentelėje.

Modelis	Ollama modelio žyma	Parametrai	Konteksto langas	Vaidmuo
„Qwen2.5-Coder”	qwen2.5-coder:7b	7 mlrd.	32K	Pasiūlytojas (Pool A)
„DeepSeek-Coder”	deepseek-coder:6.7b	6,7 mlrd.	16K	Pasiūlytojas (Pool A)
„Code Llama”	codellama:7b-instruct	7 mlrd.	16K	Pasiūlytojas (Pool A)
„Granite Code”	granite-code:8b	8 mlrd.	128K	Pasiūlytojas (Pool B)
„CodeGeeX4”	codegeex4:9b	9 mlrd.	128K	Pasiūlytojas (Pool B)
„Yi-Coder”	yi-coder:9b	9 mlrd.	128K	Pasiūlytojas (Pool B)
„DeepSeek-Coder-V2-Lite”	deepseek-coder-v2:16b	16 mlrd. bendrų / 2,4 mlrd. aktyvių	128K	Teisėjas (kodo specializacijos)
„DeepSeek-R1-Distill-Qwen-14B”	deepseek-r1:14b	14 mlrd., distiliuotas	128K	Teisėjas (samprotavimo specializacijos)

3 lentelė. Eksperimentuose naudotų modelių apžvalga. Konteksto langas pateiktas binariniu skaičiavimu (1K = 1024 žetonai).

3.3. Užduočių rinkinio sudarymas

Eksperimentinis užduočių rinkinys sudarytas iš dviejų katalogų - pirminio tasks/ ir išplėstinio tasks2/. Pirminis katalogas apima 60 užduočių, suskirstytų į keturis sudėtingumo lygius po penkiolika užduočių (lengvas, vidutinis, sudėtingas, ekstremalus); jis naudojamas kaip plataus spektro vertinimo bazė, leidžianti įvertinti debatų metodo veikimą skirtingo sudėtingumo užduotyse. Išplėstinis katalogas apima 60 užduočių (dvidešimt sudėtingo ir keturiasdešimt ekstremalaus lygio); dalis šių užduočių sutampa su pirminiu katalogu, o dalis prideda naujus sudėtingus ir ekstremalius scenarijus. Šis katalogas orientuotas išskirtinai į sudėtingesnes užduotis, kuriose pradinio vieno modelio atsakymo paprastai nepakanka. Tokia dviejų rinkinių struktūra leidžia atskirti dvi tyrimo plotmes: bendrojo metodo veikimą platesniame sudėtingumo spektre (tasks/ rinkinys) ir konkretų debatų metodologijos poveikį sudėtingoms užduotims (tasks2/ rinkinys).

Pirminis tasks/ katalogas sudarytas iš algoritminių programavimo užduočių, adaptuotų iš atvirosios programavimo užduočių platformos „LeetCode” kanoninių problemų ir kelių klasikinių kompiuterijos užduočių iš algoritmų vadovėlių. Lengvajame sudėtingumo lygyje (penkiolika užduočių) dvylika problemų atitinka „LeetCode” Easy kategorijos užduotis (pavyzdžiui, „Two Sum”, „Valid Parentheses”, „Binary Search”), o trys yra klasikiniai programavimo pratimai - Cezario šifro realizavimas, balsių skaičiavimas ir įdėtų sąrašų

išlyginimas. Vidutiniame lygyje (penkiolika užduočių) visos problemos kyla iš „LeetCode” Medium kategorijos (pavyzdžiui, „LRU Cache”, „Coin Change”, „Group Anagrams”, „Implement Trie”). Sudėtingajame lygyje (penkiolika užduočių) dvylika problemų yra adaptuotos iš „LeetCode” Hard kategorijos (pavyzdžiui, „Median of Two Sorted Arrays”, „Regular Expression Matching”, „Word Ladder”), o trys yra klasikinės kompiuterijos užduotys: trumpiausių kelių paieška grafe (Dijkstra ir Bellman-Ford algoritmai), inversijų skaičiavimas masyve bei dvikrypčio susietojo sąrašo bibliotekos realizacija. Visų užduočių aprašymai šiame darbe perrašyti, paliekant tą pačią funkcijos sąsają ir testavimo semantiką, o visi vienetiniai testai parašyti šio darbo metu ir nėra paimti iš „LeetCode” platformos uždarojo testų rinkinio.

Ekstremalaus sudėtingumo lygio užduotys (penkiolika tasks/ kataloge ir keturiasdešimt tasks2/ kataloge) yra specialiai šiam darbui parengtos. Kiekvienos ekstremalaus lygio užduoties sprendimui reikia dviejų ar trijų atskirų „Python” modulių su tarpmoduliniais importais (žr. 2.5 poskyrį), todėl modelis turi ne tik teisingai realizuoti algoritmą, bet ir nuosekliai suderinti klasių sąsajas, eksportus ir importus tarp failų. Šios kategorijos užduotys apima parserius ir interpretatorius (calculator, json_parser, expression_evaluator, mini_interpreter), klasikinius programavimo šablonus (event_system, plugin_system, state_machine, di_container, circuit_breaker), duomenų struktūras su valdymo logika (b_tree, bloom_filter, cache_with_ttl, state_persistence), programinės įrangos karkasus (http_router, http_client, mini_database, simple_orm) bei darbų ir priklausomybių valdymo scenarijus (task_scheduler, workflow_engine, dependency_resolver).

Tokia ekstremalaus lygio užduočių struktūra parengta atsižvelgiant į praktinį iššūkį, kurio standartiniai vertinimo rinkiniai tiesiogiai neaprepia. Chen ir kt. [Ctj+21] „HumanEval” ir Liu ir kt. [Lxw+23] „EvalPlus” rinkiniai yra orientuoti į vienos funkcijos užduotis, o Jimenez ir kt. [Jyw+24] „SWE-bench” rinkinys, nors ir apima realias daugiafailės programų inžinerijos problemas, yra per sudėtingas atvirųjų svorių 7-9 milijardų parametrų modeliams (žr. 1.1 poskyrį). Preliminariuose bandymuose pastebėta, kad šios klasės modeliai daugiafailėse užduotyse dažnai painioja modulių sąsajas, praleidžia būtinus importus arba grąžina nepilną sprendimą - tai motyvavo atskirą daugiafailių užduočių rinkinio sudarymą, kuris leidžia tirti, ar kelių agentų debatai gali kompensuoti ši individualių modelių gebėjimų trūkumą per kritikos ir revizijos ciklą.

Išplėstinis tasks2/ katalogas naudojamas kaip pagrindinio etapo antrosios eksperimentų bangos (žr. 3.1 poskyrį) užduočių pagrindas, įtrauktas siekiant sutelkti vertinimą į sudėtingiausius scenarijus. Iš dvidešimties tasks2/hard užduočių penkios yra naujos „LeetCode” Hard kategorijos problemos, neįtrauktos į pirminį tasks/ katalogą („Edit Distance”, „Largest Rectangle in Histogram”, „Longest Increasing Subsequence”, „N-Queens” bei „Palindrome Partitioning”); likusios penkiolika užduočių sutampa su tasks/hard užduotimis ir naudojamos kaip ta pati užduočių aibė pakartotiniam vertinimui naujose eksperimentinėse konfigūracijose. Iš keturiasdešimties tasks2/extreme užduočių dvidešimt penkios yra naujos šio darbo metu suprojektuotos daugiafailės užduotys, papildančios pirminį penkiolikos užduočių rinkinį tiek temine įvairove, tiek techniniu sudėtingumu. Bendras unikalių algoritminių problemų skaičius dviejuose kataloguose, pašalinus pasikartojimus, yra devyniasdešimt.

Kiekviena užduotis aprašoma JSON formatu su standartiniais laukais: identifikatoriumi (id), pavadinimu (name), sudėtingumo lygiu (difficulty), užduoties aprašymu (description), funkcijų sąsajomis (signature), vienetinių testų sąrašu (tests) ir apribojimais (constraints). Ekstremalaus lygio užduotys papildomai turi tris laukus: required_files (generuotinių failų sąrašas), test_imports (testavimo aplinkos importai iš sugeneruotų modulių) ir tags (užduoties klasifikacijos žymos). 16 kodo fragmente pateiktas užduoties JSON struktūros pavyzdys.

```
# tasks/easy/two_sum.json (sutrumpinta versija)
{
  "id": "two_sum",
  "name": "Two Sum",
  "difficulty": "easy",
  "description": "Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target...",
  "signature": "def two_sum(nums: list[int], target: int) -> list[int]:",
  "tests": [
    "def test_basic():\n    assert sorted(two_sum([2, 7, 11, 15], 9)) == [0, 1]",
    "def test_negative():\n    assert sorted(two_sum([-1, -2, -3, -4, -5], -8)) == [2, 4]"
  ],
  "constraints": [
    "2 <= nums.length <= 10^4",
    "Only one valid answer exists"
  ]
}
```

16 kodo fragmentas. Užduoties aprašymo JSON struktūros pavyzdys (lengvojo lygio užduotis „Two Sum“).

Vienetinių testų skaičius vienai užduočiai svyruoja nuo keturių iki šešiolikos; vidutiniškai vienai užduočiai tenka 8,91 testo (žr. 2.7.4 poskyrį, kuriame šis vidurkis aptariamas testų pakankamumo kontekste). Toks testų skaičius leidžia užtikrinti pakankamai detalių funkcinių padengimą, ypač sudėtingoms užduotims, kur reikalingi kraštinių atvejų patikrinimai.

Užduočių patikimumas tikrinamas papildomu validavimo žingsniu. Repozitorijos šaknyje sukurtas `validate_tests.py` scenarijus, kuriame pačiame scenarijuje pateiktos referencinių sprendimų realizacijos visoms sudėtingo ir ekstremalaus lygio užduotims (`HARD_IMPLEMENTATIONS` ir `EXTREME_IMPLEMENTATIONS` struktūrose). Šis scenarijus vykdo kiekvieną referencinį sprendimą su atitinkamos užduoties vienetinių testų rinkiniu ir nustato, ar visi testai išlaikomi - tai leidžia patikrinti, kad užduočių testai nėra sudaryti savavališkai, bet yra suderinami su žinomu teisingu sprendimu. Lengvojo ir vidutinio lygio užduotys patikrintos rankiniu būdu, kadangi jų sprendimai paprastai užima mažiau nei dvidešimt eilučių ir tiesiogiai atspindi gerai žinomus algoritmus.

Galiausiai reikia pažymėti svarbų vertinimo metodikos apribojimą. Lengvojo, vidutinio ir sudėtingo lygio užduočių formuluotės yra panašios į „LeetCode“ platformos problemas, kurios yra plačiai paplitusios internete ir galimai sudaro dalį šio darbo eksperimentuose naudotų atvirųjų svorių modelių pirminių treniravimo duomenų. Šis duomenų užterštumo (angl. data contamination) reiškinys yra žinomas didžiųjų kalbos modelių kodo generavimo vertinimo apribojimas, kurį pažymėjo Liu ir kt. [Lxw+23] „EvalPlus“ rinkinio darbe. Ekstremalaus lygio užduotims, specialiai šiam darbui parengtoms, ši rizika yra mažiau būdinga, todėl jos suteikia patikimesnį empirinį signalą apie modelių gebėjimą generuoti kodą mažiau žinomose daugiafailėse užduotyse. Į šį aspektą atsižvelgiama analizuojant 4 skyriaus rezultatus, ypač sudėtingo ir ekstremalaus lygio užduočių palyginimą. Visi tyrimo metu naudoti užduočių rinkiniai apibendrinti 4 lentelėje.

Sudėtingumo lygis	tasks/	tasks2/	Unikalių užduočių skaičius	Pagrindiniai šaltiniai
Lengvas	15	-	15	„LeetCode“ Easy (12); klasikiniai pratimai (3)
Vidutinis	15	-	15	„LeetCode“ Medium (15)
Sudėtingas	15	20	20	„LeetCode“ Hard (17); algoritmų vadovėliai (3)

Ekstremalus	15	40	40	Šiam darbui parengtos daugiafailės užduotys (40)
Iš viso	60	60	90	

4 lentelė. Užduočių rinkinio sudėtis pagal sudėtingumo lygį ir šaltinius. Skiltyje „Unikalių užduočių“ pateiktas atskirų algoritminių problemų skaičius po dedubliavimo tarp katalogų (tasks/hard sutampa su pirmosiomis 15 tasks2/hard užduočių, o tasks/extreme - su pirmosiomis 15 tasks2/extreme užduočių).

3.4. Eksperimentų scenarijai ir vykdymo sąlygos

Šio darbo eksperimentai vykdomi dviem etapais: preliminariniame, naudojant interaktyvią žiniatinklio sąsają, ir pagrindiniame, atliekamame paketiniu režimu VU MIF didelio našumo skaičiavimo (HPC) klasteryje. Pirmasis etapas skirtas sistemos parametrams suderinti ir komponentų sąveikai patikrinti, antrasis - sisteminiam dešimties debatų konfigūracijų ir vienos solo bazinės linijos vykdymui (žr. 3.1 poskyrį).

3.4.1. Žiniatinklio sąsaja preliminariniams bandymams

Šio darbo žiniatinklio sąsaja, techniškai aprašyta 2.8 poskyryje, eksperimentiniame kontekste naudojama preliminariniams bandymams atlikti. Šiame etape naudojant žiniatinklio sąsają paleidžiami pavieniai debatai vienai ar kelioms užduotims, siekiant suderinti sistemos parametrus prieš pradedant pagrindinį eksperimentų etapą HPC aplinkoje. Preliminariuose bandymuose interaktyviai buvo keičiami šie parametrai: maksimalus raundų skaičius, agentų rinkinio sudėtis, teisėjo agento įjungimas arba išjungimas, revizijos strategija, adaptyvios temperatūros ir kritikos istorijos įjungimo parametrai. Pagrindinis šio etapo rezultatas - galutinė sistemos konfigūracija, kuri toliau be pakeitimų taikoma visuose dešimtyje pagrindinio etapo HPC debatų eksperimentų (3.1 poskyryje aprašytas fiksuotų parametrų rinkinys). Be to, preliminariniuose bandymuose pastebėti individualių 7-9 milijardų parametrų modelių sunkumai daugiafailėse užduotyse motyvavo atskiro tasks2/ užduočių rinkinio sudarymą (žr. 3.3 poskyrį).

3.4.2. Didelio našumo skaičiavimo (HPC) aplinka

Pagrindinis eksperimentų etapas vykdomas Vilniaus universiteto Matematikos ir informatikos fakulteto (VU MIF) didelio našumo skaičiavimo (HPC) klasteryje. Paketiniam darbų valdymui naudojama „SLURM“ sistema, o programinės įrangos izoliacijai - „Singularity“ konteinerizacija. Į „SLURM“ sistemą pagrindinio etapo metu pateikiama vienuolika nepriklausomų darbų, iš kurių kiekvienas atitinka vieną iš 3.1 poskyrio 2 lentelėje aprašytų konfigūracijų. Papildomo adaptyvios temperatūros mechanizmo išjungimo A/B palyginimo (angl. ablation study) metu vėliau į „SLURM“ sistemą buvo pateikti dar keturi darbai, kuriuose tasks2/ rinkinio debatų konfigūracijos pakartotos išjungus adaptyvią temperatūrą (žr. 4.5 poskyrį). Tokia paketinė schema leidžia užduočių vykdymą perkelti iš vietinės darbo stoties į specializuotą aparatinę infrastruktūrą, kurioje ištekliai per „SLURM“ rezervuojami visam debatų ciklui.

Kiekvienam „SLURM“ darbui naudojama vienoda išteklių užklausa: viena „NVIDIA V100“ grafikos plokštė (gres=gpu:1), penki procesoriaus branduoliai (cpus-per-task=5), šešiasdešimt gigabaitų operatyviosios atminties (mem=60G) ir šešių valandų vykdymo limitas (time=06:00:00). Solo bazinės linijos darbui, apimančiam šešis modelius po šešiasdešimt užduočių, nustatomas aštuonių valandų vykdymo limitas. Darbas pateikiamas į gpu „SLURM“ skaidinį (partition=gpu), kuriame prieinamos „NVIDIA V100“ 32 GB grafikos plokštės, kurių atminties pakanka keliems 7-9 milijardų parametrų modeliams ir 14-16 milijardų parametrų teisėjo modeliui vykdyti kartu.

Debatų eksperimentų konfigūracijų valdymas paremtas bendrosios vykdymo bibliotekos principu. Pagrindinio etapo dešimt debatų apvalkalo scenarijų (angl. wrapper scripts; hpc/run_7b_*.sh ir hpc/run_9b_*.sh) yra minimalūs scenarijai, nustatantys keturis ar penkis aplinkos kintamuosius - peer modelių rinkinį, teisėjo modelį, eksperimento identifikatorių, papildomus komandinės eilutės parametrus ir užduočių katalogų rinkinį, - o tada iškviečiantys

bendrąją `hpc/_lib_run.sh` biblioteką, kuri atlieka „Ollama” serverio paleidimą, modelių atsisiuntimą, debatų ciklo vykdymą visoms pasirinkto rinkinio užduotims ir galutinį CSV eksportą. Tokia struktūra užtikrina, kad pati vykdymo logika - „Ollama” konfigūracija, klaidų apdorojimas, atskirų užduočių vykdymo ciklas - liktų vienoda visoms dešimčiai debatų konfigūracijų, o tarp jų skirtųsi tik tie kintamieji, kurie atspindi konkrečią palyginimo ašį. Solo bazinės linijos scenarijus (`hpc/run_all6_solo.sh`) paliekamas savarankiškas, nes jo vykdymo logika skiriasi nuo debatų eksperimentų: kiekvienas iš šešių modelių visas užduotis sprendžia atskirai, todėl „Ollama” paleidimo ir užduočių ciklo fazės pakartojamos paties scenarijaus viduje, o pabaigoje atliekamas atskiras CSV eksportas.

Vykdymo metu kiekvienam užduočių katalogo JSON failui paleidžiamas atskiras python3 -m `src.main` iškviatimas su atitinkamais peer modeliais ir, kai taikoma, teisėjo modeliu, perduodant pagrindinio etapo scenarijuose fiksuotus `--adaptive-temperature` ir `--critique-history` parametrus (žr. 3.1 poskyrį) bei maksimalų penkių raundų skaičių. Kiekvieno debato rezultatas iš karto įrašomas į bendrą „SQLite” duomenų bazę `debate_results.db`; pasibaigus visam darbo ciklui, python3 -m `src.analysis.csv_export` komanda eksportuoja du CSV failus, kurių pavadinimuose nurodomas eksperimento identifikatorius ir „SLURM” darbo numeris (pavyzdžiui, `summary_9b_judge2_r1_217803.csv`). Tokia eksporto schema užtikrina, kad kiekvienas eksperimentas turi atskirą rezultatų rinkinį, o duomenų bazėje įrašai kaupiami nuosekliai, todėl galima atkurti bet kurią ankstesnę rezultatų būseną.

„NVIDIA V100” 32 GB grafikos plokštės atminties biudžetas išnaudojamas beveik visiškai. Trijų peer modelių rinkinio svoriai užima nuo trylikos iki penkiolikos gigabaitų (Pool A ir Pool B atitinkamai), papildomas šešiolikos arba keturiolikos milijardų parametrų teisėjas - apie devynis gigabaitus, o raktų ir reikšmių podėlio dinaminis naudojimas svyruoja nuo keturių iki penkių gigabaitų priklausomai nuo konteksto lango ir aktyvios sekos ilgio. Iš viso atminties poreikis pasiekia apie dvidešimt šešis-dvidešimt devynis gigabaitus, paliekant rezervą „Ollama” serveriui ir „Singularity” konteinerio aplinkai. Šio biudžeto pakanka visoms aprašytoms konfigūracijoms, įskaitant labiausiai išteklių reikalaujantį derinį (Pool B kartu su „DeepSeek-R1-Distill-Qwen-14B” samprotavimo teisėju), kuriame atminties naudojimas yra ribiniame diapazone, tačiau eksperimentų metu atminties trūkumo (angl. *out-of-memory*) klaidų nebuvo užfiksuota.

3.5. Vertinimo metrikos

Šiame poskyryje aprašomos konkrečios metrikos, kuriomis vertinami eksperimentinių konfigūracijų rezultatai. Visos metrikos automatiškai kaupiamos debatų metu ir eksportuojamos į du CSV failus - apibendrinamąjį ir raundų lygmens, - todėl 4 skyriaus analizė remiasi išsamia objektyvių rodiklių baze. Metrikos apima keturias vertinimo kryptis: galutinio sprendimo teisingumą, debatų pridėtinę vertę, kodo kokybę ir skaičiavimo sąnaudas.

Pagrindinis sprendimo teisingumo rodiklis - galutinis testų išlaikymo rodiklis (`final_pass_rate`), apibrėžiamas kaip užduoties vienetinių testų, kuriuos sėkmingai išlaiko galutinis debato metu pasirinktas sprendimas, dalis. Šis rodiklis kinta intervale nuo nulio iki vieno ir tiesiogiai atspindi funkcinį sprendimo korektiškumą. Šalia pagrindinio rodiklio skaičiuojamos `pass@1` ir `pass@3` metrikos, pristatytos Chen ir kt. [Ctj+21] „HumanEval” rinkinyje (formulė pateikta 2.9.4 poskyrio 15 kodo fragmente), atitinkančios atitinkamai vieno bandymo ir trijų bandymų sėkmę. Šio darbo eksperimentuose `pass@3` natūraliai dera su trijų pasiūlytojų rinkiniu, nes leidžia įvertinti, ar bent vienas iš trijų pradinių nepriklausomų sprendimų pasiekia teisingą rezultatą, ir šį rodiklį palyginti su galutiniu debatų konsensuso rezultatu.

Norint atskirti, kuriose užduotyse debatai iš tiesų prisidėjo prie galutinio rezultato, naudojamos dvi specifinės metrikos. `peak_after_debate` (loginė reikšmė) įgyja reikšmę

„teisinga”, jei geriausias debato metu pasiektas testų išlaikymo rodiklis pirmą kartą pasiekiamas antrajame ar vėlesniame raunde, t. y. ne pirmajame pasiūlymo raunde. Ši metrika atskiria atvejus, kai debatų ciklas prisidėjo prie sprendimo pagerinimo, nuo atvejų, kai sprendimas buvo rastas iškart, o tolesni raundai tik pakartojo pirminį geriausią rezultatą. `improvement_over_best_initial` papildomai matuoja santykinį galutinio sprendimo pagerėjimą lyginant su geriausiu pradinio raundo sprendimu; teigiama reikšmė rodo debatų ciklo pagerinimą, neigiama - regresiją, kuri sistemoje neturėtų pasitaikyti dėl anti-regresijos politikos, kuri automatiškai atkuria ankstesnį geriausią sprendimą, kai revizija jį pablogina.

Greta funkcinio korektiškumo, kiekvienam sprendimui apskaičiuojami statinės analizės rodikliai - „Pylint” balas dešimtbalėje skalėje ir „Radon” ciklominis sudėtingumas. Šie rodikliai sudaro antrą vertinimo ašį: tarp sprendimų, išlaikiusių vienodą testų skaičių, kokybės rodikliai leidžia atskirti, ar konkreti konfigūracija (pavyzdžiui, debatas su teisėju) pateikia ne tik teisingesnį, bet ir struktūriškai geresnį kodą. Pradinio raundo vidurkiai (`initial_avg_pylint`, `initial_avg_complexity`) lyginami su galutinio sprendimo reikšmėmis (`final_pylint`, `final_complexity`), kad būtų matomas debatų ciklo poveikis kodo kokybei.

Debatų ciklo diagnostikai naudojamos kelios papildomos metrikos. `consensus_reached` (loginė reikšmė) ir `rounds_to_consensus` rodo, kuriose konfigūracijose ir kuriame raunde pasiekiamas formalus konsensusas - situacija, kai bent 60 procentų agentų balsų svorio tenka tam pačiam sprendimui. Ankstyvas konsensuso pasiekimas dažniausiai siejamas su lengvesnėmis užduotimis, turinčiomis vienareikšmiškai geriausią sprendimą, o vėlyvas - su sudėtingomis užduotimis arba modelių nesutarimu. Kritikos efektyvumas vertinamas `bug_fix_rate` rodikliu, kuris išreiškia ištaisytų klaidų dalį tarp visų debato metu užfiksuotų klaidų: `total_bugs_fixed` padalytas iš `total_bugs_found`. Šis rodiklis ypač svarbus vertinant teisėjo modelio įtaką - didelis aptiktų klaidų skaičius, bet maža jų ištaisymo dalis rodytų, kad teisėjas randa daugiau problemų, nei sistema sugeba ištaisyti revizijos metu.

Šio darbo eksperimentinių konfigūracijų skaičiavimo sąnaudos vertinamos trimis laiko rodikliais: `duration_seconds` (bendra debato trukmė nuo pradžios iki pabaigos), `total_llm_time` (bendras laikas, praleistas DKM iškvietimuose) ir `total_execution_time` (bendras laikas, praleistas vykdant kodą „pytest” izoliuotoje aplinkoje). Skaičiavimo sąnaudų vertinimas yra ypač svarbus praktinio metodo taikymo požiūriu - kelių agentų debatų metodas nuo solo bazinės linijos skiriasi ne tik kokybe, bet ir reikšmingai didesniu vykdymo laiku, todėl 4 skyriuje pateikiama ne tik kokybės pagerėjimo, bet ir skirtingų konfigūracijų kokybės bei sąnaudų santykio (angl. quality-vs-cost) analizė.

Visos aprašytos metrikos tiesiogiai siejasi su 3.1 poskyryje suformuluotomis palyginimo ašimis. Debatų ir solo bazinės linijos palyginimas pirmiausia remiasi `final_pass_rate` ir `improvement_over_best_initial` rodikliais; Pool A ir Pool B palyginimas - `final_pass_rate` skirstiniu pagal sudėtingumo lygius; teisėjo įjungimo poveikis vertinamas pagal `final_pass_rate` ir `bug_fix_rate` rodiklius; o kodo specializacijos ir samprotavimo teisėjo palyginimas - tomis pačiomis metrikomis kartu su skaičiavimo sąnaudų rodikliais, leidžiančiais įvertinti, ar samprotavimo teisėjo papildoma vertė pateisina galimai didesnes inferencijos sąnaudas.

Skyriaus apibendrinimas. Šiame skyriuje aprašyta tyrimo metodika remiasi kontroliuojamo palyginimo principu: atskirai vertinami agentų rinkiniai, teisėjo modelio vaidmuo, debatų ir solo režimų skirtumai bei papildomų sistemos mechanizmų poveikis. Du užduočių rinkiniai, `tasks/` ir `tasks2/`, leidžia tikrinti metodą skirtingo sudėtingumo užduotyse, o eksperimentų vykdymas VU MIF HPC aplinkoje užtikrina vienodas pagrindinių konfigūracijų vykdymo sąlygas. Pagal šią metodiką gauti rezultatai analizuojami 4 skyriuje.

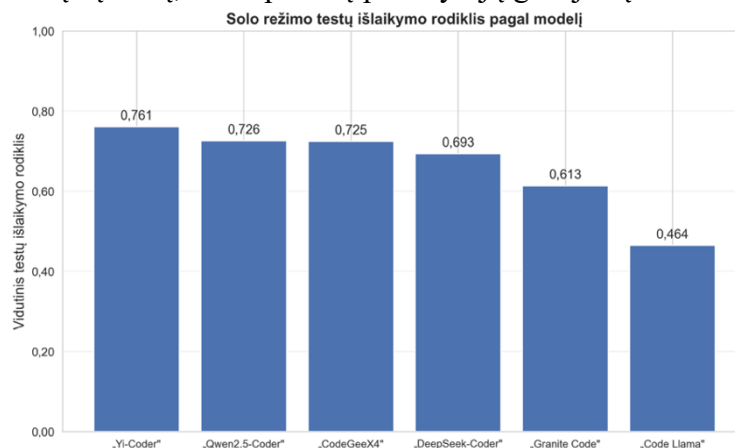
4. Eksperimentų rezultatai ir jų analizė

Šiame skyriuje pristatomi 3 skyriuje aprašytos metodikos pagrindu atliktų vienuolikos SLURM eksperimentų rezultatai - iš viso 960 unikalių vykdymo įrašų (600 debatų ir 360 solo bandymų) - ir jų analizė. Papildomai 4.5 poskyryje pateikiamas atskiras adaptyvios temperatūros mechanizmo A/B abliacijos eksperimentas. Pirma aptariama solo bazinė linija (4.1), parodanti, ką pasiekia kiekvienas iš šešių pasiūlytojų modelių, vykdomas atskirai be debatų ciklo; vėliau pateikiamas tiesioginis debatų ir solo režimo palyginimas (4.2), agentų rinkinio sudėties įtaka (4.3), teisėjo modelio tipo poveikis kartu su klaidų aptikimo ir taisymo analize (4.4) bei debatų raidos kiekybinė analizė (4.5). Atskiruose poskyriuose pateikiama kokybinė tipiskų debatų scenarijų analizė (4.6), skaičiavimo sąnaudų ir kokybės kompromiso vertinimas (4.7) bei rezultatų aptarimas literatūros kontekste (4.8). Skyriuje taikomi aprašomosios, lyginamosios ir statistinės analizės metodai - dalis svarbiausių palyginimų papildoma porinio Stjudento t kriterijaus rezultatais. Visi pateikiami skaičiai gauti tiesiogiai iš sistemos eksportuotų CSV failų (žr. 2.9.3 ir 3.4.2 poskyrius), filtruojant pagal eksperimento konfigūraciją (stulpeliai mode ir agent_models).

4.1. Solo bazinės linijos rezultatai

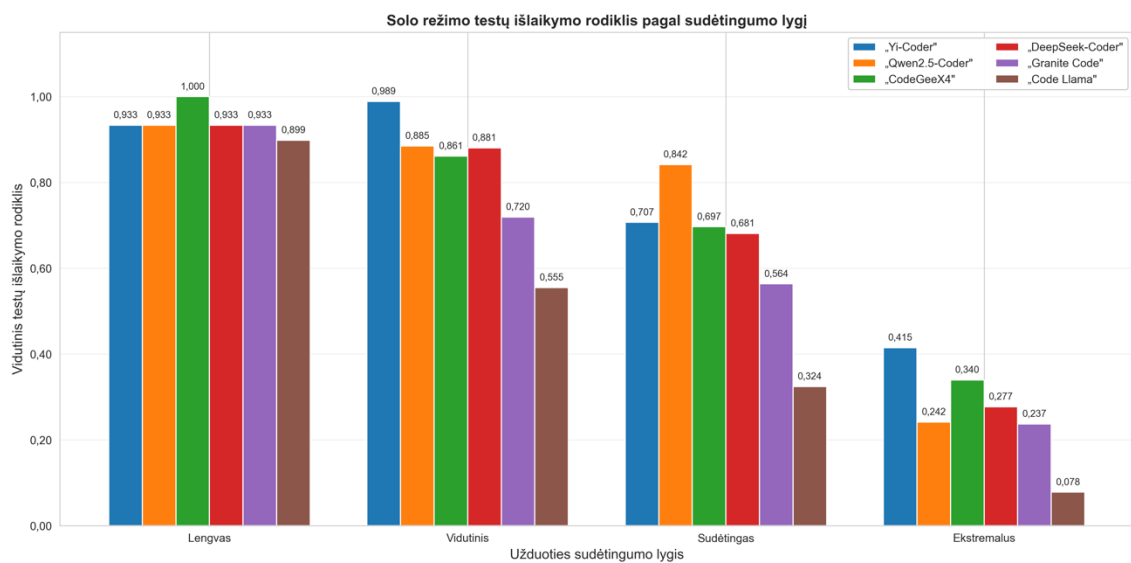
Solo bazinė linija sudaro atskaitos tašką visiems vėlesniems debatų režimo palyginimams. Kiekvienas iš šešių pasiūlytojų modelių - „Qwen2.5-Coder“, „DeepSeek-Coder“, „Code Llama“ instrukcijų vykdymo variantas, „Granite Code“, „CodeGeeX4“ ir „Yi-Coder“ (žr. 3.2 poskyrį) - savarankiškai sprendė visas šešiasdešimt pirmojo užduočių rinkinio (tasks/) užduočių be kritikos, revizijos ir balsavimo fazių. Iš viso atlikta 360 nepriklausomų solo bandymų (šeši modeliai po šešiasdešimt užduočių), kurių rezultatai eksportuoti į CSV failą `summary_all6_solo_217797.csv`. Pagrindinis vertinimo rodiklis - galutinis testų išlaikymo rodiklis (`final_pass_rate`, žr. 3.5 poskyrį), apibrėžiamas kaip užduoties vienetinių testų, kuriuos sėkmingai išlaiko modelio pateiktas sprendimas, dalis. Tai vidutinė rodiklio reikšmė per 60 užduočių, o ne visiškai išspręstų užduočių dalis - šis skirtumas detaliau aptariamas šio poskyrio pabaigoje.

Bendras testų išlaikymo rodiklis pagal modelį parodytas 2 diagramoje („Solo režimo testų išlaikymo rodiklis pagal modelį“). Aukščiausią rezultatą pasiekia „Yi-Coder“ (0,761), antras yra „Qwen2.5-Coder“ (0,726), trečias - „CodeGeeX4“ (0,725); šie trys modeliai sudaro glaudžiai išsidėsčiusią pirmaujančią grupę, kurios rezultatų skirtumas nesiekia keturių šimtųjų. Antrą lygmenį sudaro „DeepSeek-Coder“ (0,693) ir „Granite Code“ (0,613); silpniausią rezultatą pasiekia „Code Llama“ instrukcijų vykdymo variantas (0,464), atsiliekantis nuo pirmaujančio modelio beveik 30 procentinių punktų. Vidutinis rodiklis tarp visų šešių modelių yra 0,664. Pažymėtina, kad solo bazinės linijos pirmaujantis modelis - 9 milijardų parametru „Yi-Coder“ - savarankiškai jau pasiekia gerokai aukštesnį rezultatą nei silpniausi 7 milijardų parametru modeliai, todėl 4.3 poskyryje aprašomas Pool A (7B) ir Pool B (9B) palyginimas atspindi ne tik debatų sąveiką, bet ir pradinį pasiūlytojų gebėjimų skirtumą.

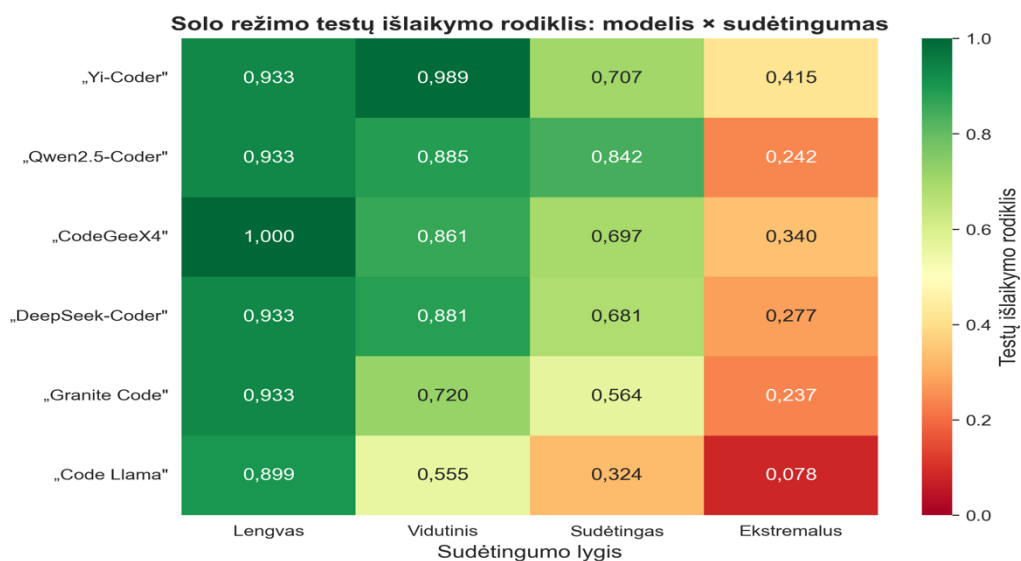


2 diagrama. Solo režimo testų išlaikymo rodiklis pagal modelį

Solo režimo rezultatų pasiskirstymas pagal sudėtingumo lygį pateiktas 3 diagramoje („Solo režimo testų išlaikymo rodiklis pagal sudėtingumo lygį“) ir 5 lentelėje, o jų apibendrintas vaizdas - 4 diagramoje („Solo režimo testų išlaikymo rodiklis: modelis $\times$ sudėtingumas“) šiluminio žemėlapiu pavidalu. Lengvojo lygio užduotyse visi modeliai pasiekia bent 0,899 rodiklį; „CodeGeeX4“ išlaiko visus testus visose penkiolikoje lengvojo lygio užduočių (1,000), o keturių kitų modelių rodiklis siekia 0,933. Vidutinio lygio užduotyse išryškėja didesnis modelių rezultatų skirtumas: „Yi-Coder“ pasiekia 0,989, tačiau „Code Llama“ rodiklis nukrenta iki 0,555, o „Granite Code“ - iki 0,720. Sudėtingajame lygyje pasiūlytojų rodikliai jau svyruoja nuo 0,324 („Code Llama“) iki 0,842 („Qwen2.5-Coder“). Įdomu tai, kad būtent šiame lygyje „Qwen2.5-Coder“ pranoksta „Yi-Coder“ (0,707), todėl bendras „Yi-Coder“ pirmavimas neišlieka vienodas visuose sudėtingumo lygiuose. Ekstremaliojo lygio užduotyse net pirmaujančio „Yi-Coder“ rodiklis krenta iki 0,415, o „Code Llama“ pasiekia tik 0,078 - tai reiškia, kad iš vidutiniškai aštuonių-devynių vienetinių testų užduotyje (žr. 3.3 poskyrį) vidutiniškai išlaikomas mažiau nei vienas testas. Visi šeši modeliai ekstremaliojo lygio užduotyse patiria reikšmingą rezultatų kritimą, kuris ir motyvuoja kelių agentų debatų metodo taikymą - būtent šio sudėtingumo užduotyse vieno modelio gebėjimai pasirodo nepakankami.



3 diagrama. Solo režimo testų išlaikymo rodiklis pagal sudėtingumo lygį

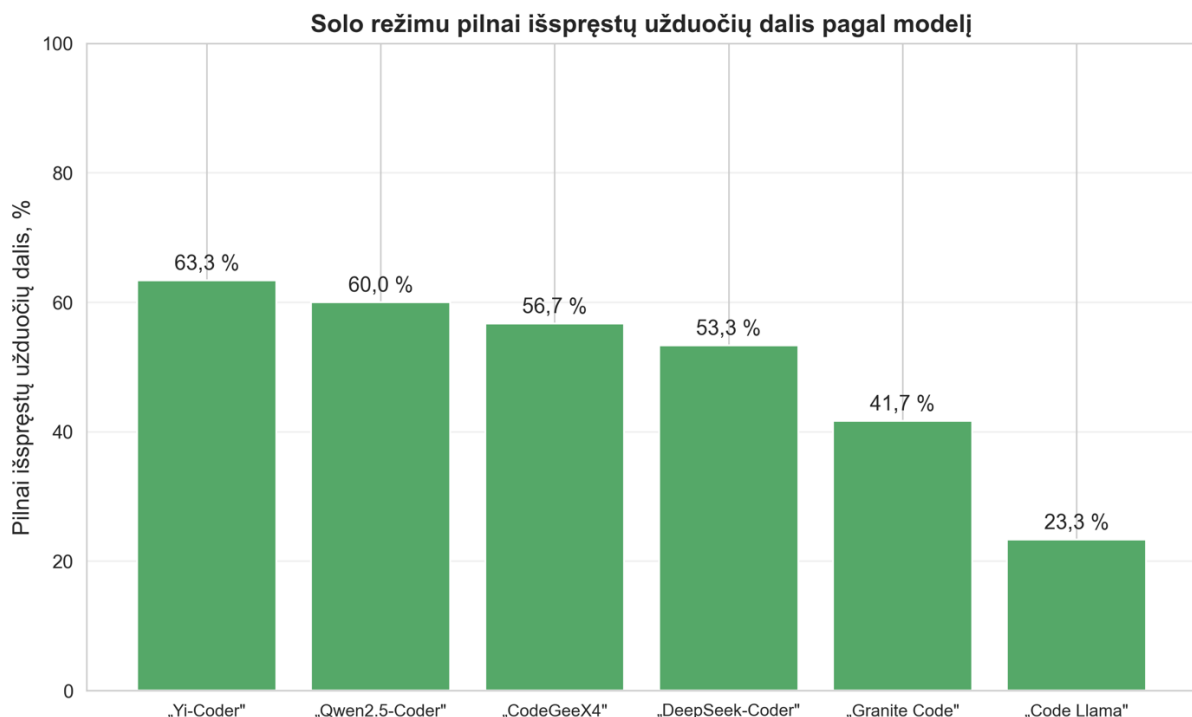


4 diagrama. Solo režimo testų išlaikymo rodiklis: modelis $\times$ sudėtingumas

Modelis	Lengvas	Vidutinis	Sudėtingas	Ekstremalus	Bendras
„Yi-Coder”	0,933	0,989	0,707	0,415	0,761
„Qwen2.5-Coder”	0,933	0,885	0,842	0,242	0,726
„CodeGeeX4”	1,000	0,861	0,697	0,340	0,725
„DeepSeek-Coder”	0,933	0,881	0,681	0,277	0,693
„Granite Code”	0,933	0,720	0,564	0,237	0,613
„Code Llama”	0,899	0,555	0,324	0,078	0,464
Vidurkis	0,939	0,815	0,636	0,265	0,664

5 lentelė. Solo bazinės linijos galutinio testų išlaikymo rodiklio (final_pass_rate) vidurkis pagal modelį ir sudėtingumo lygį (60 užduočių iš tasks/ rinkinio, 360 solo bandymų).

Vidutinis testų išlaikymo rodiklis savaime neatspindi visiškai išspręstų užduočių dalies. Užduotis laikoma visiškai išspręsta, jeigu modelio sprendimas išlaiko visus vienetinius testus (final_pass_rate=1,0); dalinis išlaikymas (pavyzdžiui, septyni iš dešimties testų), nors ir prisideda prie vidutinio rodiklio, neatspindi realiai veikiančio sprendimo. 5 diagramoje („Solo režimu visiškai išspręstų užduočių dalis pagal modelį”) pateikta visiškai išspręstų užduočių dalis kiekvienam modeliui parodo griežtesnį vaizdą: „Yi-Coder” visiškai išsprendžia 38 iš 60 užduočių (63,3%), „Qwen2.5-Coder” - 36 iš 60 (60,0%), „CodeGeeX4” - 34 iš 60 (56,7%), „DeepSeek-Coder” - 32 iš 60 (53,3%), „Granite Code” - 25 iš 60 (41,7%), o „Code Llama” - tik 14 iš 60 (23,3%). Skirtumas tarp pirmaujančio ir silpniausio modelio šiuo rodikliu siekia keturiasdešimt procentinių punktų. Ekstremaliojo lygio užduotyse situacija dar griežtesnė: „DeepSeek-Coder” ir „Code Llama” nesugeba visiškai išspręsti nė vienos iš penkiolikos ekstremaliojo lygio užduočių, „Qwen2.5-Coder” ir „Granite Code” - tik po vieną, „CodeGeeX4” - dvi, o „Yi-Coder” - tris. Šis rezultatas yra svarbus 4.5 ir 4.6 poskyrių analizei, kurioje aptariama, kaip kelių agentų debatai gali pagerinti tokių užduočių sprendimo kokybę, kai individualūs modeliai jų visiškai išspręsti negali.



5 diagrama. Solo režimu visiškai išspręstų užduočių dalis pagal modelį

Apibendrinant, solo bazinės linijos rezultatai patvirtina dvi prielaidas, kurios buvo padarytos sudarant užduočių rinkinį (žr. 3.3 poskyrį). Pirma, lengvojo ir vidutinio sudėtingumo užduotyse 7-9 milijardų parametrų modeliai dažnai pasiekia aukštą testų išlaikymą jau pirmuoju bandymu, todėl debatų pridėtinė vertė šioje srityje turėtų būti ribota - šis numatomas efektas eksperimentiškai patikrinamas 4.2 ir 4.5 poskyriuose. Antra, ekstremaliojo lygio užduotyse vieno modelio rodikliai krenta gerokai žemiau 0,50, o visiškai išspręstų užduočių dalis šiame lygyje vidutiniškai siekia tik apie aštuonis procentus (septynios visiškai išspręstos užduotys iš 90 atvejų, kai vertinami šeši modeliai po penkiolika ekstremaliojo lygio užduočių); tai rodo didelį rezultatų gerinimo potencialą, kuriame galima patikimai įvertinti debatų metodo realią naudą. Tolesnė šio skyriaus analizė remiasi šia bazine linija - visi debatų režimo rezultatai pristatomi būtent jos atžvilgiu.

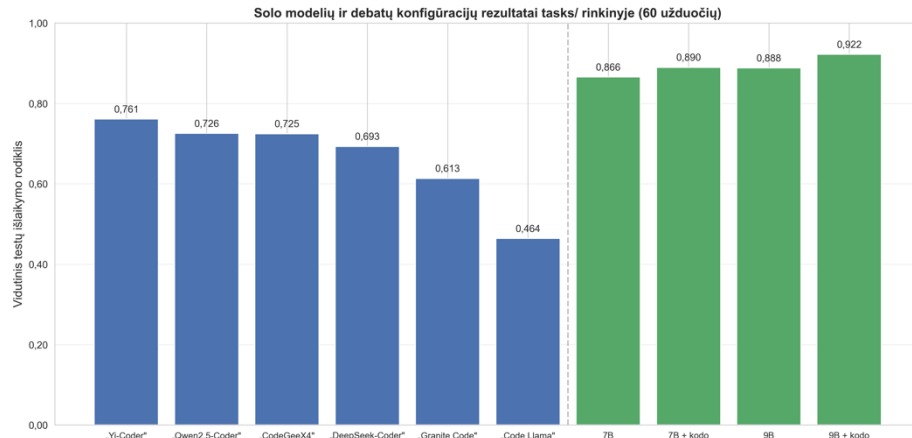
4.2. Debatų ir solo režimo palyginimas

Šis poskyris pristato tiesioginį debatų ir vieno modelio (solo) režimo palyginimą - centrinį šio darbo tyrimo klausimą. 4.1 poskyryje aprašyta solo bazinė linija sudaro atskaitos tašką, o šiame poskyryje vertinama, kiek kelių agentų debatų sistema pranoksta atskirai vykdomo modelio rezultatą. Palyginimui šio poskyrio agreguotuose rodikliuose naudojamos keturios pagrindinės debatų konfigūracijos, kurios buvo vykdytos abiejuose užduočių rinkiniuose: 7B klasės agentų rinkinys (Pool A) be teisėjo, tas pats rinkinys su „DeepSeek-Coder-V2” kodo specializacijos teisėju, 9B klasės agentų rinkinys (Pool B) be teisėjo ir su tuo pačiu teisėju. Penktoji ir šeštoji konfigūracijos - 7B ir 9B rinkiniai su samprotavimo specializacijos teisėju „DeepSeek-R1” - buvo vykdytos tik antrajame, sudėtingesniame užduočių rinkinyje (tasks2/) ir detaliai analizuojamos 4.4 poskyryje, kuriame nagrinėjama teisėjo modelio tipo įtaka. Šiame poskyryje R1 konfigūracijos atskirai aptariamos tik bendrojo stabilumo patikrinime (žr. 11 lentelę ir 11 diagramą).

Solo režimo bazinė linija prieinama pirminiame užduočių rinkinyje tasks/ (60 užduočių), kuriame buvo atliktas atskiras solo bandymas kiekvienam iš šešių pasiūlytojų modelių (360 solo įrašų). Antrajame rinkinyje tasks2/ (60 sudėtingo ir ekstremalaus lygio užduočių) atskiri solo bandymai nebuvo vykdomi - šis rinkinys buvo įtrauktas vėliau, siekiant tikslingai sutelkti vertinimą į sudėtingiausius scenarijus (žr. 3.3 poskyrį). Dėl šios priežasties tasks2/ rinkinyje vieno modelio atskaitos tašku naudojamas pirmojo debatų raundo (R1) vidurkis. Pirmajame raunde kiekvienas pasiūlytojas savarankiškai sugeneruoja sprendimą, neturėdamas kritikos, revizijos ar kitų agentų sprendimų informacijos - funkcinio požiūriu tai artima solo režimui, nes kiekvienas komandos narys užduotį sprendžia atskirai (žr. 2.2.2 poskyrį). Vis dėlto svarbu pabrėžti metodologinį skirtumą: R1 raundas vyksta debato sistemos užklausos kontekste, kuris nėra visiškai tapatus atskiros solo režimo užklausiai. Empirinė patikra tasks/ rinkinyje rodo, kad to paties modelio R1 rezultatas yra šiek tiek žemesnis už jo atskirą solo rezultatą - skirtumas svyruoja maždaug nuo 2 iki 4 procentinių punktų (pavyzdžiui, „Yi-Coder” solo režimu pasiekia 76,1 %, o R1 raunde toje pačioje 60 užduočių aplinkoje - 73,3 %). Todėl tasks2/ rinkinio rezultatai turi būti interpretuojami atsargiai: R1 atskaitos taškas nėra visiškai tapatus atskiram solo režimui ir, remiantis tasks/ rinkinio patikra, gali šiek tiek padidinti matomą debatų pranašumą tasks2/ rinkinyje. Dėl šios priežasties šiame poskyryje aiškiai atskiriamas tikrasis solo palyginimas tasks/ rinkinyje ir R1 pagrindu sudarytas palyginimas tasks2/ rinkinyje.

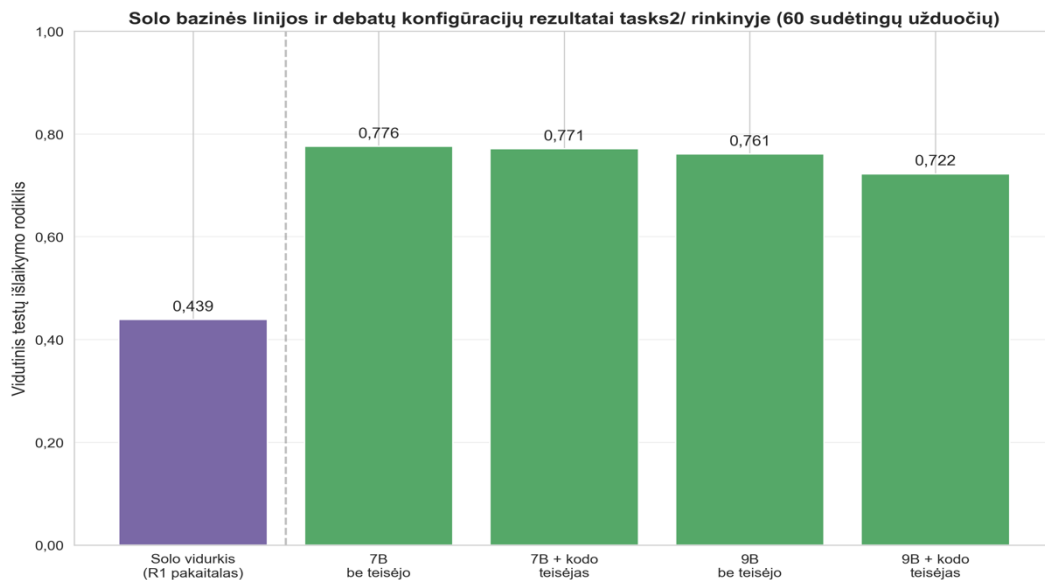
Pirminiame tasks/ rinkinyje (60 užduočių) debatai aiškiai pranoksta solo bazinę liniją (žr. 6 diagramą). Vidutinis solo režimo testų išlaikymo rodiklis tarp visų šešių modelių per visas užduotis siekia 0,664, o vidutinis debatų rodiklis tarp visų keturių konfigūracijų - 0,891. Skirtumas - 22,7 procentinio punkto. Porinis Stjudento t kriterijus, taikomas 60 užduočių poriniam palyginimui, suteikia statistinį $t = 9,47$ ir reikšmingumo lygį $p < 10^{-13}$, todėl stebimas pranašumas yra statistiškai reikšmingas. Iš 60 užduočių 49 atvejais debatų vidurkis pranoksta solo vidurkį, 11 atvejų rezultatai sutampa - daugiausia tai lengvojo ir vidutinio lygio užduotys,

kuriose abu režimai pasiekia 100% testų išlaikymą. Solo pranašumo nėra vienoje užduotyje nenustatyta.

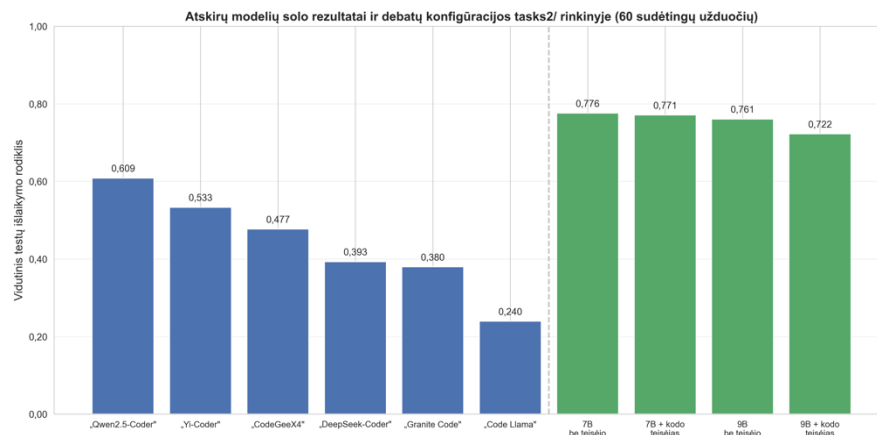


6 diagrama. Solo modelių ir debatų konfigūracijų rezultatai tasks/ rinkinyje (60 užduočių)

Antrajame tasks2/ rinkinyje, kurį sudaro 30 naujų ir 30 su tasks/ sutampančių sudėtingo ir ekstremalaus lygio užduočių, debatų pranašumas tampa dar ryškesnis (žr. 7 ir 8 diagramas). Vidutinis R1 raundo rodiklis siekia 0,439, o vidutinis debatų rodiklis - 0,758, skirtumas - 31,9 procentinio punkto. Porinis t kriterijus parodė $t = 16,76$, $p < 10^{-24}$. Iš 60 užduočių 59 atvejais debatai pranoksta R1 raundą, 1 atveju pasiekiamas tas pats rezultatas, o pralaimėjimų - 0. Tai reikšmingai didesnis pranašumas nei pirminiame rinkinyje - tiek absoliučia prasme (+31,9 prieš +22,7 procentinio punkto), tiek santykinė prasme (santykinis pagerėjimas +73% prieš +34% nuo bazinės linijos). Šis kontrastas patvirtina 3.3 poskyryje aprašytą metodologinį pasirinkimą sudaryti antrą sudėtingesnių užduočių rinkinį būtent tam, kad būtų išryškintas realus debatų pridėtinės vertės dydis užduotyse, kurioms vieno modelio sprendimo nepakanka.



7 diagrama. R1 bazinės linijos ir debatų konfigūracijų rezultatai tasks2/ rinkinyje



8 diagrama. Atskirų modelių R1 ir debatų konfigūracijų rezultatai tasks2/ rinkinyje

Sujungus abu rinkinius, gaunami 120 užduoties-rinkinio porinių palyginimų (60 iš tasks/ ir 60 iš tasks2/). Šis skaičius nereiškia 120 visiškai unikalių užduočių, nes 30 užduočių sutampa tarp abiejų rinkinių, tačiau buvo pakartotinai testuotos nepriklausomuose vykdymuose. Bendras keturių pagrindinių konfigūracijų debatų vidurkis siekia 0,824, o bazinės linijos vidurkis - 0,551, skirtumas - 27,3 procentinio punkto. Porinis t kriterijus parodė $t = 17,27$, $p < 10^{-34}$. Iš 120 porinių palyginimų debatai laimi 108, baigia lygiosiomis 12 ir nė karto nepralaimi. Šie agreguoti rodikliai pateikti 6, 7 ir 8 lentelėse.

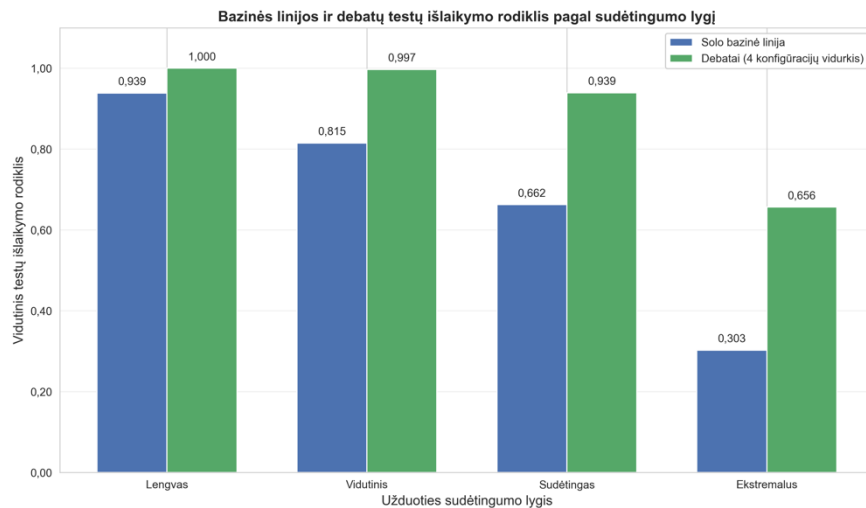
Konfigūracija	tasks/ rinkinys (60 užd.)	tasks2/ rinkinys (60 užd.)	Sujungtas (120 porų)
Bazinė linija (solo / R1)	0,664	0,439	0,551
Debatai: 7B be teisėjo	0,866	0,776	0,821
Debatai: 7B + kodo teisėjas	0,890	0,771	0,830
Debatai: 9B be teisėjo	0,888	0,761	0,824
Debatai: 9B + kodo teisėjas	0,922	0,722	0,822
Debatų vidurkis (4 konfig.)	0,891	0,758	0,824
Δ (debatai - bazinė linija)	+22,7 proc. p.	+31,9 proc. p.	+27,3 proc. p.

6 lentelė. Vidutinis galutinis testų išlaikymo rodiklis (final_pass_rate) pagal konfigūraciją ir užduočių rinkinį.

Konfigūracijų grupė	Porų skaičius	Pergalės	Lygiosios	Pralaimėjimai	t	p-value
4 pagr. konfig. - tasks/	60	49	11	0	9,47	$< 10^{-13}$
4 pagr. konfig. - tasks2/	60	59	1	0	16,76	$< 10^{-24}$
Iš viso (4 pagr. konfig.)	120	108	12	0	17,27	$< 10^{-34}$

7 lentelė. Porinis debatų ir bazinės linijos palyginimas užduočių lygyje (vidurkis tarp konfigūracijų) ir Stjudento t kriterijaus rezultatai.

Pasiskirstymas pagal sudėtingumo lygį parodo, kad debatų pridėtinė vertė monotoniškai didėja kartu su užduoties sunkumu (žr. 9 diagramą ir 8 lentelę). Lengvojo lygio užduotyse skirtumas tarp bazinės linijos ir debatų yra +6,1 procentinio punkto - debatai paprasčiausiai pakelia jau aukštą solo rodiklį iki 100%. Vidutinio lygio užduotyse skirtumas didėja iki +18,2 procentinio punkto, sudėtingajame lygyje - iki +27,7, o ekstremaliojo lygio užduotyse pasiekia +35,3 procentinio punkto. Tai reiškia, kad ekstremaliojo lygio užduotyse, kuriose vidutinis solo režimo rodiklis svyruoja apie 0,30, debatų ciklas pakelia rezultatą iki 0,66 - daugiau nei dvigubai. Šis monotoninis didėjimas tiesiogiai patvirtina 4.1 poskyrio pabaigoje suformuluotą prielaidą: debatų metodas turi didžiausią praktinę vertę būtent toms užduotims, kurioms vieno modelio sprendimas nepakanka.

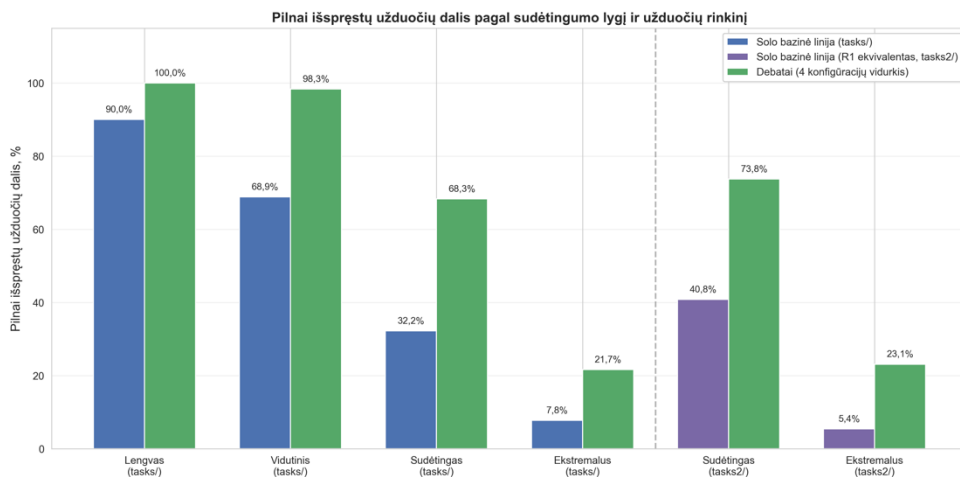


9 diagrama. Bazinės linijos ir debatų testų išlaikymo rodiklis pagal sudėtingumo lygį

Sudėtingumo lygis	Bazinė linija	Debatų	Δ (procentinių punktų)
Lengvas	0,939	1,000	+6,1
Vidutinis	0,815	0,997	+18,2
Sudėtingas	0,662	0,939	+27,7
Ekstremalus	0,303	0,656	+35,3

8 lentelė. Vidutinis galutinis testų išlaikymo rodiklis pagal sudėtingumo lygį (sujungtas tasks/ ir tasks2/ rinkinys, 4 debatų konfigūracijų vidurkis).

Vidutinis testų išlaikymo rodiklis savaime neatspindi visiškai išspręstų užduočių dalies (žr. 4.1 poskyrio pabaigą). Todėl papildomai vertinama griežtesnė vertinimo metrika - dalis užduočių, kurioms modelio sprendimas išlaikė visus vienetinius testus ($\text{final_pass_rate} = 1,0$) - rodo dar ryškesnį kontrastą tarp solo ir debatų režimo. 10 diagramoje ir 9 lentelėje pateikti šios metrikos rodikliai: pirminiame tasks/ rinkinyje vidutinė solo režimo modelio visiškai išspręstų užduočių dalis siekia 49,7%, o debatų - 72,1%; antrajame tasks2/ rinkinyje (R1 atskaitos taškas) atitinkamai 17,2% ir 40,0%. Ekstremaliojo lygio užduotyse pirminiame rinkinyje vidutinis solo modelis visiškai išsprendžia tik 7,8% užduočių, o debatai - 21,7%; antrajame rinkinyje - 5,4% prieš 23,1%. Šie skaičiai reiškia, kad sudėtingiausio tipo užduotyse debatai kelis kartus padidina visiškai išsprendžiamų užduočių dalį.



10 diagrama. Pilnai išspręstų užduočių dalis pagal sudėtingumo lygį ir užduočių rinkinį

Sudėtingumo lygis	tasks/ Bazinė linija (solo)	tasks/ Debatai	tasks2/ Bazinė linija (R1)	tasks2/ Debatai
Lengvas	90,0	100,0	-	-
Vidutinis	68,9	98,3	-	-
Sudėtingas	32,2	68,3	40,8	73,8
Ekstremalus	7,8	21,7	5,4	23,1

9 lentelė. Pilnai išspręstų užduočių dalis (procentais) pagal sudėtingumo lygį ir užduočių rinkinį.

Atskirai verta aptarti, kaip debatai lyginasi su geriausiu atskiru solo modeliu, o ne su vidurkiu tarp visų šešių (žr. 8 diagramą ir 10 lentelę). Tokia analizė reikšminga todėl, kad praktikoje sistemos kūrėjas dažnai jau žinotų stipriausią savo užduočiai modelį. 10 lentelėje pateiktas tiesioginis palyginimas: kiekviename sudėtingumo lygyje atrenkamas tos sudėtingumo grupės geriausias solo modelis ir lyginamas su debatų vidurkiu. Lengvojo lygio užduotyse geriausias modelis („CodeGeeX4“, 1,000) pasiekia tą patį rezultatą kaip debatai - 100% testų išlaikymą. Vidutinio lygio užduotyse geriausias modelis („Yi-Coder“, 0,989) yra praktiškai lygus debatams (0,997), skirtumas tik 0,8 procentinio punkto. Sudėtingajame lygyje skirtumas išryškėja: geriausias modelis „Qwen2.5-Coder“ pasiekia 0,842, o debatai - 0,924 (+8,3 procentinio punkto). Ekstremaliojo lygio užduotyse net pirmaujantis solo modelis „Yi-Coder“ pasiekia tik 0,415, o debatai - 0,644, skirtumas - 22,9 procentinio punkto. Antrajame tasks2/ rinkinyje, kuriame stipriausias modelis pagal R1 raundo rezultatus yra „Qwen2.5-Coder“, ekstremaliojo lygio užduotyse skirtumas vis tiek išlieka 21,3 procentinio punkto debatų naudai. Pažymėtina, kad geriausias solo modelis nėra tas pats per visus sudėtingumo lygius - „CodeGeeX4“ pirmauja lengvajame, „Yi-Coder“ - vidutiniame ir ekstremaliajame pirminio rinkinio lygyje, „Qwen2.5-Coder“ - sudėtingajame lygyje ir abiejuose tasks2/ lygiuose. Tai reiškia, kad nėra universaliai stipriausio modelio, o debatų sistema stabiliai pranoksta bet kurio individualaus modelio pasiekimą sudėtinguose scenarijuose.

Sudėtingumas	Rinkinys	Geriausias solo modelis	Solo rezultatas	Debatų vidurkis	Δ (proc. p.)
Lengvas	tasks/	„CodeGeeX4“	1,000	1,000	+0,0
Vidutinis	tasks/	„Yi-Coder“	0,989	0,997	+0,8
Sudėtingas	tasks/	„Qwen2.5-Coder“	0,842	0,924	+8,3
Ekstremalus	tasks/	„Yi-Coder“	0,415	0,644	+22,9
Sudėtingas	tasks2/	„Qwen2.5-Coder“ (R1)	0,929	0,950	+2,1

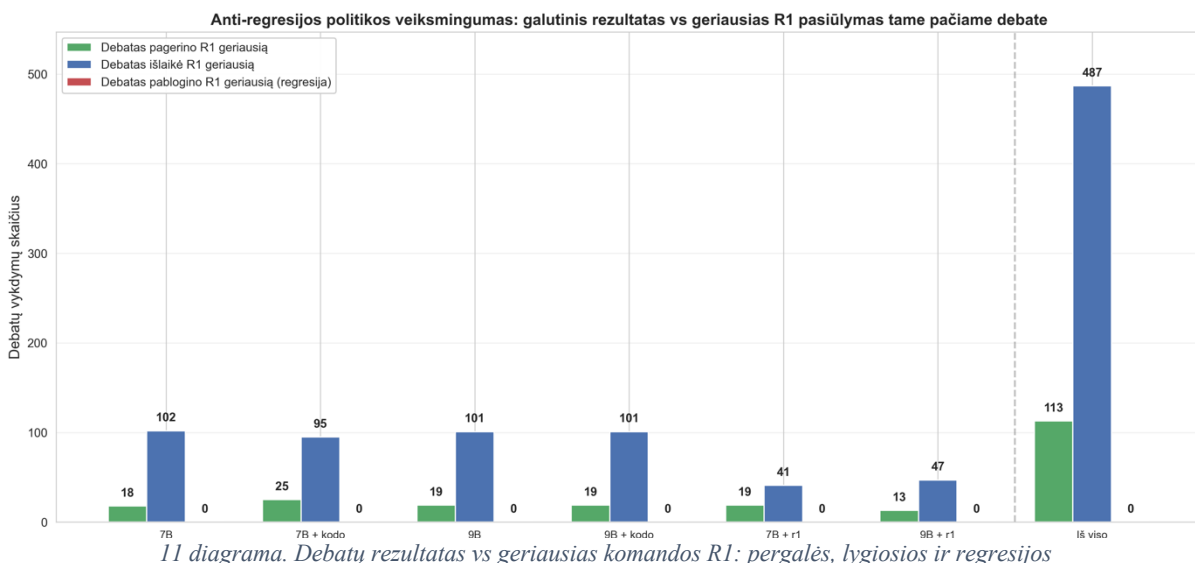
Ekstremalus	tasks2/	„Qwen2.5-Coder” (R1)	0,449	0,661	+21,3
-------------	---------	----------------------	-------	-------	-------

10 lentelė. Geriausio solo modelio ir debatų vidurkio palyginimas pagal sudėtingumo lygį ir užduočių rinkinį.

Papildomas stabilumo patikrinimas atliktas lyginant galutinį debatų rezultatą su geriausiu komandos nario R1 rezultatu tame pačiame debate. Kaip minėta anksčiau, pirmojo raundo rezultatai funkcinio požiūriu artimi agentų solo pasirodymui, tačiau nėra visiškai tapatūs atskiram solo režimui, todėl geriausio komandos nario R1 rezultatas (R1_max) gali būti interpretuojamas kaip „geriausias solo rezultatas šios komandos viduje” - tai vieno modelio atskaitos taško realizacija, taikoma visiems debatams nepriklausomai nuo užduočių rinkinio. Iš 600 atliktų debatų vykdymų visose šešiose konfigūracijose 113 atvejų (18,8 %) debatų ciklas pranoko geriausią komandos nario R1 rezultatą, 487 atvejais (81,2 %) galutinis rezultatas išliko lygus geriausiam R1, o pablogėjimo atvejų nenustatyta. Vykdytų pasiskirstymas pagal konfigūracijas pateiktas 11 lentelėje ir 11 diagramoje. Šis nuoseklus stabilumas yra tiesioginė 2.4.1 poskyryje aprašytos anti-regresijos politikos pasekmė: revizijos metu prarasti sprendimai automatiškai atstatomi iš istorinio kiekvieno agento maksimumo. Šis rezultatas turi būti interpretuojamas kaip stabilumo patikrinimas konkrečioje eksperimentinėje sistemoje, kurioje naudojama anti-regresijos politika ir vienetiniai testai. Jis nereiškia, kad bet kokia debatų sistema bet kokiomis sąlygomis negali pabloginti rezultato; šio darbo kontekste išvada siauresnė, bet empiriškai tvirta: įgyvendinta debatų sistema pagal final_pass_rate metriką nė karto nepablogino geriausio pradinio komandos sprendimo.

Konfigūracija	Vykdytų skaičius	Debatas pranoko R1_max	Debatas išlaikė R1_max	Debatas pablogino R1_max
7B be teisėjo	120	18	102	0
7B + kodo teisėjas	120	25	95	0
9B be teisėjo	120	19	101	0
9B + kodo teisėjas	120	19	101	0
7B + R1 (tik tasks2/)	60	19	41	0
9B + R1 (tik tasks2/)	60	13	47	0
Iš viso	600	113	487	0

11 lentelė. Debatų galutinio rezultato palyginimas su geriausiu komandos nario R1 rezultatu tame pačiame debate (R1_max - geriausias komandos nario rezultatas pirmajame raunde)



Pažymėtina, kad atskirų debatų vykdymų lygyje (480 pagrindinių konfigūracijų vykdymų tasks/ ir tasks2/ rinkiniuose) 7 atvejais (1,46 %) konkreti konfigūracija pateikė rezultatą, žemesnį už solo vidurkį užduotyje. Visi šie atvejai pasitaikė užduotyse, kuriose debatų komandoje nebuvo stipriausio konkrečiai užduočiai tinkamo modelio - tai atspindi komandos sudėties ribotumą, o ne debatų metodo trūkumą - agreguotame keturių konfigūracijų vidurkyje šiose pačiose užduotyse debatai nė karto nepasirodė prasčiau už solo bazinę liniją.

Papildomai įvertintas debatų gebėjimas pagerinti visiškai nesėkmingus pradinius bandymus. Tam išskirti atvejai, kai geriausias pirmojo raundo rezultatas buvo lygus nuliui ($R1_max = 0$), t. y. nė vienas iš trijų pasiūlytojų neišlaikė nė vieno testo. Keturių pagrindinių konfigūracijų grupėje aptikta 211 tokių pradinių situacijų: 49 atvejai 7B konfigūracijoje be teisėjo, 51 - 7B konfigūracijoje su kodo teisėju, 53 - 9B konfigūracijoje be teisėjo ir 58 - 9B konfigūracijoje su kodo teisėju. Iš jų debatų ciklas pagerino rezultatą iki bent dalinai veikiančio sprendimo atitinkamai 41, 48, 48 ir 54 atvejais. Tai atitinka 83,7 %, 94,1 %, 90,6 % ir 93,1 % pagerinimo rodiklius. Įtraukus ir dvi „DeepSeek-R1” teisėjo konfigūracijas, iš 282 visiškai nesėkmingų pradinių raundų debatai pagerino 254 atvejus, t. y. 90,1 %. Šis gelbėjimo rodiklis (angl. rescue rate) paaiškina, kodėl debatų pridėtinė vertė ypač išryškėja sudėtingose užduotyse: net kai visi pradiniai sprendimai yra neteisingi, kritikos ir revizijos ciklas dažnai leidžia pasiekti bent iš dalies veikiantį sprendimą.

Šis efektas ypač ryškus žvelgiant į silpniausią pasiūlytojų rinkinio modelį - „Code Llama” instrukcijų vykdymo variantą, kuris solo režimu ekstremaliojo lygio užduotyse pasiekia tik 0,078 testų išlaikymo. Tačiau šis pats modelis, dalyvaudamas 7B Pool A debatuose su kodo specializacijos teisėju, ekstremaliojo lygio užduotyse prisideda prie 0,676 vidurkio - daugiau nei aštuonis kartus aukštesnio rezultato. Tai rodo, kad komanda, kurioje dalyvauja ir silpnesnis modelis, vis tiek gali pasiekti rezultatą, gerokai aukštesnį už šio modelio solo režimą. Detalesnė kokybinė analizė su konkrečių debatų transkripto pavyzdžiais (įskaitant tipiškas „rescue effect” scenarijus, kaip užduotys „Priority Event Bus with Filters” ar „HTTP Router with Middleware”, kuriose visi pasiūlytojai pirmajame raunde pateikia 0 % rezultatą, o debatų ciklas iškelia jį iki 88-100 %) pateikiama 4.6 poskyryje.

Aprašyti rezultatai atitinka platesnio pobūdžio literatūros išvadas dėl kelių agentų debatų metodo efektyvumo, tačiau jų tiesioginis kiekybinis lyginimas yra metodologiškai apribotas. Pradinis kelių agentų debatų darbas - Du ir kt. [Dlt+23] - pademonstravo, kad debatai pagerina rezultatus matematinio bei strateginio samprotavimo užduotyse ir sumažina haliucinacijas (žr. 1.2 poskyrį). Khan ir kt. [Khv+24] tyrime, taikytame dirbtinio intelekto saugumo suderinimo (angl. AI alignment) kontekstui, parodyta, kad kai du stipresni ekspertų modeliai debatuoja, o silpnesnis modelis išrenka galutinį atsakymą, vertintojo tikslumas pakyla nuo 48% iki 76% - tai +28 procentinių punktų pagerėjimas, kuris yra ryškiausias konkretus skaitinis MAD efekto pavyzdys literatūroje. Tuo pačiu metu Smit ir kt. [Sdg+24] sisteminiame palyginime nustatė, kad MAD sistemos „nepatikimai pranoksta kitas siūlomas užklausų strategijas” be tinkamo hiperparametrų derinimo - tai svarbus kritinis kontekstas, kurį šio darbo konfigūracija (anti-regresijos politika, kritikos istorija, adaptyvi temperatūra; žr. 2.4 poskyrį) tikslingai sprendžia inžineriniu būdu. Detalesnis gretinimas su platesniu literatūros lauku (Wang ir kt. [Wws+22b] „Self-Consistency”, Shinn ir kt. [Scb+23] „Reflexion”, Hong ir kt. [Hzc+24] „MetaGPT”, Jiang ir kt. [Jdw+25] „Self-planning”) pateikiamas 4.8 poskyryje.

Šio darbo skaitinis rezultatas (+22,7 procentinio punkto pirminiame rinkinyje, +31,9 antrajame ir +27,3 sujungtai) yra tos pačios kategorijos kaip ir minėti literatūros rodikliai, tačiau tiesioginis sugretinimas yra apribotas trim aspektais. Pirma, šio darbo eksperimentuose dalyvauja atvirųjų svorių 7-9 milijardų parametrų modeliai, vykdomi vietinėje aparatūroje, o ne komerciniai didelio masto modeliai („GPT-4”, „Claude”) - todėl pradinis vieno modelio rodiklis yra žemesnis ir debatų metodo gerinimo potencialas objektyviai platesnis. Antra, šio darbo užduočių rinkinyje, ypač tasks2/ ekstremaliojo lygyje, sutelktos daugiafailės užduotys,

reikalaujančios dviejų ar trijų suderintų modulių su tarpmoduliniais importais (žr. 2.5 ir 3.3 poskyrius) - jos struktūriškai sudėtingesnės nei vienos funkcijos „HumanEval“ tipo užduotys, kurios dominuoja kituose šios srities tyrimuose. Trečia, šio darbo užduočių rinkinys nėra viešas standartinis etalonas, todėl absoliutus rodiklio dydis pats savaime nėra tiesiogiai palyginamas su „HumanEval“ ar „MATH“ bazinėmis linijomis. Dėl šių priežasčių lyginimas su literatūra yra ne kiekybinis, o tendencinis: jis parodo, kad debatų metodo pagerėjimas yra to paties dydžio kategorijos kaip ir literatūroje pranešti rezultatai, o šio darbo eksperimentinė konfigūracija - atvirųjų svorių mažo masto modeliai daugiafailėse kodo generavimo užduotyse - yra unikalūs scenarijus, kuris kituose darbuose menkai ištirtas empiriškai.

Pažymėtina, kad pats Smit ir kt. [Sdg+24] kritinis tyrimas nurodo dvi sąlygas, kuriose debatų privalumai pasireiškia ryškiausiai: „silpnųjų modelių ir ypač sudėtingų problemų“ scenarijus bei „heterogeniškų agentų įtraukimas, kuris yra universalus metodas pagerinti MAD efektyvumą“. Abi šios sąlygos kartu tenkinamos šio darbo konfigūracijoje: dalyvauja atvirųjų svorių 7-9 milijardų parametrų modeliai (silpni palyginti su uždarnosios prieigos didžiaisiais modeliais), tasks2/ rinkinyje sutelktos sudėtingiausios daugiafailės užduotys (ypač sudėtingos problemos), o pasiūlytojų rinkiniai sudaryti iš architektūriškai skirtingų modelių - „Qwen2.5-Coder“, „DeepSeek-Coder“, „Code Llama“ 7B klasėje ir „Granite Code“, „CodeGeeX4“, „Yi-Coder“ 9B klasėje (heterogeniški agentai). Šio poskyrio rezultatai būtent tai ir patvirtina empiriškai: +22,7-31,9 procentinio punkto pagerėjimas ir 0 regresijų iš 600 debatų vykdymų rodo, kad Smit ir kt. [Sdg+24] nurodytos sąlygos sudaro palankų kontekstą debatų metodo efektyvumui pasireikšti.

Šiame kontekste šio poskyrio rezultatai pateikia empirinį įrodymą, kad kelių agentų debatai išlaiko savo efektyvumą perkėlus juos į ribotos apimties atvirųjų svorių aplinką ir į daugiafailių kodo generavimo scenarijų. Galutinė šio palyginimo išvada - debatų sistema iš silpnųjų individualių modelių sukuria vieną, kolektyviai stipresnį už geriausią solo modelį, ypač sudėtingose užduotyse, ir nė karto neprarado pradinio geriausio sprendimo. Tolesni šio skyriaus poskyriai detalai nagrinėja, kurie konkretūs sistemos elementai - agentų rinkinio sudėtis (4.3), teisėjo modelio tipas (4.4), debatų raunduose vykstantys procesai (4.5) - prisideda prie šio bendro pranašumo, o skaičiavimo sąnaudų ir kokybės kompromisas aptariamas 4.7 poskyryje.

4.3. Agentų rinkinio sudėties įtaka: Pool A (7B) ir Pool B (9B) palyginimas

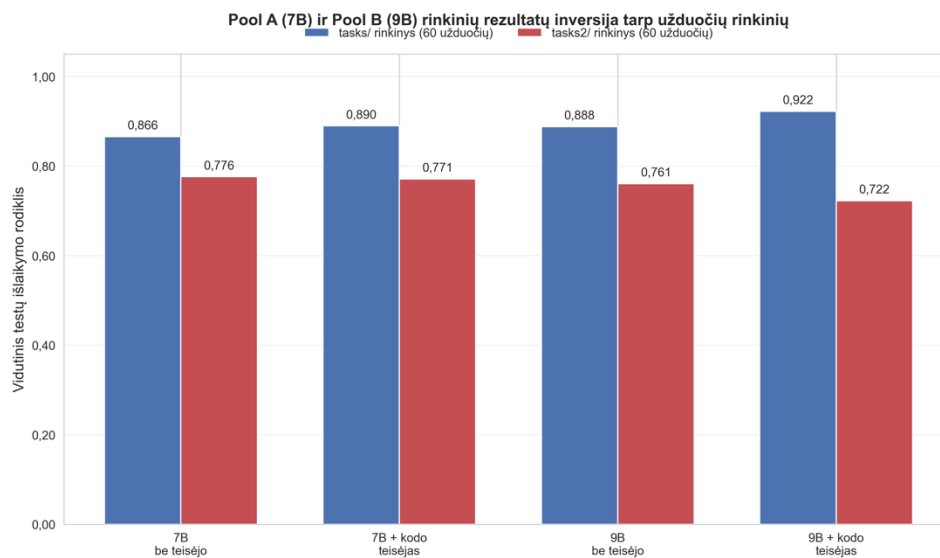
Šis poskyris atsako į antrąjį 3.1 poskyryje suformuluotą tikrinamą klausimą: ar didesnio parametrų skaičiaus pasiūlytojų rinkinys debatuose pasiekia geresnį rezultatą nei mažesnio? Hipotezė, paremta solo bazinės linijos rezultatais (4.1 poskyris), buvo tokia: Pool B, sudarytas iš 9B klasės modelių - „Granite Code“, „CodeGeeX4“ ir „Yi-Coder“ - turėtų pranokti Pool A, sudarytą iš 7B klasės modelių - „Qwen2.5-Coder“, „DeepSeek-Coder“ ir „Code Llama“ - debatuose, kadangi solo režimu Pool B modelių vidutinis testų išlaikymo rodiklis (0,700) yra aukštesnis už Pool A modelių vidurkį (0,628). Šio poskyrio rezultatai šios hipotezės nepatvirtina: empiriškai didesnio parametrų skaičiaus modelių rinkinio pranašumas debatų cikle neišlieka statistiškai reikšmingas, o tasks2/ rinkinyje viena Pool A konfigūracija agreguotai pranoksta visas Pool B konfigūracijas sudėtingiausiose užduotyse.

Bendras kiekvienos konfigūracijos testų išlaikymo rodiklis pagal užduočių rinkinį pateiktas 12 lentelėje. Sujungtame 120 užduoties-rinkinio porinių palyginimų rinkinyje Pool A vidutinis rodiklis siekia 0,821 be teisėjo ir 0,830 su kodo specializacijos teisėju, o Pool B - atitinkamai 0,824 ir 0,822. Skirtumai tarp agentų rinkinių yra labai maži: visais atvejais jie nesiekia vieno procentinio punkto. Todėl jau iš agreguotų rezultatų matyti, kad pradinis didesnio parametrų skaičiaus modelių rinkinio pranašumas debatų cikle iš esmės išnyksta. Šio rezultato negalima paaiškinti tuo, kad debatai pasiekia maksimumą - ypač ekstremaliojo lygio užduotyse rodikliai vis dar svyruoja maždaug 0,54-0,74 intervale, todėl gerinimo potencialo aiškiai yra.

Konfigūracija	tasks/ rinkinys (60 užd.)	tasks2/ rinkinys (60 užd.)	Sujungtas (120 porų)	Pokytis tarp rinkinių (proc. p.)
Pool A (7B) be teisėjo	0,866	0,776	0,821	-9,0
Pool A (7B) + kodo teisėjas	0,890	0,771	0,830	-11,9
Pool B (9B) be teisėjo	0,888	0,761	0,824	-12,7
Pool B (9B) + kodo teisėjas	0,922	0,722	0,822	-20,0

12 lentelė. Vidutinis galutinis testų išlaikymo rodiklis pagal pasiūlytojų rinkinį ir užduočių rinkinį (be samprotavimo teisėjo konfigūracijų, kurios analizuojamos 4.4 poskyryje). Paskutinis stulpelis - viso tasks2/ rinkinio rodiklio ir viso tasks/ rinkinio rodiklio skirtumas procentiniais punktais.

Detalesnis pasiskirstymas atskirai pagal užduočių rinkinius atskleidžia esminį šio poskyrio pastebėjimą - rezultatų inversiją tarp rinkinių (žr. 12 diagramą). Pirminiame tasks/ rinkinyje aukščiausią vidutinį rodiklį pasiekia Pool B su kodo specializacijos teisėju (0,922), o agentų rinkiniai išsiriškioja iš anksto numatyta tvarka: 9B + kodo teisėjas (0,922) > 7B + kodo teisėjas (0,890) > 9B be teisėjo (0,888) > 7B be teisėjo (0,866). Tačiau sudėtingesniame tasks2/ rinkinyje šis eiliškumas neišlieka: geriausią rodiklį agreguotai pasiekia Pool A be teisėjo (0,776), o žemiausią - Pool B su kodo specializacijos teisėju (0,722), kuris pirminiame rinkinyje buvo lyderis. Skirtumas tarp lyderio ir paskutinio tasks2/ rinkinyje siekia 5,4 procentinio punkto. Kritimo dydis (12 lentelės paskutinis stulpelis) tarp konfigūracijų taip pat reikšmingai skiriasi: Pool A be teisėjo praranda 9,0 procentinio punkto, o Pool B su kodo teisėju - net 20,0. Lyginant tik tasks/ rinkinio sudėtingo ir ekstremalaus lygio pogrupį (30 užduočių) su tasks2/ rinkiniu (kuris taip pat neturi lengvojo ir vidutinio lygio užduočių, todėl sudėtingumo struktūra mažiau iškreipia palyginimą), tendencija išlieka: Pool A be teisėjo netgi pakyla 4,5 procentinio punkto, o Pool B su kodo teisėju vis tiek nukrenta 12,7 procentinio punkto. Abi vertinimo perspektyvos veda prie tos pačios pagrindinės išvados - intuityviai stipriausia konfigūracija agreguotai pasirodo prasčiausiai būtent tose užduotyse, kurioms debatų metodas turi didžiausią praktinę vertę.



12 diagrama. Pool A (7B) ir Pool B (9B) rinkinių rezultatų inversija tarp užduočių rinkinių

Bendras vidutinis rodiklis nevysiškai atspindi užduočių lygio skirtumus, todėl papildomai taikytas porinis Stjudento t kriterijus, kuriame kiekviena užduotis sudaro vieną porinio palyginimo elementą (žr. 13 lentelę). Visuose keturiuose palyginimuose p reikšmė viršija 0,05 slenkstį - didžiausia p reikšmė gaunama tasks2/ rinkinyje be teisėjo ($p = 0,721$), o mažiausia - tasks2/ rinkinyje su kodo specializacijos teisėju ($p = 0,084$), tačiau ir pastarasis rezultatas

nesiekia įprastinio statistinio reikšmingumo slenksčio. Tai reiškia, kad statistiškai negalima atmesti, jog šie skirtumai kyla iš užduočių lygio variacijos. Šis rezultatas patvirtina, kad šio darbo duomenys nepalaiko hipotezės, jog 9B pasiūlytojų rinkinys debatuose statistiškai reikšmingai pranoksta 7B rinkinį.

Palyginimas	n	Pool A vid.	Pool B vid.	Δ B - A), proc. p.	A pergalės	Lygiosios	B pergalės	t	p
tasks/ be teisėjo	60	0,866	0,888	+2,3	7	43	10	+0,80	0,425
tasks2/ be teisėjo	60	0,776	0,761	-1,5	21	23	16	-0,36	0,721
tasks/ + kodo teisėjas	60	0,890	0,922	+3,2	6	43	11	+1,55	0,127
tasks2/ + kodo teisėjas	60	0,771	0,722	-4,9	22	23	15	-1,76	0,084

13 lentelė. Porinio Stjudento t kriterijaus rezultatai Pool A (7B) ir Pool B (9B) palyginimui (kiekviena užduotis - vienas porinio palyginimo elementas; t skaičiuojamas kryptimi Pool B - Pool A, todėl t ženklas atitinka Δ ženklą).

Pasiskirstymas pagal sudėtingumo lygį leidžia tiksliau lokalizuoti, kuriame lygyje agentų rinkiniai pradeda skirtis (žr. 14 lentelę). Lengvojo ir vidutinio sudėtingumo užduotyse visos keturios konfigūracijos pasiekia praktiškai 100% (vienintelis nukrypimas - Pool B su kodo teisėju vidutinio lygio užduotyse pasiekia 0,989), o sudėtingo lygio užduotyse rinkinių skirtumai išlieka maži (0,90-0,97 intervale). Tikrieji skirtumai išryškėja ekstremaliojo lygio užduotyse. Pirminiame rinkinyje ekstremaliojo lygio rodikliai išsirikiuoja taip: Pool B + kodo teisėjas (0,739), Pool A + kodo teisėjas (0,658), Pool B be teisėjo (0,641), Pool A be teisėjo (0,538) - šis eiliškumas atitinka intuityvią prielaidą, kad didesnis modelio dydis ir kodo specializacijos teisėjas suteikia papildomą naudą sudėtingiausiose užduotyse. Antrajame rinkinyje eiliškumas pasikeičia: Pool A + kodo teisėjas (0,683), Pool A be teisėjo (0,679), Pool B be teisėjo (0,658), Pool B + kodo teisėjas (0,624). Pool A konfigūracijos užima dvi pirmąsias vietas, o Pool B + kodo teisėjas - paskutinę. Šią tendenciją iliustruoja ir konkretūs atvejai - pavyzdžiui, ekstremalaus lygio užduotyje „Calculator with AST and Variables” Pool A be teisėjo pasiekė 1,000 testų išlaikymą, o Pool B su kodo teisėju - tik 0,154 (skirtumas 84,6 procentinio punkto); panašios dydžio inversijos matomos ir tokiose užduotyse kaip „JSON Parser” (1,000 prieš 0,333) bei „Simple ORM with In-Memory Storage” (1,000 prieš 0,364). Visiškai išspręstų užduočių dalis tasks2/ rinkinyje šią tendenciją apibendrina: Pool A be teisėjo visiškai išsprendžia 29 užduotis iš 60 (48,3%), Pool A su kodo teisėju ir Pool B be teisėjo - po 24 (40,0%), o Pool B su kodo teisėju - tik 19 (31,7%), todėl skirtumas tarp lyderio ir paskutinio siekia 16,6 procentinio punkto. Detalesnė kokybinė atskirų debatų transkriptų analizė pateikiama 4.6 poskyryje.

Konfigūracija	tasks/ Sudėtingas	tasks/ Ekstremalus	tasks2/ Sudėtingas	tasks2/ Ekstremalus
Pool A (7B) be teisėjo	0,925	0,538	0,970	0,679
Pool A (7B) + kodo teisėjas	0,901	0,658	0,946	0,683
Pool B (9B) be teisėjo	0,912	0,641	0,966	0,658

Pool B (9B) + kodo teisėjas	0,960	0,739	0,919	0,624
-----------------------------	-------	-------	-------	-------

14 lentelė. Vidutinis galutinis testų išlaikymo rodiklis pagal sudėtingumo lygį ir užduočių rinkinį (lengvasis ir vidutinis lygiai - tik tasks/ rinkinyje, kadangi tasks2/ rinkinyje šių lygių užduočių nėra).

Šio poskyrio rezultatai leidžia formuluoti dvi viena kitą papildančias interpretacijas, paaiškinančias, kodėl didesnis pasiūlytojų rinkinys debatuose neperteikia individualių modelių pranašumo. Pirmoji - homogeniškumo hipotezė. Pool B sudaro trys panašios kategorijos kodo modeliai, kurių solo režimo rodikliai išsidėsto siaurame 0,61-0,76 intervale (amplitudė 0,15, t. y. skirtumas tarp stipriausio ir silpniausio modelio - 14,8 procentinio punkto), todėl jų sprendimai dažnai konverguoja į panašias programines konstrukcijas, o kritikos fazėje atsiranda mažiau realių skirtumų, į kuriuos būtų galima atsižvelgti. Pool A sudėtis yra heterogeniška ne tik architektūriškai, bet ir gebėjimų atžvilgiu - „Qwen2.5-Coder“ solo pasiekia 0,73, „DeepSeek-Coder“ - 0,69, o „Code Llama“ instrukcijų vykdymo variantas - tik 0,46 (amplitudė 0,26, arba 26,2 procentinio punkto). Pool A solo rezultatų variacija (matuojama standartiniu nuokrypiu: 0,12 prieš 0,06) yra maždaug 1,8 karto didesnė nei Pool B. Toks platesnis gebėjimų išsisklaidymas sukuria daugiau įvairovės, į kurią kritikos fazė gali atsiremti, ypač sudėtingose užduotyse. Šis stebėjimas dera su bendresne Smit ir kt. [Sdg+24] išvada, kad MAD efektyvumas labai priklauso nuo konkretaus konfigūracijos pasirinkimo (žr. 1.2 poskyrį), bei papildoma Liang ir kt. [Lhj+23] DMAD darbo idėja apie agentų skirtumų svarbą debatuose - šio darbo atveju įvairumas pasireiškia ne strategijų, o pasiūlytojų pradinių gebėjimų lygmeniu.

Antroji interpretacija susijusi su kodo specializacijos teisėjo sąveika su rinkinio stiprumu. Pool B su kodo teisėju yra ta konfigūracija, kuri patiria didžiausią nuosmukį sudėtingiausiose užduotyse - sujungus stipresnių pasiūlytojų rinkinį su papildomu specializuotu kritiku, sistemos sprendimai gali per anksti susikoncentruoti ties vienu požiūriu, taip nukreipdami sprendimų paiešką į lokalų optimumą ir prarasdami alternatyvas, kurios būtų reikalingos ekstremaliojo lygio užduotims. Pool A su tuo pačiu teisėju išvengia šio efekto, kadangi pradinis pasiūlytojų sprendimų skirtumas yra didesnis, ir kodo teisėjo kritika veikia kaip papildomas filtras tarp jau įvairesnių alternatyvų. Pirminė šio efekto nuoroda matoma jau lyginant kodo ir samprotavimo teisėjų rezultatus tose pačiose 40 ekstremaliojo lygio tasks2/ užduočių - Pool B atveju samprotavimo teisėjas pasiekia 0,667, o kodo teisėjas - tik 0,624 (samprotavimo teisėjo naudai +4,3 procentinio punkto); Pool A atveju kodo teisėjas (0,683) ir samprotavimo teisėjas (0,674) duoda praktiškai identišką rezultatą. Šis kontrastas struktūriškai panašus į Khan ir kt. [Khv+24] „stiprių ekspertų + stipraus teisėjo“ konfigūraciją, kuri jų darbe veikė kaip pranašumo šaltinis, tačiau šio darbo daugiafailių užduočių kontekste pasirodo prasčiausiai. Detalesnis teisėjo režimo efekto vertinimas pateiktas 4.4 poskyryje.

Apibendrinant galima teigti, kad hipotezė, jog didesnio parametrų skaičiaus pasiūlytojų rinkinys užtikrina geresnį debatų rezultatą, šio darbo duomenyse nepasitvirtino. Sujungti Pool A ir Pool B rodikliai praktiškai sutampa (0,821 prieš 0,824 be teisėjo; 0,830 prieš 0,822 su kodo specializacijos teisėju), porinis Studento t kriterijus nė viename iš keturių palyginimų neaptiko statistiškai reikšmingo skirtumo, o sudėtingiausiose tasks2/ rinkinio užduotyse Pool A konfigūracijos agreguotai pranoko Pool B. Centrinė šio poskyrio empirinė išvada formuluojama taip: šio darbo eksperimentuose pasiūlytojų sprendimų įvairumas pasirodo ne mažiau svarbus, o kai kuriais atvejais net svarbesnis už atskirų modelių solo stiprumą - kodo specializacijos teisėjas, dirbantis su stipresnių pasiūlytojų rinkiniu, gali per anksti nukreipti sprendimų paiešką į lokalų optimumą ir taip apsunkinti alternatyvų tyrimą, būtiną ekstremaliojo lygio užduotims. Praktiniu požiūriu tai reiškia, kad atvirųjų svorių 7-9 milijardų parametrų modelių srityje vien modelio dydis nėra pakankamas kriterijus optimaliam debatų agentų rinkiniui sudaryti - rinkinio įvairumas yra mažiausiai tiek pat svarbus veiksnys. Šio poskyrio pagrindinį principą galima formuluoti glaustai: 7-9 milijardų parametrų atvirųjų svorių modelių srityje debatų komandos formavimo strategija „rinktis stipriausius“ yra suboptimali - geresnis kriterijus yra

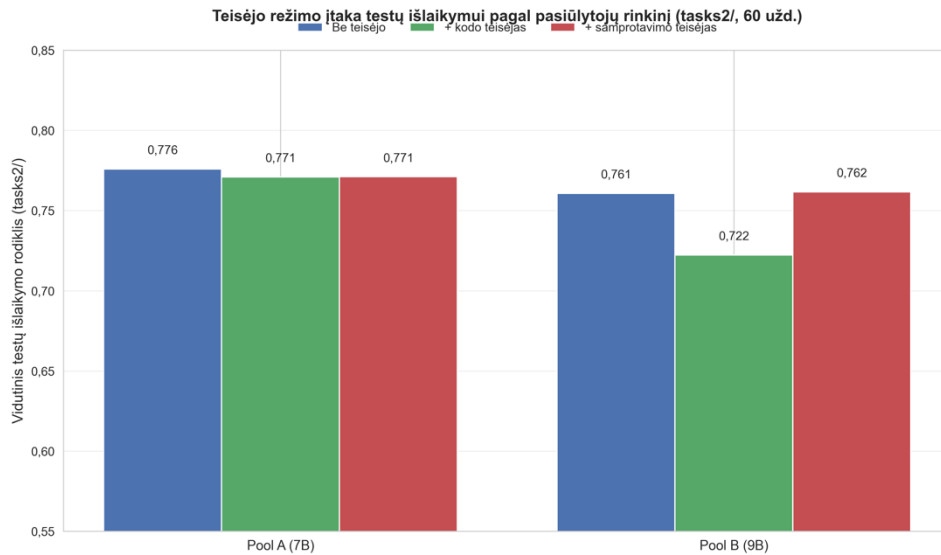
„rinktis įvairiausias”. Tolesniame 4.4 poskyryje analizuojama trečioji palyginimo ašis - teisėjo režimas (be teisėjo, su kodo specializacijos teisėju ir su samprotavimo specializacijos teisėju).

4.4. Teisėjo režimo įtaka: be teisėjo, kodo specializacijos ir samprotavimo specializacijos teisėjai

Šis poskyris atsako į trečiąjį 3.1 poskyryje suformuluotą tiriamąjį klausimą: ar teisėjo agento įtraukimas į debatų ciklą pagerina rezultatą, ir jeigu taip, ar teisėjo specializacijos tipas (kodo ar samprotavimo) turi reikšmingos įtakos. Lyginami trys režimai: tas pats pasiūlytojų rinkinys be teisėjo, su kodo specializacijos teisėju „DeepSeek-Coder-V2” (16B mišrių ekspertų modelis, žr. 3.2 poskyrį) ir su samprotavimo specializacijos teisėju „DeepSeek-R1” (14B distiliuotas modelis). Šis poskyris naudoja išskirtinai tasks2/ užduočių rinkinio duomenis dėl dviejų priežasčių. Pirmą, samprotavimo teisėjas buvo vykdytas tik šiame rinkinyje (žr. 3.1 poskyrio 2 lentelę), todėl visiems trimis režimams kartu prieinama užduočių aibė yra apribota tasks2/ rinkiniu - tik šis pasirinkimas leidžia palyginti tris teisėjo režimus tose pačiose užduotyse (apples-to-apples palyginimas, po $n = 60$ vykdymų kiekvienai konfigūracijai). Antra, tasks2/ rinkinys sąmoningai sudarytas iš 20 sudėtingo ir 40 ekstremalaus lygio užduočių (žr. 3.3 poskyrį) - būtent tokiose sudėtingose užduotyse papildomos kritikos nauda iš teisėjo turėtų pasireikšti ryškiausiai, todėl tai yra metodologiškai stipriausia aplinka teisėjo režimo įtakai vertinti. Bendresnio pobūdžio teisėjo poveikis tasks/ rinkinyje (kuriame yra ir lengvojo bei vidutinio lygio užduotys) jau buvo aptartas 4.2 poskyryje. Konceptualiai šis poskyris realizuoja „LLM-as-Judge” paradigmos (Zheng ir kt. [Zcs+23]; žr. 1.3 poskyrį) empirinę patikrą šio darbo daugiafailio kodo generavimo užduočių kontekste.

Konfigūracija	Bendras (60 užd.)	Sudėtingas (20 užd.)	Ekstremalus(40 užd.)
Pool A be teisėjo	0,776	0,970	0,679
Pool A + kodo teisėjas	0,771	0,947	0,683
Pool A + samprotavimo teisėjas	0,771	0,966	0,674
Pool B be teisėjo	0,761	0,966	0,658
Pool B + kodo teisėjas	0,722	0,919	0,624
Pool B+ samprotavimo teisėjas	0,762	0,950	0,667

15 lentelė. Vidutinis galutinis testų išlaikymo rodiklis pagal teisėjo režimą tasks2/ rinkinyje (po 60 užduočių kiekvienai konfigūracijai, iš jų 20 sudėtingo ir 40 ekstremalaus lygio)



13 diagrama. Teisėjo režimo įtaka testų išlaikymui pagal pasiūlytojų rinkinį (tasks2/, 60 užd.)

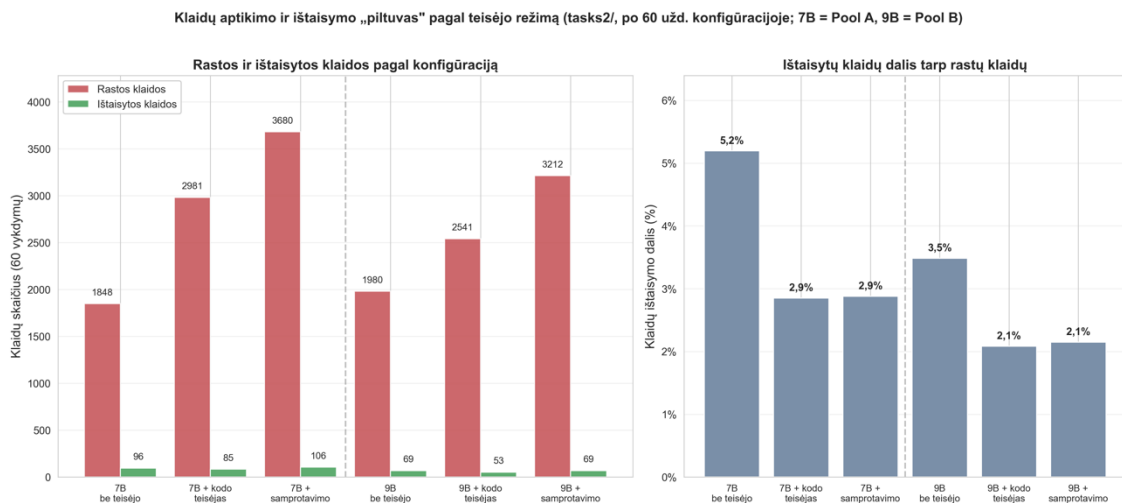
Pagrindiniai rezultatai pateikti 15 lentelėje ir 13 diagramoje. Svarbiausias matomas dėsningumas yra tas, kad teisėjo įtaka labai priklauso nuo pasiūlytojų rinkinio. Pool A (7B klasė) atveju visi trys režimai duoda praktiškai vienodus rezultatus: be teisėjo - 0,776, su kodo teisėju - 0,771, su samprotavimo teisėju - 0,771; skirtumai tarp jų neviršija 0,5 procentinio punkto. Pool B (9B klasė) atveju vaizdas kitoks - be teisėjo režimas (0,761) ir samprotavimo teisėjas (0,762) duoda praktiškai identišką rezultatą, o kodo specializacijos teisėjas rezultatą sumažina iki 0,722 (skirtumas siekia –3,9 procentinio punkto, palyginti su variantu be teisėjo). Ekstremaliojo lygio užduotyse efektas dar ryškesnis: Pool B su kodo teisėju nukrenta iki 0,624, t. y. 3,4 procentinio punkto žemiau be teisėjo režimo (0,658), o samprotavimo teisėjas (0,667) išlaiko stabilumą. Visiškai išspręstų užduočių dalies (*final_pass_rate* = 1,0) tendencija atitinka vidutinio rodiklio analizę, tačiau išryškina ją kontrastingiau: Pool A be teisėjo - 48,3%, Pool A + samprotavimo - 45,0%, Pool A + kodo - 40,0%, Pool B be teisėjo - 40,0%, Pool B + samprotavimo - 36,7%, Pool B + kodo - 31,7%, t. y. skirtumas tarp geriausio ir prasčiausio rezultato siekia 16,6 procentinio punkto.

Šis rezultatas atitinka 4.3 poskyryje aptartą interpretaciją, kad kodo specializacijos teisėjo poveikis priklauso nuo pasiūlytojų rinkinio savybių. Šio poskyrio duomenyse kodo teisėjas Pool A atveju nepagerina rezultato, tačiau jo poveikis išlieka beveik neutralus, o Pool B atveju jis aiškiai pablogina rezultatą. Tai leidžia manyti, kad kai pasiūlytojai yra panašaus stiprumo, specializuoto kritiko įtraukimas gali skatinti per ankstyvą sprendimų konsolidaciją ties lokaliu optimumu.

Norint suprasti, kodėl teisėjo įtraukimas tampa neutralus arba net neigiamas, naudinga analizuoti klaidų aptikimo ir ištaisymo „piltuvą“ (žr. 16 lentelę ir 14 diagramą). Kiekvienam debatui sistema fiksuoja, kiek klaidų agentai aptiko savo arba kitų sprendimuose kritikos fazėje (*total_bugs_found* stulpelyje) ir kiek iš jų buvo realiai ištaisyta revizijos fazėje (*total_bugs_fixed* stulpelyje; žr. 2.6.3 poskyrį). Teisėjo įtraukimas smarkiai padidina rastų klaidų skaičių, tačiau ištaisytų klaidų skaičius beveik nepasikeičia - dėl to ištaisymo dalis sumažėja. Pool A be teisėjo per 60 vykdymų agentai rado 1 848 klaidas ir ištaisė 96 (ištaisymo dalis 5,2%); su kodo teisėju rastų klaidų skaičius pakyla iki 2 981 (+61%), tačiau ištaisytų - sumažėja iki 85 (-11), o ištaisymo dalis nukrenta iki 2,9%. Su samprotavimo teisėju klaidų randama dar daugiau (3 680, t. y. +99% palyginti su variantu be teisėjo), ištaisoma 106, o ištaisymo dalis išlieka 2,9%. Pool B atveju tendencija ta pati - rastų klaidų skaičius su kodo teisėju išauga iki 2 541 (palyginti su 1 980 klaidų be teisėjo režimu), o ištaisytų sumažėja iki 53 (palyginti su 69); ištaisymo dalis krenta iki 2,1%.

Konfigūracija	Rastos klaidos	Ištaisytos klaidos	Ištaisymo dalis (%)
Pool A be teisėjo	1 848	96	5,2
Pool A + kodo teisėjas	2 981	85	2,9
Pool A + samprotavimo teisėjas	3 680	106	2,9
Pool B be teisėjo	1 980	69	3,5
Pool B + kodo teisėjas	2 541	53	2,1
Pool B + samprotavimo teisėjas	3 212	69	2,1

16 lentelė. Klaidų aptikimo ir ištaisymo „piltuvus“ pagal teisėjo režimą (sumos per 60 vykdymų kiekvienoje konfigūracijoje, tasks2/).



14 diagrama. Klaidų aptikimo ir ištaisymo „piltuvus“ pagal teisėjo režimą (tasks2/, po 60 užd. kiekvienoje konfigūracijoje; diagramoje 7B atitinka Pool A, 9B - Pool B)

Šie duomenys empiriškai kvestionuoja platesnę prielaidą, kad „kuo daugiau kritikos, tuo geriau“ - šio darbo duomenyse priešingai, papildomas kritikos signalas iš teisėjo padidina kritikos triukšmą (rastų klaidų skaičių), tačiau iš jo gaunamas realių pataisymų signalas išlieka stabilus arba net silpnėja. Tai paaiškina, kodėl agreguotas testų išlaikymo rodiklis su teisėju nepagerėja - daugumos papildomai rastų „klaidų“ agentai nepaverčia konkrečiais kodo pataisymais, nes grįžtamojo ryšio signalas nėra pakankamai aiškiai pritaikomas revizijos fazėje. Šis empirinis rezultatas nepatvirtina supaprastintos „LLM-as-Judge“ paradigmos prielaidos (žr. 1.3 poskyrį), kad bet kuris papildomas vertintojas automatiškai pagerina sistemos rezultatą - vertintojo reali nauda priklauso ne tik nuo jo gebėjimo aptikti klaidas, bet ir nuo to, ar jo kritikos signalas yra suprantamas pasiūlytojų agentams ir paverčiamas konkrečiu kodo pataisymu.

Atskirai vertinant samprotavimo specializacijos teisėjo elgseną, atsiranda subtilesnė tendencija. Pool B + samprotavimo teisėjas grįžta į režimo be teisėjo lygį (0,762 prieš 0,761) ir net ekstremaliojo lygio užduotyse pasiekia šiek tiek aukštesnį rodiklį nei kodo teisėjas (0,667 prieš 0,624, skirtumas +4,3 procentinio punkto samprotavimo teisėjo naudai). Pritaikius porinį Stjudento t kriterijų 40 ekstremaliojo lygio užduočių poroms, gauta $t = +0,87$, $p = 0,390$ - efekto kryptis matoma (samprotavimo teisėjas 17 atvejų pranoksta kodo teisėją, 13 atvejų rezultatai sutampa, 10 atvejų kodo teisėjas geresnis), tačiau jis nesiekia statistinio reikšmingumo slenksčio. Pool A atveju skirtumas tarp samprotavimo ir kodo teisėjo nereikšmingas net efekto krypties prasme ($\Delta = -0,9$ procentinio punkto, $p = 0,829$). Šio efekto mechanizmas atskleidžiamas lyginant pirmojo raundo (R1) vidutinį rezultatą su galutiniu: Pool B + kodo teisėjas atveju ekstremaliojo lygio užduotyse R1 vidurkis yra 0,342, o galutinis - 0,624 (debatų ciklo prieaugis +28,2 procentinio punkto); Pool B + samprotavimo teisėjas atveju R1 vidurkis šiek tiek žemesnis (0,334), tačiau galutinis - 0,667, todėl debatų ciklo prieaugis siekia +33,3

procentinio punkto. Šis +5,1 procentinio punkto didesnis debatų ciklo prieaugis paaiškina galutinį +4,3 procentinio punkto skirtumą samprotavimo teisėjo naudai. Papildomas peak_after_debate rodiklis (vykdymų dalis, kuriose galutinis rezultatas viršija geriausią pirmojo raundo pasiūlymą; žr. 3.5 poskyrį) tasks2/ ekstremaliojo lygio užduotyse Pool A + kodo teisėjas atveju yra 42,5%, Pool A + samprotavimo teisėjas - 37,5%, Pool B + kodo teisėjas - 32,5%, o Pool B + samprotavimo teisėjas - 27,5%. Iš jo matyti, kad samprotavimo teisėjas pakelia rezultatą virš pirmojo raundo rečiau, tačiau kai pakelia, vidutinis prieaugis yra didesnis. Ši tendencija struktūriškai primena Khan ir kt. [Khv+24] tyrime aprašytą scenarijų, kuriame samprotavimo specializacijos modelis veikia kaip teisėjas, vertinantis stipresnių „ekspertų“ debatą - tačiau, skirtingai nei jų DI saugumo suderinimo užduotyse, šio darbo daugiafailio kodo užduočių aplinkoje samprotavimo teisėjo pranašumas prieš kodo specializacijos teisėją negali būti laikomas statistiškai patvirtintu ($p = 0,390$ ekstremaliojo lygio užduotyse).

Skaičiavimo sąnaudų požiūriu (žr. 17 lentelę) trys režimai skiriasi pastebimai. Be teisėjo režimas vidutiniškai trunka 170-203 sekundes per debatą, kodo teisėjas - 197-232 sekundes (apie 27-29 sek. daugiau), o samprotavimo teisėjas - 269-315 sekundžių, t. y. apie 99-112 sek. daugiau lyginant su be teisėjo režimu (priklausomai nuo pasiūlytojų rinkinio). Šis gerokai didesnis sąnaudų skirtumas tarp samprotavimo ir kodo teisėjo paaiškinamas tuo, kad „DeepSeek-R1“ generuoja vidinį samprotavimo grandinės tekstą (angl. chain-of-thought) prieš galutinį atsakymą, kurio žetonų skaičius dažnai daugiau kaip dvigubai viršija paties kritinio sprendimo apimtį. Atskirai pažymėtina, kad teisėjo įtraukimas Pool B atveju gerokai padidina konsensuso pasiekimo dažnį (60,0% be teisėjo prieš 80,0% su bet kuriuo teisėju), tačiau šis aukštesnis konsensusas neperauga į aukštesnį testų išlaikymą - tai dera su klaidų aptikimo „piltuvo“ išvada: agentai greičiau susitaria, tačiau pats susitarimas formuojamas apie tą patį sprendimų klasterį, kurio papildoma kritika negali iš esmės pakeisti. Detalesnis skaičiavimo sąnaudų ir kokybės kompromiso vertinimas pateikiamas 4.7 poskyryje.

Konfigūracija	Vidutinis raundų skaičius	Pasiekto konsensuso dalis (%)	Vidutinė trukmė (s)	Peak_after_debate (ekstr. lygis, %)
Pool A be teisėjo	2,63	68,3	203,1	27,5
Pool A + kodo teisėjas	2,78	66,7	232,4	42,5
Pool A + samprotavimo teisėjas	2,72	71,7	314,9	37,5
Pool B be teisėjo	2,77	60,0	169,9	30,0
Pool B + kodo teisėjas	2,62	80,0	197,2	32,5
Pool B + samprotavimo teisėjas	2,50	80,0	268,8	27,5

17 lentelė. Papildomi debatų eigos rodikliai pagal teisėjo režimą (tasks2/ rinkinyje, po 60 vykdymų)

Apibendrinant, hipotezė, kad teisėjo agento įtraukimas savaime pagerina debatų rezultatą, šio darbo duomenyse nepasitvirtino. Pool A atveju visi trys režimai veikia ekvivalentiškai (skirtumai neviršija 0,5 procentinio punkto), o Pool B atveju teisėjo režimo pasirinkimas tampa esminiu projektiniu sprendimu: be teisėjo varianto rezultatas (0,761) ir samprotavimo teisėjo variantas (0,762) yra praktiškai lygiaverčiai, tuo tarpu kodo specializacijos teisėjas rezultatą pablogina iki 0,722 (−3,9 procentinio punkto neigiamas efektas, +3,9 procentinio punkto pranašumas tiek be teisėjo, tiek samprotavimo teisėjo variantams). Klaidų aptikimo ir ištaisymo

„piltuvo” analizė atskleidžia šio rezultato mechanizmą: teisėjo įtraukimas padidina kritikos triukšmą (rastų klaidų skaičius išauga 30-100%), bet beveik nepakelia ištaisymo signalo (absoliutus ištaisytų klaidų skaičius išlieka panašus arba net mažėja). Šio poskyrio pagrindinį principą galima formuluoti glaustai: šio darbo eksperimentinėje aplinkoje papildomas teisėjo kritikos signalas smarkiai didina kritikos kiekį, bet ne kritikos efektyvumą - „daugiau kritikos” nereiškia „daugiau pataisymų”. Praktinis pasirinkimas tarp trijų režimų priklauso ne nuo galutinio testų išlaikymo (kuris Pool A atveju praktiškai sutampa, o Pool B atveju rodo, kad be teisėjo ir samprotavimo teisėjo variantai yra beveik lygiaverčiai, tačiau kodo teisėjo variantas aiškiai prastesnis), o nuo papildomų kriterijų: konsensuso pasiekimo dažnio (kuris su teisėju aukštesnis, ypač 9B atveju - 80,0% prieš 60,0%) ir skaičiavimo sąnaudų (kurios su samprotavimo teisėju apie 55-58% didesnės nei be teisėjo režimo). Tolesniame 4.5 poskyryje analizuojama, kokios konkrečios debatų raundų dinamikos sudaro šių rezultatų pagrindą.

4.5. Debatų raidos analizė

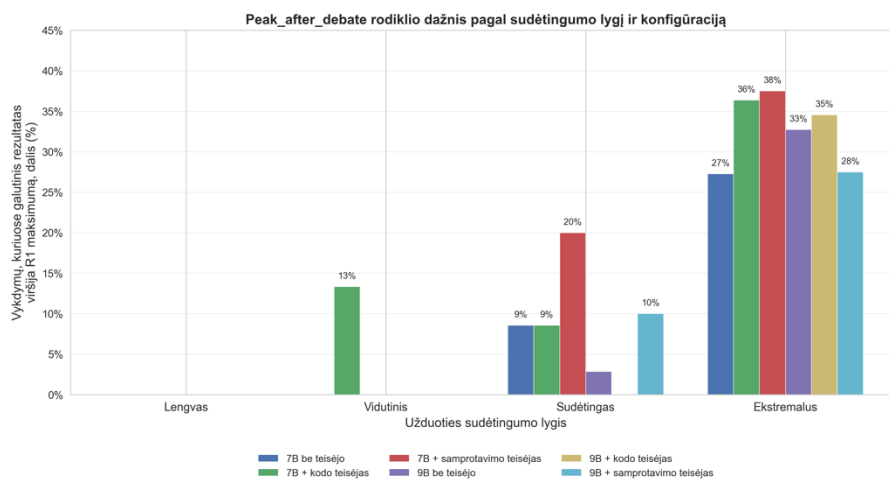
Šis poskyris atsako į klausimą, kaip debatų rezultatas formuojasi debatų eigoje: kuriais sudėtingumo lygiais debatų ciklas iš tiesų prideda vertę po pirmojo raundo, kiek raundų debatai vidutiniškai trunka ir koks vidinis mechanizmas labiausiai siejasi su rezultato pagerėjimu. Analizėje naudojami visi 600 debatų vykdymų iš šešių konfigūracijų, atrenkant tris skirtingus dinamikos rodiklius: *peak_after_debate* dažnį pagal sudėtingumo lygį (vykdymų, kuriuose galutinis rezultatas viršija geriausią pirmojo raundo pasiūlymą, dalis), raundų skaičiaus pasiskirstymą ir Pirsono koreliacijas tarp debatų eigos rodiklių ir santykinio rezultato pagerėjimo (*improvement_over_best_initial*, žr. 3.5 poskyrį). Pažymėtina, kad šis poskyris analizuoja debatų ciklo vidinę dinamiką (kaip rezultatas formuojasi laikui bėgant), o ne bendrą metodo efektyvumą - pastarasis jau įvertintas 4.2 poskyryje, kuriame debatai pranoksta solo režimą 27,3 procentinio punkto.

Pirmasis rodiklis - *peak_after_debate* dažnis pagal sudėtingumo lygį - rodo bendrą augimo tendenciją didėjant užduoties sudėtingumui, nors atskirose konfigūracijose ši tendencija nėra visiškai monotoniška (žr. 18 lentelę ir 15 diagramą). Lengvojo ir vidutinio sudėtingumo užduotyse *peak_after_debate* dažnis yra praktiškai nulinis - tik Pool A su kodo teisėju vidutinio lygio užduotyse pasiekia 13,3%, o visi kiti penki režimai - 0,0% (ši pati Pool A + kodo konfigūracija sudėtingajame lygyje atvirkščiai krenta į 8,6%, todėl jos elgsena tarp lengvojo ir sudėtingo lygio nėra griežtai monotonišė). Sudėtingajame lygyje dažnis svyruoja 0-20% intervale (Pool A + samprotavimo teisėjas pasiekia aukščiausią 20,0%, o Pool B + kodo teisėjas - 0,0%), o ekstremaliojo lygio užduotyse jis šokteli iki 27-38% intervalo (žemiausias Pool B + samprotavimo - 27,5%, aukščiausias Pool A + samprotavimo - 37,5%). Šis dėsningumas yra centrinis empirinis šio poskyrio rezultatas: debatų ciklas po pirmojo raundo realiai prideda vertę pirmiausia tose užduotyse, kuriose pirmasis raundas dažniau nepateikia teisingo sprendimo. Lengvojo ir vidutinio lygio užduotyse pirmojo raundo pasiūlymai jau dažniausiai būna teisingi, todėl tolesni raundai negali jų pagerinti.

Konfigūracija	Lengvas	Vidutinis	Sudėtingas	Ekstremalus	Bendras
Pool A be teisėjo	0,0% (15)	0,0% (15)	8,6% (35)	27,3% (55)	15,0% (120)
Pool A + kodo teisėjas	0,0% (15)	13,3% (15)	8,6% (35)	36,4% (55)	20,8% (120)
Pool A + samprotavimo teisėjas	-	-	20,0% (20)	37,5% (40)	31,7% (60)
Pool B be teisėjo	0,0% (15)	0,0% (15)	2,9% (35)	32,7% (55)	15,8% (120)
Pool B + kodo teisėjas	0,0% (15)	0,0% (15)	0,0% (35)	34,5% (55)	15,8% (120)

Pool B + samprotavimo teisėjas	-	-	10,0% (20)	27,5% (40)	21,7% (60)
--------------------------------	---	---	------------	------------	------------

18 lentelė. *Peak_after_debate* rodiklio dažnis (procentais) pagal teisėjo režimą ir užduoties sudėtingumo lygį. Skliausteliuose - vykdymų skaičius kiekvienoje kategorijoje. Samprotavimo teisėjo konfigūracijose nėra lengvojo ir vidutinio lygio užduočių, nes šis teisėjas buvo vykdytas tik tasks2/ rinkinyje (žr. 3.1 poskyrio 2 lentelę); dėl šios priežasties jų bendras stulpelis (60 vykdymų) nėra tiesiogiai palyginamas su kitais režimais (120 vykdymų), todėl pagrindinės šio poskyrio išvados daromos atskirai pagal sudėtingumo lygius. Be to, ekstremaliojo lygio dažnio reikšmės šioje lentelėje gali skirtis nuo 4.4 poskyryje pateiktų, nes 4.4 poskyrio analizė apsiribojo tik tasks2/ rinkiniu, o šio poskyrio analizė apima ir tasks/ rinkinio ekstremaliojo lygio užduotis.



15 diagrama. *Peak_after_debate* rodiklio dažnis pagal sudėtingumo lygį ir konfigūraciją (tuščios vietos lengvojo ir vidutinio lygio grupėse reiškia 0% reikšmes arba duomenų nebuvimą samprotavimo teisėjo konfigūracijose)

Antrasis dinamikos rodiklis - faktinio debatų raundų skaičiaus pasiskirstymas - papildomai patvirtina šią tendenciją. Iš 600 debatų vykdymų 47,3% (284) pasibaigė po pirmojo raundo, 23,2% (139) - po antrojo, 4,5% (27) - po trečiojo, 2,8% (17) - po ketvirtojo, o 22,2% (133) pasiekė maksimalų penkių raundų limitą. Vidutinis raundų skaičius yra 2,29, mediana - 2,0, tačiau vidutinis raundų skaičius labai priklauso nuo užduoties sudėtingumo: lengvojo lygio užduotyse vidutiniškai naudojamas 1,00 raundas (visada anksti pasibaigia po R1), vidutinio - 1,07 raundo, sudėtingo - 1,56 raundo, o ekstremaliojo - 3,24 raundo (mediana 3,0). Tai reiškia, kad ekstremaliojo lygio užduotims vidutiniškai prireikia maždaug tris kartus daugiau raundų nei vidutinio lygio užduotims, ir būtent šiame sudėtingumo lygyje *peak_after_debate* dažnis pasiekia 27-38% (priešingai, lengvojo lygio užduotys, kurios visada baigiasi po pirmojo raundo, negali turėti teigiamos *peak after debate* reikšmės). Iš 600 vykdymų konsensusas pasiektas 77,3% atvejų - tai leidžia manyti, kad anksti pasibaigiantys debatai (1-2 raundai) dažniausiai tai daro pasiekę pasiūlytojų sutarimą, o ne todėl, kad būtų nutraukti pasiekus maksimalią raundų ribą.

Trečiasis dinamikos rodiklis - Pirsono koreliacija tarp debatų eigos rodiklių ir santykinio rezultato pagerėjimo (*improvement_over_best_initial*) - leidžia nustatyti rodiklį, labiausiai susijusį su debatų rezultato pagerėjimu (žr. 19 lentelę). Stipriausią koreliaciją turi *total_bugs_fixed* (Pirsono $r = +0,446$, $n = 600$, $p < 10^{-30}$) - tai vidutinio stiprumo, statistiškai labai reikšminga teigiama koreliacija, rodanti, kad realiai ištaisytų klaidų skaičius yra esminis rodiklis, susijęs su rezultato pagerėjimu po debatų. Išskaidžius rezultatus pagal pasiūlytojų rinkinius, koreliacija dar stipresnė Pool A atveju ($r = +0,560$, $n = 300$, $p < 10^{-25}$), o Pool B atveju išlieka panaši ($r = +0,426$, $n = 300$, $p < 10^{-14}$). Kitos koreliacijos yra silpnesnės: kritikos įrašų skaičius ($r = +0,175$), raundų skaičius ($r = +0,169$) bei raundų iki konsensuso ($r = +0,252$) - visos teigiamos ir statistiškai reikšmingos ($p < 0,001$), tačiau jų koreliacijos yra gerokai silpnesnės nei ištaisytų klaidų skaičiaus koreliacija. Šis rezultatas atitinka 4.4 poskyryje pateiktą klaidų aptikimo „piltuvo“ išvadą, tačiau papildo ją kitu aspektu: 4.4 poskyryje buvo parodyta,

kad teisėjo įtraukimas didina rastų klaidų skaičių, bet ne ištaisytų, o šis poskyris atskleidžia, kad būtent ištaisytų klaidų (o ne rastų ar kritikų skaičiaus) kintamasis stipriausiai prognozuoja, kiek debatas pagerins pradinį pasiūlytojų rezultatą. Pažymėtina, kad Pirsono koreliacija nustato statistinį ryšį, bet ne tiesioginį priežastinį dėšningumą.

Debatų eigos rodiklis	Pirsono r	p reikšmė	Stiprumo interpretacija
Ištaisytų klaidų skaičius (total bugs fixed)	+0,446	$< 10^{-30}$	vidutinė teigiama
Raundų iki konsensuso (rounds to consensus)	+0,252	$< 10^{-9}$	silpna teigiama
Kritikos įrašų skaičius (total critiques)	+0,175	$< 10^{-4}$	silpna teigiama
Debatų raundų skaičius (total rounds)	+0,169	$< 10^{-4}$	silpna teigiama

19 lentelė. Pirsono koreliacijos koeficientai tarp debatų eigos rodiklių ir santykinio rezultato pagerėjimo (improvement_over_best_initial) per visus 600 debatų vykdymų. Visos koreliacijos statistiškai reikšmingos ($p < 0,001$).

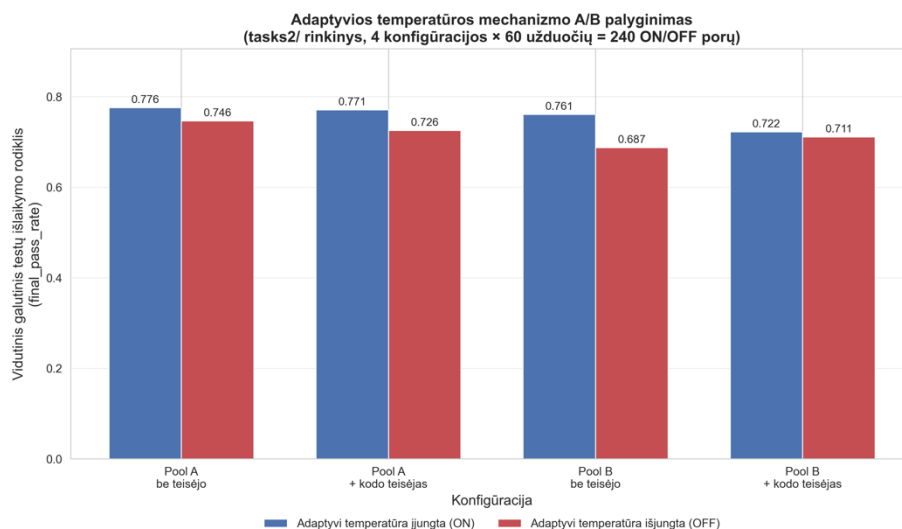
Iki šios vietos pristatyti dinamikos rodikliai (peak_after_debate dažnis, raundų skaičiaus pasiskirstymas, Pirsono koreliacijos) yra grindžiami stebėjimo duomenimis - šeši pagrindiniai HPC eksperimentai buvo vykdyti vienoda sistemos konfigūracija (žr. 3.4 poskyrį), kurioje 2.4.3 poskyryje aprašyta adaptyvi temperatūra buvo įjungta. Toks pasirinkimas kelia atvirą klausimą: ar šis mechanizmas iš tikrųjų prisideda prie debatų rezultato pagerėjimo, ar jo įtaka yra praktiškai nereikšminga? Siekiant atsakyti į šį klausimą, papildomai atliktas kontroliuotas A/B palyginimo eksperimentas, kuriame ta pati debatų sistema buvo vykdyta dviem režimais - su įjungta ir su išjungta adaptyvia temperatūra. Tokiu būdu vertinta, ar stagnacijos metu didinamas generavimo nepastovumas realiai prisideda prie galutinio testų išlaikymo rodiklio.

A/B palyginime keturios pagrindinės debatų konfigūracijos (Pool A be teisėjo, Pool A + kodo teisėjas, Pool B be teisėjo ir Pool B + kodo teisėjas) pakartotinai vykdytos toje pačioje HPC aplinkoje, naudojant tasks2/ rinkinį (60 sudėtingo ir ekstremaliojo lygio užduočių), tik išjungus adaptyvios temperatūros mechanizmą. Visi kiti hiperparametrai - revizijos pagrindinė temperatūra (0,4), kritikos istorija, anti-regresijos politika, ne daugiau kaip penkių raundų riba, geriausio kito agento sprendimo rodymo režimas, vienoda revizijos užklausų strategija - liko nepakitę. Kiekvienai užduočiai ir kiekvienai konfigūracijai gauta reikšmių pora: vienas vykdymas su įjungta adaptyvia temperatūra (toliau - ON) iš pagrindinio etapo ir vienas vykdymas su išjungta (toliau - OFF) iš A/B eksperimento. Bendra palyginimo apimtis - keturios konfigūracijos $\times$ 60 užduočių = 240 porinių palyginimų. Kadangi ON vykdymai jau priklauso pagrindinio etapo 960 įrašų aibe, papildomą duomenų apimtį šiame darbe sudaro 240 OFF vykdymų, dėl to bendra unikalių vykdymo įrašų apimtis padidėja nuo 960 iki 1200.

Pagrindiniai rezultatai pateikti 20 lentelėje ir 16 diagramoje. Sujungtoje 240 porų aibėje vidutinis testų išlaikymo rodiklis su įjungta adaptyvia temperatūra siekia 0,758, o su išjungta - 0,718; skirtumas sudaro 3,99 procentinio punkto ON režimo naudai. Porinis Stjudento t kriterijus parodė statistiškai reikšmingą skirtumą ($t = +2,60$, $p = 0,010$), o neparametrinis porinis Vilkoksono ženklinių rangų kriterijus patvirtino tą pačią išvadą ($W = 2864$, $p = 0,012$). Iš 240 porinių palyginimų ON pranoko OFF 76 atvejais, 116 atvejų rezultatai sutapo, o 48 atvejais OFF pasirodė geriau - taigi ON laimi maždaug 1,58 karto dažniau nei OFF. Efekto kryptis yra nuosekli visose keturiuose konfigūracijose - kiekvienoje iš jų ON vidurkis viršija OFF, nors atskirai paimtos konfigūracijos statistinio reikšmingumo lygis (taikant Bonferroni pataisą) išlieka tik kaip tendencijos signalas (Pool B be teisėjo - $p = 0,026$, neperžengia Bonferroni slenksčio 0,0125; kitose trijose konfigūracijose $p > 0,05$). Sujungtas rezultatas yra įtikinamesnis nei bet kuri atskira konfigūracija, nes remiasi keturis kartus didesne porinių palyginimų aibe.

Konfigūracija	n	ON	OFF	Δ (proc. p.)	Pergalės	Lygiosios	Pralaimėjimai	t	p	Bonf. p
Pool A be teisėjo	60	0,776	0,746	+3,0	15	32	13	+0,82	0,413	1,000
Pool A + kodo teisėjas	60	0,771	0,726	+4,5	19	31	10	+1,92	0,060	0,239
Pool B be teisėjo	60	0,761	0,687	+7,3	23	27	10	+2,28	0,026	0,106
Pool B + kodo teisėjas	60	0,722	0,711	+1,1	19	26	15	+0,37	0,709	1,000
Sujungtas	240	0,758	0,718	+3,99	76	116	48	+2,60	0,010	-

20 lentelė. Adaptyvios temperatūros A/B palyginimo rezultatai pagal konfigūraciją (tasks2/ rinkinys, po 60 porinių palyginimų kiekvienoje konfigūracijoje). „Pergalės“ - porų, kuriose ON rezultatas didesnis už OFF, skaičius; „lygiosios“ - porų, kuriose ON = OFF, skaičius; „pralaimėjimai“ - porų, kuriose ON < OFF, skaičius. Bonferroni pataisos koeficientas - 4 (lygiagrečiai atliktų keturių porinių t kriterijų skaičius). Δ reikšmės apskaičiuotos iš neapvalintų ON ir OFF vidurkių, todėl gali nežymiai skirtis nuo lentelės stulpelių apvalintų reikšmių skirtumo.



16 diagrama. Adaptyvios temperatūros mechanizmo A/B palyginimas tasks2/ rinkinyje (4 konfigūracijos $\times$ 60 užduočių = 240 ON/OFF porų). Kiekvienoje konfigūracijoje ON režimo vidurkis viršija OFF režimo vidurkį.

Atskira analizė pagal užduoties sudėtingumo lygį atskleidžia, kad stipriausias adaptyvios temperatūros efektas užfiksuotas tose pačiose sudėtingiausiose užduotyse, kuriose anksčiau šiame poskyryje aprašytas peak_after_debate dažnis yra aukščiausias. Pool B be teisėjo konfigūracijoje ekstremaliojo lygio užduotyse vidurkių skirtumas siekia +10,6 procentinio punkto (ON 0,658, OFF 0,552, $n = 40$, $t = +2,26$, $p = 0,029$); šis rezultatas pasiekia reikšmingumo lygį prieš taikant daugybinių palyginimų pataisą, todėl laikytinas stiprių papildomu signalu, o ne atskiru galutiniu įrodymu. Pool A + kodo teisėjas konfigūracijoje ekstremaliajame lygyje skirtumas yra +5,9 procentinio punkto, o jo statistinis reikšmingumas išlieka ribinis ($p = 0,093$). Sudėtingo lygio užduotyse efektai yra mažesni ir niekur nepasiekia statistinio reikšmingumo - tai dera su anksčiau šiame poskyryje aprašytu dėsningumu, jog debatų ciklo raidos mechanizmai daugiausia veikia tuose sudėtingumo lygmenyse, kuriuose pirmojo raundo pasiūlymai dažniausiai dar netinkami.

Adaptyvios temperatūros efekto mechanizmą tiesiogiai atskleidžia anksčiau šiame poskyryje aptartas raundų skaičiaus rodiklis. Pool B be teisėjo konfigūracijoje su įjungta adaptyvia temperatūra vidutinis raundų skaičius siekia 2,77 raundo, o su išjungta - 3,27 raundo (porinis Stjudento t kriterijus $t = -2,25$, $p = 0,028$). Tai reiškia, kad adaptyvi temperatūra ne tik pagerina galutinį testų išlaikymo rodiklį šioje konfigūracijoje, bet ir tai daro statistiškai reikšmingai sumažindama vidutinį raundų skaičių - sistemos sprendimas randamas anksčiau.

Šis stebėjimas atitinka 2.4.3 poskyryje aprašytą mechanizmo logiką: stagnacijos metu padidintas generavimo nepastovumas sukuria didesnę pasiūlytojų atsakymų įvairovę ir leidžia greičiau ištrūkti iš lokaliajo optimumo, kuriame agentai be šios įvairovės dažnai užstringa. Likusiose trijose konfigūracijose raundų skaičiaus skirtumas yra mažesnis ir statistiškai nereikšmingas, tačiau efekto kryptis trijose iš keturių konfigūracijų yra ta pati - su įjungta adaptyvia temperatūra raundų skaičius vidutiniškai mažesnis arba lygus.

Pažymėtina, kad šio A/B palyginimo „pralaimėjimų“ sąvoka iš esmės skiriasi nuo 4.2 poskyryje aptartų porinių palyginimų prieš solo bazinę liniją. Čia lyginami du nepriklausomi to paties debato eksperimento vykdymai (su ir be adaptyvios temperatūros), kuriuose anti-regresijos politika veikia kiekviename vykdyme atskirai, bet ne tarp jų. Atskirai patikrinta, kad nė viename iš 240 ON vykdymų ir nė viename iš 240 OFF vykdymų galutinis rezultatas nebuvo prastesnis už geriausią to paties vykdymo pradinį pasiūlymą - taigi 4.2 poskyryje aprašytas „nulinio regresijų iš 600“ rezultatas pagal šio A/B eksperimento duomenis išsilaiko ir abliacijos sąlygomis. Vis dėlto pirmojo raundo pasiūlytojų sprendimų stochastinis pobūdis (žr. 2.4.3 poskyrį) reiškia, kad ON ir OFF vykdymai gali turėti šiek tiek skirtingus pirmojo raundo maksimumų pasiskirstymus, todėl atskirose užduotyse OFF gali atsitiktinai pasirodyti geriau nei ON. Tačiau agreguotame 240 porų vaizde ši stochastika nusveria bendros sisteminės tendencijos (76 pergalės prieš 48 pralaimėjimus, $t = +2,60$, $p = 0,010$), todėl sprendimas dėl mechanizmo efektyvumo priimamas pagal agreguotą rezultatą, o ne pagal atskirų užduočių rezultatų svyravimus.

Apibendrinant, šio poskyrio dinamikos rodikliai ir kontroliuotas A/B palyginimas leidžia atsekti nuoseklų sistemos elgsenos dėsningumą. Pirmą, debatų ciklas po pirmojo raundo prideda vertę pirmiausia ekstremaliojo lygio užduotyse (27-38% *peak_after_debate*), o lengvojo ir vidutinio lygio užduotyse jis yra beveik visada perteklinis - tai leidžia praktiškai interpretuoti, kad anksti pasibaigiantys debatai (47,3% atvejų po pirmojo raundo) yra ne sistemos trūkumas, o tinkama elgsena. Antra, vidutinio raundų skaičiaus pasiskirstymas (lengvas 1,00 → ekstremalus 3,24) tiesiogiai atspindi sudėtingumą - sistema tęsia debatus papildomais raundais tik tada, kai jų reikia. Trečia, ištaisytų klaidų skaičiaus stipriausia koreliacija su pagerėjimu ($r = +0,446$, $n = 600$) atspindi tą patį mechanizmą, kuris padeda paaiškinti 4.4 poskyryje aptartą teisėjo įtraukimo ribotumą. Ketvirta, kontroliuotas adaptyvios temperatūros A/B palyginimas (240 porinių palyginimų) patvirtina, kad šis 2.4.3 poskyryje aprašytas mechanizmas suteikia statistiškai reikšmingą pridėtinę vertę (+3,99 procentinio punkto sujungtame palyginime, $p = 0,010$). Stipriausias signalas gautas Pool B be teisėjo konfigūracijoje ekstremaliojo lygio užduotyse (+10,6 procentinio punkto, $p = 0,029$ prieš daugybinių palyginimų pataisą), o mechanizmo veikimo požymis matomas ir kaip statistiškai reikšmingas vidutinio raundų skaičiaus sumažėjimas šioje konfigūracijoje (nuo 3,27 iki 2,77 raundo, $p = 0,028$).

Esminis šio poskyrio empirinis rezultatas yra toks: debatų metodo veikimo mechanizmas yra ne pats kritikos generavimas, o kritikos pavertimas konkrečiais kodo pataisymais. Sistemos viduje veikiantys raidos kontrolės elementai - anti-regresijos politika, adaptyvi temperatūra ir kritikos istorija - sudaro praktines sąlygas, kad ši transformacija realiai įvyktų. Adaptyvios temperatūros A/B palyginimas atskirai patvirtina vieno iš šių elementų empirinę vertę.

Tolesniame 4.6 poskyryje pateikiama kokybinė atskirų debatų transkriptų analizė, kuri konkrečiomis užduotimis iliustruoja, kaip šios dinamikos rodikliai pasireiškia individualių debatų eigoje.

4.6. Kokybinė debatų analizė: tipiški scenarijai

Šis poskyris papildo ankstesnių poskyrių kiekybinę analizę kokybiniu konkrečių debatų transkriptų ir CSV įrašų nagrinėjimu, siekiant atskleisti, kaip 4.5 poskyryje aptarti dinamikos

rodikliai (peak_after_debate, raundų skaičius, ištaisytų klaidų skaičius) pasireiškia individualių debatų eigoje. Iš 600 atliktų debatų vykdymų atrenkami keturi tipiški scenarijai, kurie kartu apima pagrindinius sistemos elgsenos režimus: sėkmingas pasiūlymo perėmimas (debatų konsensusas susidaro apie geriausią pradinį pasiūlymą), ryški „gelbėjimo“ situacija (visi pasiūlytojai pirmajame raunde nepateikia visiškai veikiančio sprendimo, o debatas iškelia rezultata virš 80 procentų), debatų sąstingis (rezultatas nepasiekiamas net per visus penkis raundus) ir „neištaisytos kritikos“ reiškinys (didelis rastų klaidų skaičius nepaverčiamas faktiniais kodo pataisymais). Pasirinkti kiekvieno scenarijaus atstovai paimti iš tasks/ ir tasks2/ užduočių rinkinių. Pažymėtina, kad konkrečios užduoties analizėje toliau šiame poskyryje gali būti minima iki dešimties vykdymų - tai apima keturias pagrindines konfigūracijas (Pool A ir Pool B, su kodo teisėju ir be jo) po du kartus (tasks/ ir tasks2/ rinkiniuose, kai užduotis pateko į abu rinkinius) bei dvi samprotavimo teisėjo konfigūracijas (vykdytas tik tasks2/ rinkinyje), iš viso susidaro iki dešimties vykdymų vienai užduočiai. Poskyrio pabaigoje pateikiama 21 lentelė su penkiolika užduočių, kuriose debatai pasiekė ryškiausią pagerėjimą nuo pradinio nulinio pass@3 rezultato.

Pirmasis scenarijus - sėkmingas pasiūlymo perėmimas - geriausiai iliustruoja sudėtingo lygio užduotis „Minimum Window Substring“, kurioje visi dešimt išbandytų vykdymų pasiekė tą patį galutinį 0,889 rezultatą (8 iš 9 testų). Pavyzdiniame Pool B be teisėjo debate (debate_id 97f084fc) pirmajame raunde pasiūlytojas „Granite Code“ pateikė sprendimą, kuris išlaikė tik 4 iš 9 testų (44,4%), o „CodeGeeX4“ ir „Yi-Coder“ - po 8 iš 9 testų (88,9%). Antrajame raunde, susipažinęs su stipresnių komandos narių pasiūlymais ir gavęs kritiką, „Granite Code“ perdarė savo sprendimą ir taip pat pasiekė 88,9% rezultatą - revizijos metu jo realizacija struktūriškai priartėjo prie „CodeGeeX4“ varianto, naudojančio Counter klasę ir aiškesnę slankiojančio lango logiką. Debatas pasibaigė po antrojo raundo pasiektu konsensusu. Šis atvejis rodo svarbų sistemos veikimo bruožą: net kai pirmajame raunde silpniausias pasiūlytojas pateikia akivaizdžiai prastesnį sprendimą, debatų ciklas leidžia jam adaptuoti komandos žinias - stipresni pasiūlytojai netiesiogiai „pakelia“ silpnesnį, o galutinis sprendimas priartėja prie stipresnių pasiūlytojų varianto. Su anti-regresijos politika (žr. 2.4.1 poskyrį) toks scenarijus iš esmės negali pabloginti galutinio testų rodiklio. Platesniame raundų pasiskirstyme matyti, kad sistema dažnai sustoja anksti: 47,3% visų debatų pasibaigia jau po pirmojo raundo (žr. 4.5 poskyrio raundų pasiskirstymą). Vis dėlto ši statistika labiau apibūdina ankstyvo konsensuso dažnį, o ne išskirtinai pasiūlymo perėmimo scenarijų.

Antrasis scenarijus - ryški „gelbėjimo“ situacija - rodo, kaip debatas iškelia rezultatą iš visiškai nesėkmingo pradinio bandymo. Ekstremaliojo lygio užduotis „Plugin System“ iliustruoja šį scenarijų ypač ryškiai: iš dešimties išbandytų vykdymų šešiuose pradinis pass@3 rezultatas buvo nulinis (nė vienas iš trijų pasiūlytojų pirmajame raunde nepateikė visiškai veikiančio sprendimo - visi turėjo bent vieną neišlaikytą testą), tačiau galutinis rezultatas svyravo nuo 0,667 iki 1,000. Pool A be teisėjo atveju iš 23 kritikos metu rastų klaidų faktiškai ištaisytos 6, o galutinis pass_rate pakilo iki 0,889. Pool B be teisėjo atveju panašus debatas po penkių raundų taip pat pasiekė 0,889 rezultatą, nors konkrečių pataisymų skaičius buvo mažesnis. Be to, keturiuose iš dešimties vykdymų Plugin System buvo visiškai išspręsta (pass_rate = 1,000) - dviejuose iš jų pradiniame raunde jau buvo veikiantis sprendimas (pass@3 = 1,000, debatas pabaigtas po pirmojo raundo), o kituose dviejuose pradinis pass@3 buvo tik dalinis (0,250 ir 0,500), kurį debatų ciklas iškėlė iki visiško testų išlaikymo. Šis modelis empiriškai iliustruoja 4.5 poskyryje nustatytą stipriausią koreliaciją - būtent realiai ištaisytų klaidų skaičius, o ne kritikos apimtis, yra esminis veiksnys, susijęs su rezultato pagerėjimu po debatų.

Trečiasis scenarijus - debatų sąstingis - aprašo atvejus, kai net po penkių debatų raundų galutinis pass_rate išlieka 0,000. Iš 600 debatų vykdymų 28 atvejais buvo pasiektas maksimalus penkių raundų limitas su galutiniu pass_rate = 0,000. Sąstingio atvejai dažniausiai pasitaikė šiose užduotyse: „Mathematical Expression Evaluator“ (šešios iš dešimties vykdymų), „Mini

Language Interpreter” (taip pat šešios), „SQL-like Query Engine” (keturios) ir „Command Pattern with Undo/Redo Stack” (keturios). Visoms šioms užduotims būdingas tikslus rekursinės arba būsenos perdavimo logikos modeliavimas - išraiškų medžio konstravimas, kintamųjų aplinkos valdymas ar transakcijų atsukimas, kurio 7-9 milijardų parametrų klasės modeliai dažnai nesukonstruoja pakankamai tiksliai. Sąstingio atvejais raundų skaičius nemažėja, o klaidų randama daug (28 sąstingio atvejuose tarpkvartilinis rastų klaidų intervalas yra 72-130, vidutinis - 105), tačiau realių pataisymų - vienetai arba nuliai. Tai rodo, kad debatų sistema neturi mechanizmo atpažinti esminę pasiūlytojų galimybių ribą - ji toliau generuoja papildomą kritiką, bet negali kompensuoti modelių žinių ar gebėjimų stokos.

Ketvirtasis scenarijus - „neištaisytos kritikos” reiškiny - tiesiogiai pratęsia trečiąjį ir kartu paaiškina, kodėl sąstingis pasitaiko nepaisant gausios kritikos. Iš 600 vykdymų 115 atvejų pasiūlytojai aptiko trisdešimt arba daugiau klaidų, tačiau nė vienos jų neištaisė - kritikos signalas neperaugo į kodo pakeitimą. Ypač ryškus pavyzdys yra ta pati „Mathematical Expression Evaluator” užduotis Pool A su kodo teisėju konfigūracijoje: kritikai bendrai rado 143 klaidas penkiuose debatų raunduose, tačiau realiai ištaisytų - nė vienos (final = 0,000). Toje pačioje užduotyje Pool A su samprotavimo teisėju konfigūracija rodo priešingą vaizdą - rastų klaidų skaičius gerokai mažesnis (18), tačiau faktiškai ištaisytos 3, ir galutinis rezultatas pakilo iki 0,889. Šis kontrastas - 143 rastos klaidos ir 0 ištaisytų klaidų bei 0,000 pass_rate vienu atveju, palyginti su 18 rastų klaidų, 3 ištaisytomis klaidomis ir 0,889 pass_rate kitu atveju - empiriškai iliustruoja 4.5 poskyrio centrinę išvadą: kritikos kokybė ir jos transformacija į konkrečius kodo pataisymus yra svarbesnė nei kritikos apimtis. Pažymėtina, kad terminas „neištaisyta kritika” šiame darbe vartojamas operacine prasme - kritikos teiginys, kuris nebuvo paverstas kodo pataisymu, nepriklausomai nuo to, ar jis identifikavo tikrą defektą, ar buvo perteklinis komentaras.

Užduotis	Sudėtingumas	Geriausias galutinis pass_rate	Pradinis pass@3
Stream Processing Pipeline with Operators	Ekstremalus	0,938	0,000
Calculator with AST and Variables	Ekstremalus	0,923	0,000
State Snapshot and Restore	Ekstremalus	0,923	0,000
Pub/Sub System with Hierarchical Topics	Ekstremalus	0,909	0,000
Read-Write Lock Manager (Non-blocking)	Ekstremalus	0,909	0,000
Simple ORM with In-Memory Storage	Ekstremalus	0,909	0,000
Bloom Filter with Counting Variant	Ekstremalus	0,900	0,000
Graph Library with Algorithms	Ekstremalus	0,900	0,000
Mini Key-Value Database	Ekstremalus	0,900	0,000
Palindrome Partitioning Min Cuts	Sudėtingas	0,900	0,000
Retry Policy with Backoff Strategies	Ekstremalus	0,900	0,000
Data Transformation Pipeline	Ekstremalus	0,889	0,000
Doubly Linked List Library	Sudėtingas	0,889	0,000
Event-Driven System	Ekstremalus	0,889	0,000
JSON Parser	Ekstremalus	0,889	0,000

21 lentelė. Penkiolika sudėtingo ir ekstremalaus lygio užduočių iš 39 panašių atvejų, kuriose bent vienoje konfigūracijoje debatai iškėlė rezultatą nuo pradinio nulinio pass@3 iki bent 0,8 galutinio pass_rate. Pateikiamas geriausias kiekvienos užduoties debatų rezultatas; kai keletas užduočių pasiekia tą patį rezultatą (12 užduočių pasiekė 0,889), atrenkamos pirmosios pagal abėcėlinę tvarką, todėl į pateiktą sąrašą nepateko kai kurios šiame poskyryje atskirai aptartos užduotys (pvz., „Minimum Window Substring” ir „Plugin System”).

Iš 21 lentelės matoma, kad ryškus „gelbėjimo“ efektas yra ne pavienis atvejis, o reguliarius debatų ciklo elgsenos bruožas tose užduotyse, kuriose individualūs pasiūlytojai negali pateikti tinkamo sprendimo. Pažymėtina, kad nė vienoje iš šių penkiolikos užduočių pradinis pass@3 nebuvo aukštesnis už nulį - tai reiškia, kad nė vienas iš trijų pasiūlytojų pirmajame raunde nepateikė visiškai veikiančio sprendimo. Vis dėlto bendras debatų ciklo galutinis rezultatas viršija 0,88, o keliuose atvejuose pasiekia 0,90 ir aukštesnį pass_rate. Tai iliustruoja 4.2 poskyryje pateiktą gelbėjimo dažnio rodiklį (84-94% pagal konfigūraciją, vidutiniškai 90,1%), papildant jį konkrečių užduočių pavyzdžiais.

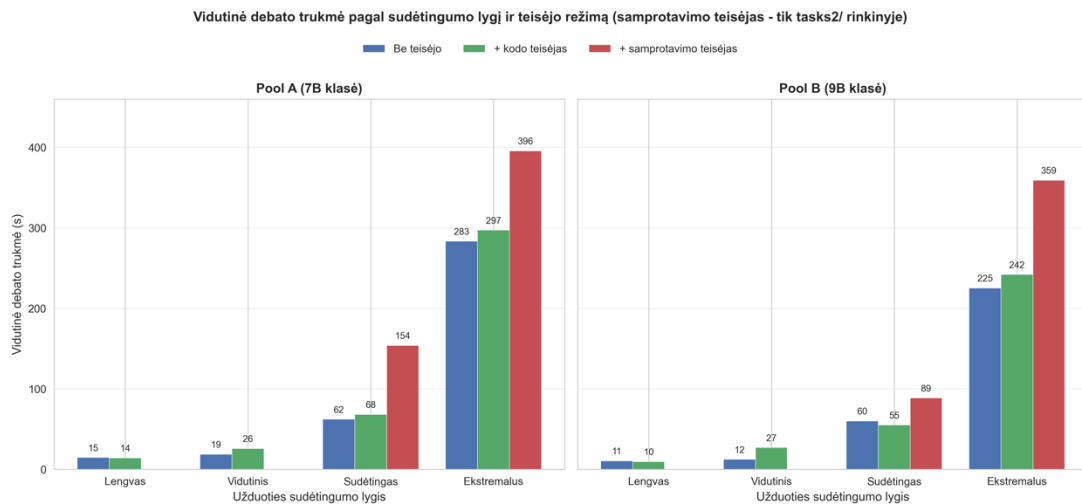
Apibendrinant kokybinę analizę, keturi atskleisti scenarijai sudaro nuoseklią debatų sistemos elgsenos klasifikaciją. Sėkmingo perėmimo ir ryškaus gelbėjimo scenarijai parodo, kuriose situacijose debatai realiai prideda vertę - kai pasiūlytojų pradiniai sprendimai skiriasi savo kokybe arba kai nė vienas iš jų pirmajame raunde nepateikia veikiančio sprendimo. Sąstingio ir neištaisytos kritikos scenarijai parodo šio metodo ribas - debatų sistema negali kompensuoti esminių pasiūlytojų gebėjimų stokos, o pati gausi kritika be efektyvios transformacijos į kodo pataisymus tampa triukšmu, kuris neperauga į realų rezultato pagerėjimą. Tolesniame 4.7 poskyryje analizuojamos skaičiavimo sąnaudos bei statinio kodo kokybės rodikliai (Pylint, Radon), papildantys kokybinę dinamikos analizę kiekybinėmis kompromiso metrikomis. Vėliau, 4.8 poskyryje, šie empiriniai rezultatai gretinami su platesnėmis literatūroje aptariamomis išvadomis apie kelių agentų debatų metodo ribas ir potencialą.

4.7. Skaičiavimo sąnaudos ir statinė kodo kokybė

Šis poskyris kiekybiškai įvertina kompromisą tarp debatų metodo testų išlaikymo prieaugio (4.2-4.5 poskyriuose) ir to prieaugio kainos: skaičiavimo laiko bei statinės kodo kokybės pokyčio. Analizė remiasi visais 600 debatų vykdymų (po 120 keturioms pagrindinėms konfigūracijoms ir po 60 dviem samprotavimo teisėjo konfigūracijoms) ir 360 solo bazinės linijos vykdymų. Vertinami trys rodikliai: vidutinė debato trukmė pagal sudėtingumo lygį, statinis Pylint įvertis prieš ir po debatų ciklo bei Radon biblioteka apskaičiuotas ciklominis kompleksiskumas. Skirtingai nuo 4.4 poskyrio 17 lentelės, kurioje pateikti agreguoti trukmės vidurkiai tik tasks2/ rinkinyje (po 60 vykdymų konfigūracijoje), šio poskyrio trukmės analizė atskleidžia, kaip sąnaudos auga didėjant užduoties sudėtingumui. Pylint ir kompleksiskumo analizė skaičiuojama per visus 600 debatų vykdymų, todėl agreguoti vidurkiai gali šiek tiek skirtis nuo siauresnių 4.4 poskyrio rodiklių.

Konfigūracija	Lengvas	Vidutinis	Sudėtingas	Ekstremalus	Bendras	n
Pool A be teisėjo	14,6	18,6	62,1	283,3	152,1	120
Pool A + kodo teisėjas	14,1	25,9	68,3	297,0	161,0	120
Pool A + samprotavimo teisėjas	-	-	153,7	395,5	314,9	60
Pool B be teisėjo	10,5	12,4	60,1	225,2	123,6	120
Pool B + kodo teisėjas	9,6	27,0	55,0	241,9	131,5	120
Pool B + samprotavimo teisėjas	-	-	88,6	358,9	268,8	60

22 lentelė. Vidutinė debato trukmė (sekundėmis) pagal teisėjo režimą ir užduoties sudėtingumo lygį (visi 600 debatų vykdymų). Brūkšneliai samprotavimo teisėjo eilutėse žymi tai, kad ši konfigūracija buvo vykdyta tik tasks2/ rinkinyje, kuriame nėra lengvojo ir vidutinio lygio užduočių (žr. 3.1 poskyrio 2 lentelę). Todėl samprotavimo teisėjo eilučių „Bendras“ vidurkis skaičiuojamas tik iš sudėtingo ir ekstremaliojo lygio užduočių (n = 60) ir nėra tiesiogiai palyginamas su 120 vykdymų apimančių konfigūracijų bendrais vidurkiais



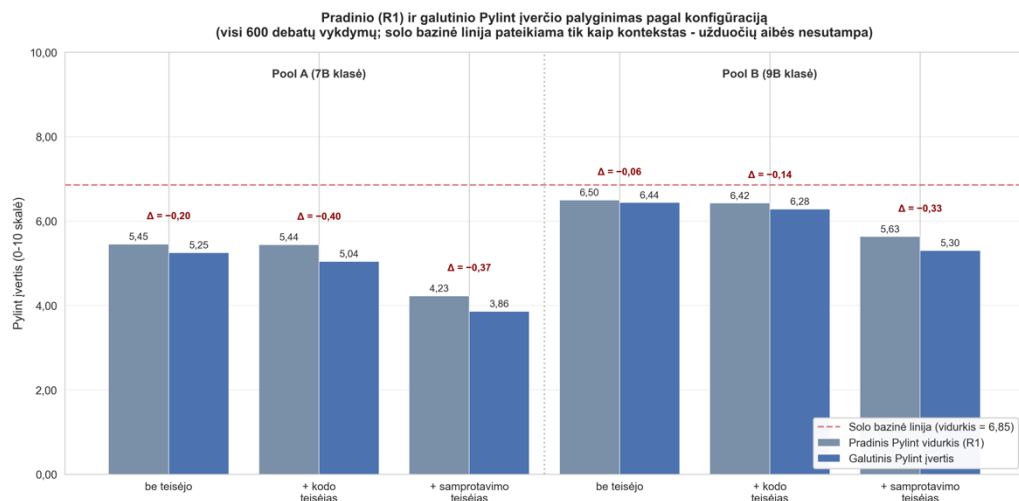
17 diagrama. Vidutinė debato trukmė pagal sudėtingumo lygį ir teisėjo režimą (lengvojo ir vidutinio lygio grupėse samprotavimo teisėjo stulpelių nėra dėl duomenų nebuvimo - ši konfigūracija vykdyta tik tasks2/ rinkinyje)

Pagrindiniai trukmės dėsniniai pateikiami 22 lentelėje ir 17 diagramoje. Pirma, debato trukmė monotoniškai auga didėjant užduoties sudėtingumui: Pool A be teisėjo lengvojo lygio užduotys vidutiniškai trunka 14,6 sek., o ekstremaliojo - 283,3 sek., t. y. apie 19 kartų ilgiau; Pool B be teisėjo atveju santykis dar didesnis - nuo 10,5 sek. lengvajame iki 225,2 sek. ekstremalajame lygyje (apie 21 kartą). Šis ryškus trukmės šuolis tarp sudėtingo ir ekstremaliojo lygio (pavyzdžiui, Pool A be teisėjo: 62,1 → 283,3 sek., t. y. $\times 4,6$) atspindi 4.5 poskyryje aptartą raundų skaičiaus augimą: lengvojo lygio užduotys vidutiniškai pasibaigia po 1,00 raundo, o ekstremaliojo - po 3,24 raundo. Tai didina bendrą debatų trukmę, o ilgesni pasiūlymų ir kritikos tekstai papildomai ilgina atskirus raundus. Antra, agreguotai Pool B (9B klasė) yra šiek tiek greitesnis nei Pool A (7B klasė): be teisėjo režime atitinkamai 123,6 prieš 152,1 sek. (Pool A maždaug 23 procentais lėtesnis), nepaisant didesnių 8-9 milijardų parametrų modelių. Šis efektas paaiškinamas tuo, kad Pool A sudaro „CodeLlama“ 7B variantas, kuris pasirodo lėčiausias iš visų šešių pasiūlytojų (žr. 3.2 poskyrį) ir tempia visą Pool A vidurkį aukštin, ypač ekstremaliojo lygio užduotyse.

Trečia, teisėjo įtraukimo papildoma trukmė absoliučiais skaičiais labiausiai išryškėja ekstremaliojo lygio užduotyse. Pool A atveju ekstremalajame lygyje kodo teisėjas pridėjo tik +4,8% trukmės (297,0 prieš 283,3 sek.), o samprotavimo teisėjas - +39,6% (395,5 prieš 283,3 sek.); Pool B atveju kodo teisėjas pridėjo +7,4% (241,9 prieš 225,2 sek.), o samprotavimo teisėjas - +59,4% (358,9 prieš 225,2 sek.). Lyginant vienodomis sąlygomis tasks2/ rinkinyje (kuris atitinka 4.4 poskyrio 17 lentelę) samprotavimo teisėjo perteklinė trukmė yra +55,1% Pool A atveju ir +58,2% Pool B atveju, palyginti su tos pačios konfigūracijos be teisėjo varianto, o kodo teisėjas atitinkamai pridėdavo tik +14,4% ir +16,0% trukmės. Šis didžiulis sąnaudų skirtumas tarp samprotavimo ir kodo teisėjo paaiškinamas tuo, kad „DeepSeek-R1“ prieš galutinį atsakymą generuoja vidinį samprotavimo grandinės tekstą (angl. chain-of-thought), kurio žetonų skaičius dažnai gerokai viršija paties kritinio sprendimo apimtį (žr. 4.4 poskyrį). Palyginimui pažymėtina, kad solo režime vidutinė vykdymo trukmė per visus 360 vykdymų yra 7,9 sek. (mediana 5,8 sek.; pagal sudėtingumo lygį - nuo 4,2 sek. lengvajame iki 14,1 sek. ekstremalajame) - net agreguotas debatų vidurkis (123-315 sek.) yra maždaug 15-40 kartų ilgesnis, todėl debatų metodas reikalauja didelių skaičiavimo sąnaudų ir tampa pateisinamas tik kartu su 4.2 poskyryje pateiktu 27,3 procentinio punkto testų išlaikymo prieaugiu.

Konfigūracija	Pradinis Pylint	Galutinis Pylint	Δ Pylint	Pradinis kompleksišku mas	Galutinis kompleksišku mas	Δ kompleksiškumas
Pool A be teisėjo	5,45	5,25	-0,20	4,49	4,52	+0,03
Pool A + kodo teisėjas	5,44	5,04	-0,40	4,44	4,35	-0,09
Pool A + samprotavimo teisėjas	4,23	3,86	-0,37	4,07	4,12	+0,05
Pool B be teisėjo	6,50	6,44	-0,06	4,12	4,26	+0,14
Pool B + kodo teisėjas	6,42	6,28	-0,14	4,19	4,20	+0,01
Pool B + samprotavimo teisėjas	5,63	5,30	-0,33	3,80	3,78	-0,02

23 lentelė. Statinės kodo kokybės rodikliai prieš ir po debatų ciklo (visi 600 debatų vykdymų; pradinė reikšmė - pirmojo raundo trijų pasiūlymų vidurkis, galutinė - galutinio konsoliduoto sprendimo įvertis)



18 diagrama. Pradinio (R1) ir galutinio Pylint įverčio palyginimas pagal konfigūraciją (visi 600 debatų vykdymų; solo bazinė linija pateikiama tik kaip kontekstas - užduočių aibės nesutampa)

Statinės kodo kokybės rezultatai pateikti 23 lentelėje ir 18 diagramoje. Pylint įvertis (0-10 skalė, didesnė reikšmė reiškia geresnę statinę kodo kokybę) visose šešiose konfigūracijose po debatų ciklo šiek tiek sumažėja. Sumažėjimo dydis svyruoja nuo -0,06 balo (Pool B be teisėjo) iki -0,40 balo (Pool A su kodo teisėju), t. y. nuo 0,6% iki 4,0% visos 10 balų skalės. Bendras dėsniumas - konfigūracijose su teisėju Pylint sumažėjimas dažniausiai yra didesnis nei režime be teisėjo: be teisėjo režime sumažėjimas yra mažiausias (Pool A -0,20, Pool B -0,06), kodo teisėjo režime - vidutinis (-0,40 ir -0,14), o samprotavimo teisėjo režime - panašaus dydžio kaip kodo teisėjas (-0,37 ir -0,33). Atkreiptinas dėmesys, kad samprotavimo teisėjo konfigūracijose absoliutus galutinis Pylint įvertis yra žemiausias (3,86 Pool A ir 5,30 Pool B), tačiau šis žemumas iš esmės atspindi ne stipresnį debatų neigiamą poveikį, o tai, kad ši konfigūracija vykdyta tik tasks2/ rinkinyje, kuriame yra tik sudėtingo ir ekstremaliojo lygio užduotys (sudėtingesnis kodas paprastai surenka mažesnę Pylint balą dėl ilgesnių funkcijų, didesnio kintamųjų skaičiaus ir gausesnių stilistinių pastabų). Pradinis (R1) Pylint vidurkis šioms konfigūracijoms taip pat yra žemiausias (4,23 ir 5,63), todėl absoliutus pokytis (-0,37 ir

-0,33) išlieka panašaus dydžio kaip kitose konfigūracijose - debatų ciklas pats statinės kokybės dramatiškai nepablogina, tiesiog pradinis lygis yra žemesnis.

Solo bazinės linijos Pylint vidurkis per 360 vykdymų yra 6,85 (visi modeliai kartu) - ši reikšmė 17 diagramoje pateikiama tik kaip kontekstas, bet ji nėra tiesiogiai palyginama su debatų finaliniais įverčiais, nes solo režimas apima visą sudėtingumo spektrą (po 15 lengvojo, vidutinio, sudėtingo ir ekstremaliojo lygio užduočių), o debatų finaliniai įverčiai apima ir tasks2/ rinkinio papildomas sudėtingo bei ekstremaliojo lygio užduotis, kuriose statinė kokybė natūraliai yra žemesnė. Korektiškas palyginimas su solo bazine linija būtų galimas tik suvienodinus užduočių aibes, tačiau toks vertinimas peržengia šio poskyrio apimtį - svarbiausia šio poskyrio išvada yra ne absoliutus solo prieš debatus Pylint skirtumas, o tai, kad debatų ciklas pats savaime statinės kodo kokybės iš esmės nebloginą.

Trečiasis statinės kokybės rodiklis - Radon ciklomatinis kompleksiskumas, t. y. kontrolės srauto sudėtingumo rodiklis, susijęs su nepriklausomų vykdymo kelių skaičiumi funkcijose; mažesnė reikšmė rodo paprastesnę kontrolės struktūrą - po debatų ciklo išlieka praktiškai stabilus. Pokyčiai visose šešiose konfigūracijose svyruoja tik nuo -0,09 (Pool A su kodo teisėju) iki +0,14 (Pool B be teisėjo), t. y. absoliutus pokyčio dydis niekur neviršija 0,15 vieneto, kai vidutinės kompleksiskumo reikšmės yra apie 4. Tai reiškia, kad debatų ciklas nei pastebimai supaprastina, nei papildomai sukomplikuoja kodo kontrolės struktūrą - galutinis sprendimas ciklomatiniu požiūriu panašus į pradinį trijų pasiūlymų vidurkį. Šis stabilumas yra teigiamas rezultatas dviem aspektais: pirma, jis rodo, kad pasiūlytojai revizijos fazėje nesistengia „pataisyti“ kodo įvesdami papildomus sąlyginius blokus ar atvejus tik tam, kad pridengtų rastas kritikas (kas būtų nesveikas signalas); antra, jis dera su Pylint analizės išvada, kad debatas neperdaro kodo struktūros iš esmės - jis veikiau pakeičia atskiras eilutes (algoritminius nukrypimus arba ribinių atvejų apdorojimą), išlaikydamas bendrą sprendimo formą.

Apibendrinant kompromiso vaizdą, šio poskyrio rezultatai leidžia suformuluoti tris praktines išvadas. Pirma, samprotavimo teisėjo įtraukimas yra pati brangiausia konfigūracijos modifikacija: lyginant vienodomis sąlygomis ji prideda +55-58% trukmės palyginti su atitinkamu be teisėjo režimu, o pati šios pridėtinės trukmės nauda 4.4 poskyryje pasirodė esanti labai ribota (Pool A trijų režimų skirtumai neviršija 0,5 procentinio punkto testų išlaikymo, Pool B atveju samprotavimo teisėjas tik išlaiko be teisėjo režimo lygį - 0,762 prieš 0,761). Antra, kodo teisėjas yra pigesnis (+14-16% trukmės) ir jis Pool A atveju veikia neutraliai, tačiau Pool B atveju jis aiškiai pablogina rezultatą (0,722 prieš 0,761 be teisėjo, žr. 4.4 poskyrio 15 lentelę) - todėl jo pridėtinė trukmė neperauga į kokybės gerinimą. Trečia, statinės kodo kokybės požiūriu (Pylint -0,06...-0,40 ir kompleksiskumas $|\Delta| \leq 0,14$) debatas yra konservatyvus procesas: jis nei pastebimai pagerina, nei pablogina sprendimo skaitomumo bei struktūros rodiklius. Tai rodo, kad debatų metodo nauda yra koncentruota būtent testų išlaikymo dimensijoje (4.2 poskyryje pateikti +27,3 procentinio punkto), o ne platesnėje kodo kokybės dimensijoje - debatas funkciškai „pataiso“ algoritmines klaidas, bet stilistiškai išlaiko sprendimą beveik tokį pat, koks buvo pirmajame raunde.

Šio poskyrio empirinė esmė vienoje frazėje: skaičiavimo sąnaudų požiūriu samprotavimo teisėjas yra brangiausias režimas su mažiausiu testų išlaikymo prieaugio pagrindimu, o pati debatų sistema, nepaisant maždaug 15-40 kartų didesnės trukmės palyginti su solo režimu, statinės kodo kokybės nepablogina ir struktūrinio kompleksiskumo nepadidina. Tolesniame 4.8 poskyryje šie empiriniai rezultatai gretinami su platesnėmis literatūroje aprašytais kelių agentų debatų metodo išvadomis, aptariamais šio darbo apribojimai ir formuluojamos praktinės rekomendacijos.

4.8. Rezultatų aptarimas literatūros kontekste

Šiame poskyryje sintezuojami visų 4.1-4.7 poskyrių empiriniai rezultatai, jie gretinami su 1 skyriuje aptartomis kelių agentų debatų ir susijusių užklausų strategijų tyrimų išvadomis, įvardijami šio darbo metodologiniai apribojimai, formuluojamos tolesnių tyrimų kryptys ir praktinės rekomendacijos. Skirtingai nuo ankstesnių 4 skyriaus poskyrių, kuriuose buvo pristatomi konkretūs kiekybiniai rezultatai, šis poskyris atlieka aiškinamąjį vaidmenį - apibendrina pagrindines empirines išvadas vienoje vietoje ir paaiškina, kaip jos atitinka arba nukrypsta nuo platesnio literatūros lauko.

Pagrindinių rezultatų santrauka. Pagrindinė empirinė šio darbo išvada yra ta, kad kelių agentų debatų metodas, taikomas atvirųjų svorių 7-9 milijardų parametrų kodo generavimo modeliams daugiafajilių užduočių aplinkoje, suteikia statistiškai labai reikšmingą galutinio testų išlaikymo rodiklio prieaugį, palyginti su vieno modelio solo režimu. Sujungtame 120 užduoties-rinkinio porinių palyginimų rinkinyje keturių pagrindinių debatų konfigūracijų vidurkis (0,824) pranoksta solo bazinę liniją (0,551) 27,3 procentinio punkto skirtumu (Stjudento t kriterijus $t = 17,27$, $p < 10^{-34}$; 4.2 poskyris), o pirminiame ir antrajame rinkiniuose skirtumai atskirai yra +22,7 ir +31,9 procentinio punkto. Iš 120 porinių palyginimų debatai laimėjo 108, baigėsi lygiosiomis 12 atvejų ir nė karto nepralaimėjo solo bazinei linijai. Iš 600 debatų vykdymų nė viename neaptikta regresijos, palyginti su geriausiu komandos nario pirmojo raundo sprendimu - tai tiesioginė 2.4.1 poskyryje aprašytos anti-regresijos politikos pasekmė. Iš 282 visiškai nesėkmingų pradinių situacijų ($R1_max = 0$) debatų ciklas 254 atvejais pagerino rezultatą bent iki iš dalies veikiančio sprendimo, t. y. gelbėjimo rodiklis siekia 90,1% (4.2 poskyris).

Pasiūlytojų rinkinio sudėties hipotezė, kad didesnis parametrų skaičius (Pool B, 9B klasė) lems geresnį debatų rezultatą už mažesnį (Pool A, 7B klasė), šio darbo duomenyse nepasitvirtino: skirtumas tarp dviejų rinkinių sujungtame palyginime neviršija vieno procentinio punkto. Be to, kaip papildomas požymis matyti, kad sudėtingiausiose tasks2/ rinkinio ekstremaliojo lygio užduotyse 7B rinkinys be teisėjo (0,679) netgi pranoksta 9B rinkinį su kodo specializacijos teisėju (0,624; 4.3 poskyris), nors šis palyginimas kartu apima ir teisėjo režimo poveikį.

Teisėjo modelio įtraukimas tasks2/ rinkinio palyginime nesuteikė papildomos naudos: Pool A atveju visi trys režimai davė praktiškai vienodus rezultatus, o Pool B atveju kodo specializacijos teisėjas rezultatą pablogino 3,9 procentinio punkto (4.4 poskyris). Šio efekto mechanizmą atskleidžia klaidų aptikimo „piltuvai“: teisėjas padidina rastų klaidų skaičių 30-100%, tačiau faktiškai ištaisytų klaidų dalis sumažėja nuo 5,2% iki 2,1%. Pats ištaisytų klaidų skaičius yra stipriausias debatų rezultato pagerėjimą prognozuojantis veiksnys - Pirsono $r = +0,446$ ($n = 600$, $p < 10^{-30}$; 4.5 poskyris).

Skaičiavimo sąnaudų požiūriu samprotavimo teisėjas yra brangiausias režimas, trukmę padidinantis 55-58%, palyginti su atitinkamu režimu be teisėjo, tačiau testų išlaikymo rodiklio prieaugio jis beveik nesuteikia (4.7 poskyris). Statinė kodo kokybė (Pylint ir Radon ciklomatinis kompleksiskumas) po debatų ciklo išlieka praktiškai stabili - „Pylint“ rodiklio sumažėjimas 10 balų skalėje siekia 0,6-4,0%, o kompleksiskumo pokyčiai neviršija 0,15 vieneto.

Lyginimas su literatūra. Šio darbo rezultatai gretinami su vienuolika 1 skyriuje aptartų darbų, kuriuose nagrinėjami kelių agentų debatai, vidinė modelio refleksija, vertintojo paradigma ar struktūrinės vaidmenų sistemos. Du ir kt. [Dlt+23] pradiname kelių agentų debatų darbe pademonstravo, kad debatai pagerina rezultatus matematinio bei strateginio samprotavimo užduotyse - šio darbo skaitinis pagerėjimas (+22,7 procentinio punkto pirminiame rinkinyje, +31,9 antrajame) yra panašaus masto, tačiau pasiektas kitoje srityje (daugiafajilis kodo generavimas) ir su gerokai mažesniais modeliais (7-9 milijardų parametrų), palyginti su komerciniais didelio masto modeliais. Liang ir kt. [Lhj+23] DMAD darbe

argumentuojama, kad debatų efektyvumas remiasi agentų strategijų įvairove - šio darbo 4.3 poskyrio rezultatas, kad architektūriškai įvairesnis Pool A pranoksta homogeniškesnį Pool B sudėtingiausiose užduotyse, atitinka šią poziciją ir ją papildo tuo, kad įvairumas šio darbo atveju pasireiškia ne strategijų, o pradinių pasiūlytojų gebėjimų lygmeniu (Pool A solo rezultatų variacija maždaug 1,8 karto didesnė nei Pool B). Smit ir kt. [Sdg+24] sisteminiame palyginime nustatė, kad MAD sistemos be tinkamo hiperparametrų derinimo nepatikimai pranoksta paprastesnes užklausų strategijas - šio darbo eksperimentinė aplinka su anti-regresijos politika, adaptyvia temperatūra, kritikos istorija ir sąmoningai parinktais heterogeniškais pasiūlytojų rinkiniais atitinka būtent tokį atidaus derinimo atvejį, todėl šio darbo rezultatai gali būti interpretuojami kaip empirinis Smit ir kt. iškeltos sąlygos „atidžiai sureguliuotos MAD sistemos pranoksta paprastesnes strategijas“ patikrinimas mažesnių modelių aplinkoje (žr. 4.2 poskyrio pabaigą).

Khan ir kt. [Khv+24] darbas yra vienas aktualiausių teisėjo režimo palyginimui: jų konfigūracijoje silpnescio vertintojo modelio tikslumas QuALITY skaitymo supratimo užduotyse pakyla nuo 48% iki 76%, kai jis stebi dviejų stipresnių ekspertų debatą, palyginti su naiviaja bazinio vertinimo strategija. Šio darbo 4.4 poskyrio rezultatai šios bendros tendencijos nepatvirtina - kodo generavimo aplinkoje teisėjo įtraukimas arba neturėjo poveikio (Pool A), arba pablogino rezultatą (Pool B su kodo teisėju). Empiriškai šis kontrastas paaiškinamas tuo, kad Khan ir kt. užduotys yra teksto supratimo, o vertintojo gebėjimas perskaityti debatą ir įvertinti argumentų logiką tinka tokiai užduočiai gerokai geriau nei kodo defektų aptikimas ir jų transformavimas į konkrečius pataisymus. Šio darbo „klaidų piltuvo“ analizė (4.4 poskyris) konkrečiai parodo, kurioje grandyje šis perėjimas nutrūksta: nors teisėjas randa daugiau klaidų, tų klaidų dalis, kurią pasiūlytojai paverčia konkrečiais kodo pakeitimais, sumažėja. Zheng ir kt. [Zcs+23] sistemingai išanalizavo „LLM-as-Judge“ paradigmą ir stipresnio modelio gebėjimą veikti dialogų vertintojo vaidmenyje - autoriai parodė, kad „GPT-4“ lygio vertintojas pasiekia daugiau nei 80% sutapimą su žmogiškuoju vertinimu, kartu įvardydami ir šios paradigmos apribojimus (pozicijos, žodingumo bei savaiminio palankumo šališkumai). Šio darbo 4.4 poskyrio rezultatai papildo Zheng ir kt. išryškintus šios paradigmos apribojimus: daugiafailio kodo generavimo aplinkoje vertintojo nauda yra dar labiau ribota - net jei teisėjas aptinka daugiau klaidų, jo signalas privalo būti perteikiamas konkrečiais kodo pataisymais, kitaip jis padidina tik kritikos triukšmą.

Wei ir kt. [Wws+22a] Chain-of-Thought darbas pademonstravo, kad vidinis samprotavimo grandinės tekstas pagerina modelio rezultatą sudėtinguose uždaviniuose, o Wang ir kt. [Wws+22b] „Self-Consistency“ idėja apibendrina šį principą taikydama balsavimą tarp kelių nepriklausomų to paties modelio bandymų. Šio darbo samprotavimo teisėjas „DeepSeek-R1“ yra CoT idėjos pritaikymas kritiko vaidmeniui, o pati daugiaagentinė debatų architektūra struktūriškai primena „Self-Consistency“, tik pakartotinius vieno modelio kvietimus pakeitus skirtingų modelių sprendimais. Skaitiniu požiūriu šio darbo samprotavimo teisėjas pasirodė brangus (+55-58% trukmės) be aiškos naudos kodo generavimo užduotyse - tai dera su 4.7 poskyrio kompromiso analizės išvada. Madaan ir kt. [Mtg+23] „Self-Refine“ metode vienas modelis iteruojamas kritikos ir revizijos cikle; šio darbo metodas yra šios idėjos daugiaagentis variantas, kuriame kritiką teikia ne pats sprendimą pateikęs modelis, o kiti rinkinio nariai (ir teisėjas, jei jis įtrauktas). Shinn ir kt. [Scb+23] „Reflexion“ darbe modelis mokosi iš savo klaidų per žmogiškąją arba programinę grįžtamąją ryšį - šio darbo aplinkoje šį vaidmenį atlieka vienetinių testų rezultatai bei kitų agentų kritika, todėl 4.5 poskyrio koreliacija tarp ištaisytų klaidų skaičiaus ir rezultato pagerėjimo ($r = +0,446$) gali būti laikoma „Reflexion“ tipo grįžtamojo ryšio efektyvumo empiriniu rodikliu. Hong ir kt. [Hzc+24] „MetaGPT“ siūlo struktūrinius vaidmenis - produkto vadybininkas, architektas, inžinierius - kurie surinkimo linijos principu paskirstomi tarp skirtingų agentų; šio darbo architektūra šių vaidmenų aiškiai neišskiria (visi agentai veikia kaip pasiūlytojai, o teisėjas, kai naudojamas, veikia atskirai), tačiau revizijos fazė funkciškai vykdo „MetaGPT“ peržiūros etapo vaidmenį. Galiausiai Jiang ir kt. [Jdw+25] „Self-planning“ metodas pasiekė iki 25,4% santykinį Pass@1 pagerėjimą savo

užduočių rinkinyje, palyginti su tiesioginiu kodo generavimu - šio darbo absoliutus +22,7 ir +31,9 procentinio punkto pagerėjimas santykinė prasme atitinka +34% ir +73% pagerėjimą nuo bazinės linijos (žr. 4.2 poskyrį), todėl šio darbo rezultatai yra panašaus arba didesnio masto nei Jiang ir kt., tačiau pasiekti multiagentinės debatų sistemos pagrindu, o ne vidinio savarankiško planavimo - tai rodo, kad panašaus masto pagerėjimas gali būti pasiektas ir paskirstytu būdu su mažesniais modeliais.

Apribojimai. Pirmasis ir svarbiausias apribojimas yra tai, kad kiekviena konfigūracija šio darbo eksperimentuose buvo vykdyta vieną kartą, t. y. buvo atliktas vienas vykdymas kiekvienai konfigūracijos ir užduoties porai, be pakartojimų su skirtingomis atsitiktinumų sėklomis. Tai reiškia, kad porinio Studento t kriterijaus rezultatai (4.2 poskyris) yra skaičiuojami pagal užduotis kaip porinius elementus, o ne pagal nepriklausomus pakartotinius vykdymus, todėl statistinis reikšmingumas atspindi efekto kryptį visame 120 užduoties-rinkinio porų rinkinyje, bet ne efekto stabilumą pakartotinai vykdant tą pačią užduotį. Pasiūlytojų temperatūra (0,3-0,90, žr. 2.4 poskyrį) ir „DeepSeek-R1” vidinio samprotavimo netiesioginis atsitiktinumas reiškia, kad atskirų vykdymų rezultatai gali šiek tiek varijuoti, tačiau šio variavimo dydis šio darbo duomenyse neįvertintas. Tolesnis daugkartinio vykdymo (5-10 sėklų per konfigūraciją per užduotį) tyrimas leistų atskirti sistemos elgsenos stabilumą nuo atsitiktinio nukrypimo. Vis dėlto adaptyviosios temperatūros - vieno iš svarbiausių sistemos mechanizmų - pridėtinė vertė šiame darbe buvo patikrinta atskiru kontroliuotu A/B palyginimu (žr. 4.5 poskyrį, 20 lentelę ir 16 diagramą), kuris parodė +3,99 procentinio punkto pagerėjimą sujungtame 240 porų palyginime ($p = 0,010$). Antrasis apribojimas yra tai, kad šio darbo eksperimentuose dalyvauja išskirtinai atvirųjų svorių 7-9 milijardų parametrų modeliai, vykdomi vietinėje aparatūroje, o komerciniai didelio masto modeliai („GPT-4”, „Claude”) į palyginimą nebuvo įtraukti. Tai reiškia, kad šio darbo rezultatų negalima tiesiogiai taikyti komercinių modelių aplinkoje, kurioje pradinis vieno modelio rodiklis yra žymiai aukštesnis ir gerinimo potencialas debatų metodu objektyviai siauresnis. Trečiasis apribojimas yra tai, kad antrasis užduočių rinkinys (tasks2/), apimantis 60 sudėtingo ir ekstremalaus lygio užduočių, yra šiame darbe sudarytas rinkinys, o ne viešas standartinis etalonas, todėl absoliutus rodiklio dydis nėra tiesiogiai palyginamas su „HumanEval”, „MBPP” ar kitais nusistovėjusiais kodo generavimo etalonais. Užduotys buvo sąmoningai sudarytos kaip daugiafailės su tarpmoduliniais importais (žr. 2.5 ir 3.3 poskyrius), todėl jos struktūriškai sudėtingesnės nei vienos funkcijos „HumanEval” tipo užduotys, kurios dominuoja srities tyrimuose. Ketvirtasis, mažesnio reikšmingumo, apribojimas yra tai, kad šio darbo eksperimentai vykdomi tik „Python” kalbos aplinkoje - kitų programavimo kalbų (Java, C++, JavaScript) aplinkoje debatų metodo elgsena gali skirtis dėl skirtingo kompiliatoriaus ar interpretatoriaus diagnostikos pobūdžio. Galiausiai svarbu pažymėti, kad „nė karto neaptiktos regresijos” rezultatas (4.2 poskyris) atspindi būtent šio darbo eksperimentinę sistemą, kurioje veikia anti-regresijos politika - kitose debatų sistemose, kuriose tokios politikos nėra, regresijos atvejų gali pasitaikyti dažniau.

Tolesnių tyrimų kryptys. Šio darbo rezultatai atveria kelias tolesnių tyrimų kryptis, kurios galėtų patikslinti čia nustatytas tendencijas. Pirmoji - daugkartinio vykdymo verifikacija: pakartoti pagrindinius eksperimentus su 5-10 atsitiktinumų sėklų per užduotį leistų įvertinti rezultatų stabilumą ir apskaičiuoti aiškesnius pasitikėjimo intervalus tokiems rodikliams kaip peak_after_debate dažnis (4.5 poskyris) ir gelbėjimo rodiklis (4.2 poskyris). Antroji - užduočių rinkinio plėtra: integravus „HumanEval+”, „MBPP+”, „APPS” arba „SWE-bench” etalonus, šio darbo rezultatai taptų tiesiogiai palyginami su platesniu literatūros lauku, o tasks2/ rinkinys liktų kaip papildomas šiame darbe sudarytas daugiafailių užduočių pogrupis. Trečioji - komandos dydžio kaita: visuose šio darbo eksperimentuose buvo naudojami trys pasiūlytojai, o didesnio komandos dydžio (4-5 narių) bandymai leistų įvertinti, ar 4.3 poskyryje aptarta įvairumo nauda monotoniškai didėja plečiant komandą, ar ji pasiekia prisotinimo tašką. Ketvirtoji - adaptyvus teisėjo pasirinkimas: šio darbo duomenys rodo, kad teisėjo nauda priklauso nuo pasiūlytojų rinkinio savybių (Pool A atveju poveikis neutralus, o Pool B su kodo

teisėju atveju - neigiamas); galima tirti meta-strategiją, kurioje teisėjo režimas pasirenkamas automatiškai pagal pradinio raundo rezultatų variaciją. Penktoji - daugiakalbio kodo generavimo plėtra: šio darbo metodika, perkėlus į kitas programavimo kalbas, leistų patikrinti, ar 4.4 poskyrio „klaidų piltuvo“ išvada (teisėjas didina kritikos triukšmą, bet ne pataisymų skaičių) yra ypatinga „Python“ kalbos savybė ar bendresnis kelių agentų sistemų reiškiny. Galiausiai - komerciniai didelio masto modeliai: šio darbo metodika, taikoma „GPT-4“ arba „Claude“ klasės modeliams, leistų patikrinti, ar šio darbo nustatyti dėsningumai išlieka, kai pradinis vieno modelio rodiklis yra žymiai aukštesnis.

Skyriaus apibendrinimas. Šio darbo eksperimentai parodė, kad kelių agentų debatų metodas yra empiriškai veiksminga mažesnio masto atvirųjų svorių kodo generavimo modelių rezultatų gerinimo strategija, jei jis derinamas su anti-regresijos politika, kritikos istorija, adaptyvia temperatūra ir sąmoningai heterogenišku pasiūlytojų rinkiniu. Šio metodo pridėtinė vertė yra didžiausia sudėtingiausiose užduotyse, kuriose vieno modelio gebėjimų nepakanka, o pati metodo esminė mechanika - ne vien kritikos generavimas, o jos transformavimas į konkrečius kodo pataisymus - yra pagrindinis veiksnys, paaiškinantis, kuriose situacijose debatų sistema pasiekia realią naudą. Šio darbo metodika ir rezultatai sukuria atspirties tašką tolesniems tyrimams ne tik kelių agentų debatų, bet ir platesnio atvirųjų svorių modelių taikymo daugiafailio kodo generavimo srityje. Konkrečios praktinės rekomendacijos sistemų taikytojams, inžineriniai pasiūlymai kūrėjams bei detalūs tolesnių tyrimų pasiūlymai apibendrinti atskirame skyriuje „Išvados ir pasiūlymai“.

Išvados ir pasiūlymai

Šiame skyriuje pateikiamos pagrindinės šio darbo išvados ir pasiūlymai, suformuluoti remiantis teorinės dalies, sukurtos sistemos, tyrimo metodikos ir eksperimentinių rezultatų visuma. Pirmosios penkios išvados atspindi įvade suformuluotų penkių uždavinių įvykdymą, o tolesnės šešios apibendrina šio darbo empirinius rezultatus ir mokslinį indėlį.

Išvados:

1. Literatūros analizės pagrindu identifikuota mažai ištirta tyrimo niša - kelių agentų debatų (angl. multi-agent debate, MAD) metodo taikymas atvirųjų svorių 7-9 milijardų parametrų kalbos modeliams daugiafailio kodo generavimo užduotyse. Dauguma cituojamų MAD darbų (Du ir kt. [Dlt+23], Liang ir kt. [Lhj+23], Khan ir kt. [Khv+24]) naudoja didelio masto uždarus modelius ar netaiko sudėtingų daugiafailių programų inžinerijos užduočių, todėl atvirųjų svorių mažesnio masto modelių elgsena šiame scenarijuje iki šio darbo buvo empiriškai mažai pažinta. Pagal nustatytus atrankos kriterijus (atvirumas, septynių-devynių milijardų parametrų klasė, konteksto langas, kodo specializacija) atrinkti šeši pasiūlytojų modeliai ir du teisėjo modeliai.
2. Suprojektuota ir įgyvendinta modulinė kelių agentų debatų sistema - apie 24 tūkstančius eilučių sistemos kodo (~18 tūkst. eilučių „Python“ kodo, ~4,3 tūkst. eilučių žiniatinklio sąsajos šablono, ~1,3 tūkst. eilučių paleidimo scenarijų) ir apie 3 tūkst. eilučių užduočių aprašymų JSON formatu (iš viso 27 tūkst.), sluoksniuota šešių paketų architektūra, vienoda viešoji programavimo sąsaja trimis paleidimo režimams: komandinei eilutei, žiniatinklio sąsajai ir paketiniams HPC scenarijams. Pagrindiniai inžineriniai sprendimai - anti-regresijos politika su istoriniu geriausio sprendimo grąžinimu, adaptyvi temperatūra stagnacijai mažinti, kritikos istorija, dalinis (angl. chunked) generavimas daugiafailėms užduotims bei trijų matomumo lygių derinys - leido pritaikyti Du ir kt. [Dlt+23] originalią MAD schemą riboto pajėgumo 7-9 milijardų parametrų modeliams. Sistemos modulinė architektūra leidžia ateityje pakeisti atskirus komponentus (modelio klientą, saugyklą, kontrolės mechanizmus) nekeičiant kitų sluoksnių.
3. Sudaryta dviejų rinkinių užduočių aibė, iš viso 90 unikalių užduočių. Pirminis rinkinys tasks/ apima 60 užduočių keturiuose sudėtingumo lygiuose (po 15 lengvojo, vidutinio, sudėtingo ir ekstremaliojo), o išplėstinis tasks2/ rinkinys apima 60 sudėtingo ir ekstremaliojo lygio užduočių, iš kurių 30 sutampa su pirminio rinkinio užduotimis (pakartotinai vertinamos nepriklausomuose vykdymuose), o kitos 30 yra naujos, šio darbo metu suprojektuotos. Iš tasks2/ rinkinio 40 ekstremaliojo lygio daugiafailių užduočių su tarpmoduliniais importais (parseriai, interpretatoriai, klasikiniai programavimo šablonai, duomenų struktūros, programinės įrangos karkasai) yra specialiai šio darbo metu suprojektuotos, leidžiančios įvertinti debatų pridėtinę vertę tose situacijose, kuriose vieno modelio sprendimo nepakanka.
4. VU MIF didelio našumo skaičiavimo (HPC) klasteryje pagrindiniame eksperimento etape atlikta vienuolika SLURM eksperimentų, iš viso gauta 960 unikalų vykdymo įrašų - 600 kelių agentų debatų ir 360 solo bandymų. Eksperimentai apėmė tris palyginimo ašis: pasiūlytojų rinkinio sudėtį (Pool A su trimis septynių milijardų parametrų modeliais prieš Pool B su trimis aštuonių-devynių milijardų parametrų modeliais), teisėjo modelio tipą (be teisėjo, su kodo specializacijos teisėju „DeepSeek-Coder-V2“, su samprotavimo specializacijos teisėju „DeepSeek-R1“) bei debatų ir solo režimų palyginimą. Visuose pagrindinio etapo eksperimentuose buvo naudojama vienoda sistemos konfigūracija: adaptyvi temperatūra, kritikos istorija, ne daugiau kaip penki debatų raundai, geriausio kito agento sprendimo rodymo režimas ir vienoda „uniform“ revizijos strategija (žr. 3.4 poskyrį). Trijų pasirinktų ašių variavimas leido atsakyti į pagrindinius tiriamuosius klausimus kontroliuojamo lyginamojo eksperimento principu. Papildomai atliktas atskiras kontroliuotas adaptyvios

temperatūros mechanizmo A/B abliacijos eksperimentas: keturi papildomi SLURM darbai sudarė 240 porinių ON/OFF palyginimų. Šis eksperimentas patvirtino mechanizmo pridėtinę vertę: sujungtame palyginime vidutinis efektas siekė +3,99 procentinio punkto ($p = 0,010$), o stipriausias signalas užfiksuotas Pool B be teisėjo konfigūracijoje ekstremaliojo lygio užduotyse (+10,6 procentinio punkto, $p = 0,029$ prieš daugybinių palyginimų pataisą). Taigi, įskaitant papildomą abliacijos eksperimentą, tyrime iš viso panaudota 15 SLURM darbų ir 1200 unikalių vykdymo įrašų.

5. Šio darbo duomenyse kelių agentų debatai stabiliai pranoksta vieno modelio bazinę liniją (tasks/ rinkinyje atskirai vykdytą solo režimą, tasks2/ rinkinyje - pirmojo debatų raundo R1 rezultatą). Sujungtame 120 užduoties-rinkinio porinių palyginimų rinkinyje debatų konfigūracijų vidurkis (0,824) pranoksta šią bazinę liniją (0,551) 27,3 procentinio punkto skirtumu (porinis Stjudento t kriterijus $t = 17,27$, $p < 10^{-34}$). Atskirai pirminiame tasks/ rinkinyje skirtumas siekia +22,7 procentinio punkto, o antrajame, sudėtingesniame tasks2/ rinkinyje - +31,9 procentinio punkto. Iš 120 porinių palyginimų debatai laimėjo 108, baigėsi lygiosiomis 12 atvejų ir nė karto nepralaimėjo solo bazinei linijai. Iš 282 visiškai nesėkmingų pradinių situacijų ($R1_max = 0$) debatų ciklas 254 atvejais pagerino rezultatą bent iki iš dalies veikiančio sprendimo (gelbėjimo rodiklis 90,1%). Šis pranašumas yra monotoniškai didėjantis pagal sudėtingumo lygį - nuo +6,1 procentinio punkto lengvojo lygio užduotyse iki +35,3 procentinio punkto ekstremaliojo lygio užduotyse.
6. Pasiūlytojų modelių parametrų klasės hipotezė, kad didesnio parametrų skaičiaus rinkinys (Pool B, 9B klasė) pranoks mažesnę (Pool A, 7B klasė), šio darbo duomenyse nepasitvirtino. Skirtumai tarp dviejų rinkinių sujungtame palyginime neviršija vieno procentinio punkto (be teisėjo: Pool A - 0,821, Pool B - 0,824; su kodo teisėju: Pool A - 0,830, Pool B - 0,822), o visuose keturiuose porinio Stjudento t kriterijaus palyginimuose statistinis reikšmingumas nepasiekiamas ($p > 0,05$). Kaip papildomas požymis matyti, kad sudėtingiausiose tasks2/ ekstremaliojo lygio užduotyse stebima rezultatų inversija - 7B rinkinys be teisėjo (0,679) netgi pranoksta 9B rinkinį su kodo specializacijos teisėju (0,624; nors šiame palyginime kartu kinta ir teisėjo režimas). Pasiūlytojų sprendimų įvairumas šio darbo eksperimentuose pasirodė ne mažiau svarbus, o kai kuriais atvejais net svarbesnis veiksnys nei atskirų modelių solo stiprumas, o tai sutampa su Liang ir kt. [Lhj+23] DMAD darbo pagrindine idėja.
7. Khan ir kt. [Khv+24] užfiksuotas teisėjo agento poveikis (silpnescio vertintojo tikslumo padidėjimas nuo 48% iki 76% teksto supratimo užduotyse) šio darbo daugiafailio kodo generavimo aplinkoje nepasitvirtino. Pool A atveju visi trys teisėjo režimai davė praktiškai vienodus rezultatus (skirtumai neviršija 0,5 procentinio punkto), o Pool B atveju kodo specializacijos teisėjas rezultatą pablogino 3,9 procentinio punkto (0,722, palyginti su 0,761 be teisėjo). Samprotavimo specializacijos teisėjas Pool B atveju išlaikė be teisėjo lygį (0,762, palyginti su 0,761). Šis kontrastas su literatūra empiriškai parodo, kad „LLM-as-Judge“ paradigmos nauda nėra universali, o priklauso nuo užduočių pobūdžio ir nuo to, kiek teisėjo signalą galima paversti konkrečiais kodo pataisymais - teksto supratimo užduotyse argumentų logikos vertinimas vyksta lengviau nei kodo defektų aptikimas ir jų transformavimas į konkrečius pataisymus.
8. Atskleistas debatų metodo efektyvumo vidinis mechanizmas - klaidų aptikimo ir ištaisymo „piltuvas“. Teisėjas padidina rastų klaidų skaičių 30-100%, tačiau faktiškai ištaisytų klaidų dalis sumažėja nuo 5,2% (be teisėjo, Pool A) iki 2,1% (su teisėju, Pool B), t. y. „daugiau kritikos“ nereiškia „daugiau pataisymų“. Šio darbo Pirsono koreliacijos analizė per visus 600 debatų vykdymų atskleidė, kad būtent ištaisytų klaidų skaičius yra stipriausias rezultato pagerėjimą prognozuojantis veiksnys ($r = +0,446$, $n = 600$, $p < 10^{-30}$), o ne kritikos apimtis, raundų skaičius ar konsensuso pasiekimo dažnis. Tai patvirtina pagrindinę šio darbo vidinio mechanizmo išvalgą: debatų metodo nauda kyla ne iš kritikos generavimo, o iš jos transformavimo į konkrečius kodo pataisymus.

9. Skaičiavimo sąnaudų požiūriu samprotavimo specializacijos teisėjas „DeepSeek-R1” yra brangiausias režimas, trukmę padidinantis 55-58%, palyginti su atitinkamu režimu be teisėjo, tačiau testų išlaikymo rodiklio prieaugio jis beveik nesuteikia. Visas debatų metodas yra 15-40 kartų lėtesnis nei solo režimas (vidutiniškai 123-315 sekundžių, palyginti su 7,9 sekundės solo režimu) - todėl praktiškai jis pasiteisina tik toms užduotims, kuriose gaunamas reikšmingas testų išlaikymo prieaugis (sudėtingos ir ekstremalios užduotys), o paprastesnėms užduotims ekonomiškai labiau pagrįstas yra solo režimas.
10. Statinės kodo kokybės rodikliai po debatų ciklo iš esmės išlieka stabilūs, nors „PyLint” įvertis visose šešiose konfigūracijose nežymiai sumažėja - sumažėjimas svyruoja nuo -0,06 iki -0,40 balo, t. y. nuo 0,6% iki 4,0% visos dešimties balų skalės. „Radon” ciklomatinio kompleksiskumo pokyčiai visose konfigūracijose neviršija 0,15 vieneto, kai vidutinės kompleksiskumo reikšmės yra apie 4. Iš 600 debatų vykdymų nė viename neaptikta regresijos, palyginti su geriausiu komandos nario pirmojo raundo sprendimu - tai tiesioginė sistemoje veikiančios anti-regresijos politikos pasekmė. Apibendrinant galima teigti, kad debatų ciklas lemia nedidelį „PyLint” įverčio sumažėjimą, tačiau nepablogina struktūrinio kompleksiskumo rodiklių - jo nauda koncentruojama būtent funkcinio testų išlaikymo dimensijoje.
11. Šio darbo skaitiniai pagerėjimai (+22,7 procentinio punkto pirminiame rinkinyje, +31,9 antrajame ir +27,3 sujungtame) yra panašaus masto kaip ir literatūroje pranešti rezultatai: Du ir kt. [Dlt+23] MAD nauda matematinio bei strateginio samprotavimo užduotyse, Khan ir kt. [Khv+24] +28 procentinių punktų teksto supratimo užduotyse, Jiang ir kt. [Jdw+25] iki 25,4% santykinis Pass@1 pagerėjimas vidinio savarankiško planavimo metodu. Vis dėlto šie rezultatai pasiekti unikaloje aplinkoje - atvirųjų svorių 7-9 milijardų parametrų modeliai daugiafailio kodo generavimo užduotyse - kuri ankstesniuose darbuose buvo menkai ištirta empiriškai. Pagrindinis šio darbo indėlis į kelių agentų debatų lauką yra empirinis Smit ir kt. [Sdg+24] iškeltos sąlygos „atidžiai sureguliuotos MAD sistemos pranoksta paprastesnes strategijas” patikrinimas mažesnių modelių aplinkoje su konkrečiais identifikuotais sistemos elementais (anti-regresijos politika, adaptyvi temperatūra, kritikos istorija, heterogeniškas pasiūlytojų rinkinys), prisidedančiais prie metodo efektyvumo.

Pasiūlymai:

Remiantis šio darbo rezultatais, galima suformuluoti pasiūlymus, kurie tinka skirtingoms auditorijoms, pavyzdžiui, sistemų taikytojams ir sistemų kūrėjams.

1. Pradinę debatų konfigūraciją rekomenduojama sudaryti iš pasiūlytojų be teisėjo, ypač naudojant 7B klasės modelius - šio darbo duomenys rodo, kad tai ekonomiškai efektyviausias režimas, kurio rezultatas tasks2/ rinkinyje (0,776) nesiskiria nuo režimų su teisėju (atitinkamai 0,771 su kodo teisėju ir 0,771 su samprotavimo teisėju), tačiau skaičiavimo sąnaudos yra mažiausios.
2. Lengvojo ir vidutinio sudėtingumo užduotims taikyti solo režimą, o ne kelių agentų debatus - šiame sudėtingumo diapazone individualus modelis dažnai jau pasiekia aukštą testų išlaikymo rodiklį, todėl debatai prideda mažai pridėtinės vertės (vidutinio lygio užduotyse prieaugis siekia +18,2 procentinio punkto, tačiau solo rezultatas jau viršija 0,80), o skaičiavimo sąnaudos yra apie 15-40 kartų didesnės.
3. Sudėtingojo ir ekstremaliojo lygio užduotims rekomenduojama taikyti kelių agentų debatų metodą - šio darbo 90,1% gelbėjimo rodiklis užduotyse, kuriose visi trys pasiūlytojai pirmajame raunde nepateikia visiškai veikiančio sprendimo, suteikia ryškią pridėtinę vertę, kuri solo režimu nepasiekiamą.
4. Formuojant pasiūlytojų rinkinį, teikti pirmenybę architektūriškai įvairesniam modelių derinimui, o ne vien mechaniškai rinkti tris stipriausius modelius pagal solo rezultatus - šio darbo 4.3 poskyrio rezultatai rodo, kad rinkinio įvairumas yra mažiausiai tiek pat

- svarbus veiksnys kaip ir atskirų modelių dydis, o stipriausių modelių derinys gali skatinti per ankstyvą sprendimų konsolidaciją ties lokaliuoju optimumu.
5. Kodo specializacijos teisėjo („DeepSeek-Coder-V2” ar analogiško) įtraukimą vertinti su atsargumu - stipresnių pasiūlytojų rinkinio (Pool B) atveju jis šio darbo eksperimentuose pablogino rezultatą sudėtingiausiose užduotyse 3,9 procentinio punkto. Rekomenduojama prieš diegimą atlikti porinį palyginimo bandymą (A/B testavimą) su projektui reprezentatyviomis užduotimis ir įjungti kodo teisėją tik tada, kai gauti rezultatai rodo, kad jis statistškai pranoksta režimą be teisėjo konkrečioje aplinkoje su konkrečiu pasiūlytojų rinkiniu.
 6. Samprotavimo specializacijos teisėją („DeepSeek-R1” ar analogišką) rekomenduojama taikyti tik tose specifinėse situacijose, kai būtinas išsamus kritikos pagrindimas argumentacijos forma, o ne pats galutinis testų išlaikymo rodiklis - šio darbo duomenyse šis režimas yra brangiausia konfigūracijos modifikacija (+55-58% trukmės), kurios nauda kodo generavimo užduotyse pasirodė labai ribota.
 7. Įgyvendinti anti-regresijos politiką su geriausio istorinio sprendimo grąžinimu - be šio mechanizmo negalima garantuoti, kad atskirų debatų rezultatai nepablogės, palyginti su geriausiu jau pasiektu sprendimu. Šio darbo preliminariniuose bandymuose buvo pastebėta, kad 7B klasės modeliai kartais po sėkmingo pirmojo raundo vėlesniame raunde grąžina prastesnį kodą; pagrindiniame etape įdiegta anti-regresijos politika užtikrino 0 regresijų iš 600 vykdymų pagal funkcinio testų išlaikymo metriką.
 8. Konteksto lango dydžiui (parametras „num_ctx”) parinkti bent 32 768 žetonus 7-9 milijardų parametru modeliams, ypač daugiafailio generavimo scenarijuose - numatytasis „Ollama” dydis („num_ctx = 2048”) yra nepakankamas ir lemia nepastebimą atsakymų sutrumpinimą (angl. silent truncation), iškreipiantį visus debatų protokolus. Šis nustatymas turi būti papildomas sutrumpinimo aptikimo mechanizmu, registruojančiu kiekvieną tokį atvejį.
 9. Daugiafailių užduočių atveju taikyti dalinį (angl. chunked) generavimą su atskira užklausa kiekvienam failui, perduodant jau sugeneruotų failų kodą kaip kontekstą - tai sumažina konteksto lango išnaudojimo riziką ir leidžia 7-9 milijardų parametru modeliams atlikti realistiškas dviejų ir daugiau modulių programų inžinerijos užduotis.

Rezieme (Summary)

Large language models are reshaping software development, but the strongest systems still rely on closed-source commercial models such as GPT-4 or Claude. Open-weight 7-9 billion parameter models that can be run locally are an attractive alternative, yet on complex code generation tasks they often produce incorrect, incomplete or poorly structured code. Multi-agent debate (MAD), in which several model instances iteratively propose, critique and revise each other's solutions, is a promising technique for closing this gap, but its behaviour with smaller open-weight models on multi-file code generation tasks has been largely uninvestigated.

This work designs, implements and empirically evaluates a modular four-phase multi-agent debate system tailored to 7-9 billion parameter models. The system (about 27 thousand lines in total: 18 thousand of Python code, 4.3 thousand of web template, 1.3 thousand of HPC scripts and 3 thousand of JSON task descriptions in six layered packages) integrates an anti-regression policy, adaptive temperature, critique history, chunked multi-file generation and three solution-visibility levels suited to the limited context window of small models. A custom benchmark of 90 unique tasks across four difficulty levels was developed, including 40 multi-file extreme-level tasks specifically designed for this work. Eleven SLURM experiments on the VU MIF high-performance computing cluster produced 960 execution records (600 debates and 360 solo runs) across three comparison axes: proposer pool composition (7B vs 9B class), judge type (none, code-specialised, reasoning-specialised) and debate vs solo. In addition, a separate controlled A/B ablation experiment was conducted for the adaptive temperature mechanism: four additional SLURM jobs produced 240 paired ON/OFF comparisons, in which the OFF runs were compared with the corresponding ON runs from the main experimental stage. This ablation confirmed the mechanism's added value, with a pooled effect of +3.99 percentage points ($p = 0.010$) and the strongest signal in the Pool B no-judge configuration on extreme-difficulty tasks (+10.6 percentage points, $p = 0.029$ before multiple-comparison correction). In total, the experimental base of this work comprises 15 SLURM jobs and 1200 unique execution records.

The main findings are as follows. Multi-agent debate stably outperformed the single-model baseline by 27.3 percentage points (0.824 vs 0.551 average pass rate, paired Student's t-test $t = 17.27$, $p < 10^{-34}$; 108 wins, 12 ties and 0 losses across 120 task-set pairs), with a 90.1% rescue rate on initially failed cases. The hypothesis that a 9B pool would outperform a 7B pool was not confirmed; on the hardest tasks the architecturally diverse 7B pool even surpassed the more homogeneous 9B one, suggesting that pool diversity matters at least as much as raw parameter count. The judge benefit reported by Khan et al. in textual debates (+28 percentage points) did not transfer to code: a code-specialised judge worsened the result by 3.9 percentage points on the strongest pool. A bug detection-and-correction “funnel” analysis showed that adding a judge increased reported bugs by 30-100% while the fix ratio dropped from 5.2% to 2.1%, and that the strongest predictor of improvement was the number of corrected bugs (Pearson $r = +0.446$, $n = 600$, $p < 10^{-30}$), not the volume of criticism. A reasoning-specialised judge added 55-58% computation time without commensurate gains, static code quality (Pylint, Radon) was preserved within 4%, and the anti-regression policy ensured zero regressions across all 600 runs.

The work empirically confirms that carefully tuned multi-agent debate, paired with anti-regression mechanisms and heterogeneous proposers, can meaningfully boost small open-weight models on multi-file engineering tasks. It also shows that the “LLM-as-Judge” paradigm does not transfer unchanged from text to code synthesis - the limiting factor is the transformation of criticism into concrete code edits, not the volume of criticism produced.

Keywords: multi-agent debate, large language models, open-weight models, code generation, LLM-as-Judge, multi-file programming tasks.

Literatūros sąrašas

[Ant24] Anthropic. (2024). Claude 3 Model Family. Prieinama: <https://www.anthropic.com/news/claude-3-family>.

[Ccs+23] Chan, C.-M., Chen, W., Su, Y., Yu, J., Xue, W., Zhang, S., Fu, J., & Liu, Z. (2023). ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate. arXiv preprint arXiv:2308.07201.

[Ctj+21] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., ... & Zaremba, W. (2021). Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374.

[Dee24] DeepSeek-AI. (2024). DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence. arXiv preprint arXiv:2406.11931.

[Dee25] DeepSeek-AI. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv preprint arXiv:2501.12948.

[Dlt+23] Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2023). Improving Factuality and Reasoning in Language Models through Multiagent Debate. arXiv preprint arXiv:2305.14325.

[Dst24] Dillmann, M., Siebert, J., & Trendowicz, A. (2024). Evaluation of large language models for assessing code maintainability. arXiv preprint arXiv:2401.12714.

[Gg24] Gao, Y., & GitHub Customer Research. (2024). Research: Quantifying GitHub Copilot's impact in the enterprise with Accenture. GitHub Blog. Prieinama: <https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-in-the-enterprise-with-accenture/>.

[Gzy+24] Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., ... & Liang, W. (2024). DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence. arXiv preprint arXiv:2401.14196.

[Hyc+24] Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., ... & Lin, J. (2024). Qwen2.5-Coder Technical Report. arXiv preprint arXiv:2409.12186.

[Hzc+24] Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., & Schmidhuber, J. (2024). MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. arXiv preprint arXiv:2308.00352.

[Hzl+24] Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2024). Large Language Models for Software Engineering: A Systematic Literature Review. ACM Transactions on Software Engineering and Methodology. arXiv preprint arXiv:2308.10620.

[Jdw+25] Jiang, X., Dong, Y., Wang, L., Fang, Z., Shang, Q., Li, G., Jin, Z., & Jiao, W. (2025). Self-planning Code Generation with Large Language Models. arXiv preprint arXiv:2303.06689.

- [Jsm+23] Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., ... & Sayed, W. E. (2023). Mistral 7B. arXiv preprint arXiv:2310.06825.
- [Jyw+24] Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv preprint arXiv:2310.06770.
- [Khv+24] Khan, A., Hughes, J., Valentine, D., Ruis, L., Sachan, K., Radhakrishnan, A., Grefenstette, E., Bowman, S. R., Rocktäschel, T., & Perez, E. (2024). Debating with More Persuasive LLMs Leads to More Truthful Answers. arXiv preprint arXiv:2402.06782.
- [Lhj+23] Liang, T., He, Z., Jiao, W., Wang, X., Wang, Y., Wang, R., Yang, Y., Tu, Z., & Shi, S. (2023). Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate. arXiv preprint arXiv:2305.19118.
- [Lxw+23] Liu, J., Xia, C. S., Wang, Y., & Zhang, L. (2023). Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. arXiv preprint arXiv:2305.01210.
- [Ms2+24] Mishra, M., Stallone, M., Zhang, G., Shen, Y., Prasad, A., Soria, A. M., ... & Panda, R. (2024). Granite Code Models: A Family of Open Foundation Models for Code Intelligence. arXiv preprint arXiv:2405.04324.
- [Mtg+23] Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., ... & Clark, P. (2023). Self-Refine: Iterative Refinement with Self-Feedback. arXiv preprint arXiv:2303.17651.
- [Ope23] OpenAI. (2023). GPT-4 Technical Report. arXiv preprint arXiv:2303.08774.
- [Pkc+23] Peng, S., Kalliamvakou, E., Cihon, P., & Demirel, M. (2023). The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv preprint arXiv:2302.06590.
- [Qll+23] Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., & Sun, M. (2023). ChatDev: Communicative Agents for Software Development. arXiv preprint arXiv:2307.07924.
- [Rgg+23] Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., ... & Synnaeve, G. (2023). Code Llama: Open Foundation Models for Code. arXiv preprint arXiv:2308.12950.
- [Scb+23] Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv preprint arXiv:2303.11366.
- [Sdg+24] Smit, A. P., Duckworth, P., Grinsztajn, N., Barrett, T. D., & Pretorius, A. (2024). Should we be going MAD? A Look at Multi-Agent Debate Strategies for LLMs. arXiv preprint arXiv:2311.17371.
- [Vsp+17] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L.,

Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30* (NeurIPS 2017). arXiv preprint arXiv:1706.03762.

[Wbz+23] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv preprint arXiv:2308.08155.

[Wws+22a] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv preprint arXiv:2201.11903.

[Wws+22b] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. (2022). Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv preprint arXiv:2203.11171.

[Ycl+24] Young, A., Chen, B., Li, C., Huang, C., Zhang, G., Zhang, G., Wang, G., Li, H., Zhu, J., Chen, J., ... & Dai, Z. (2024). Yi: Open Foundation Models by 01.AI. arXiv preprint arXiv:2403.04652.

[Yyz+25] Yang, A., Yang, B., Zhang, B., et al. (2025). Qwen2.5 Technical Report. arXiv preprint arXiv:2412.15115.

[Zcm24] Zheng, J., Cao, B., & Ma, Z. (2024). Beyond Correctness: Benchmarking Multi-dimensional Code Generation for Large Language Models. arXiv preprint arXiv:2407.11470.

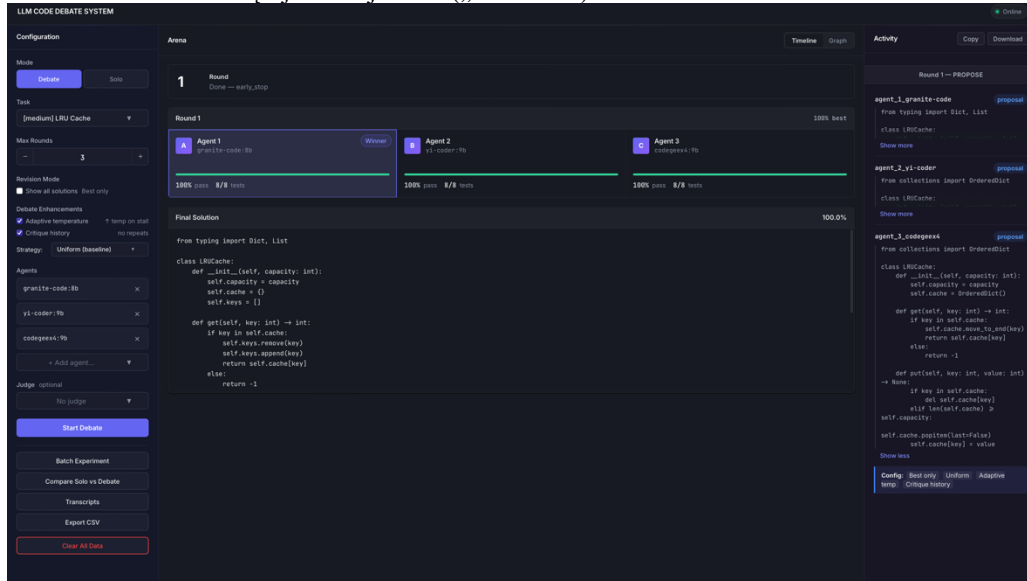
[Zcs+23] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv preprint arXiv:2306.05685.

[Zxz+23] Zheng, Q., Xia, X., Zou, X., Dong, Y., Wang, S., Xue, Y., Wang, Z., Shen, L., Wang, A., Li, Y., Su, T., Yang, Z., & Tang, J. (2023). CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Benchmarking on HumanEval-X. arXiv preprint arXiv:2303.17568.

Priedai

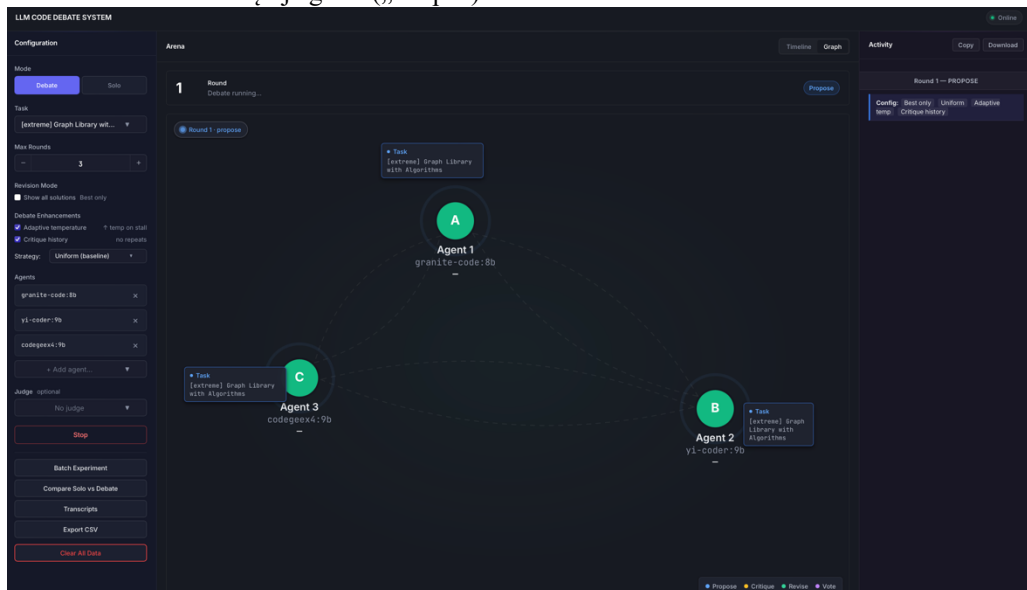
Šio darbo metu sukurti komponentai - kelių agentų debatų sistema, jos vienetinių testų rinkinys, didelio našumo skaičiavimo (HPC) paleidimo scenarijai ir užduočių rinkiniai, yra viešai prieinami „GitHub“ saugykloje: GitHub: <https://github.com/lukamaga/llm-code-debate>

Sistemos žiniatinklio sąsaja laiko juostos („Timeline“) režimu:



1 ekrano kopija. Timeline view

Sistemos žiniatinklio sąsaja grafo („Graph“) režimu:



2 ekrano kopija. Graph view

Dirbtinio intelekto įrankių naudojimo deklaracija:

Rengiant šį darbą generatyvinio dirbtinio intelekto įrankiai buvo naudojami pagalbinei kalbos redagavimo, gramatinių klaidų paieškos ir taisymo bei teksto loginio nuoseklumo patikros funkcijai. Tyrimo idėja, sistemos architektūra, programinis kodas, eksperimentų vykdymas, rezultatų analizė ir išvados parengti autoriaus savarankiškai.